

COMPUTATIONAL PHYSICS

Oscillatory Systems — Python Simulation Study

Simple Harmonic Motion: Comprehensive Analysis and Animation

A Python-based comprehensive study of pendulum and mass-spring-damper oscillations, combining analytical theory, 4th-order Runge-Kutta integration, phase-space portraits, energy evolution, and animated visualization.

Simple Pendulum • Mass-Spring-Damper System • RK4 Numerical Integration

AUTHOR

Puspendu Rana

 github.com/PuspenduPH

DATE

March 8, 2026

SOURCE CODE



[PuspenduPH/SimpleHarmonicMotion-Simulation](https://github.com/PuspenduPH/SimpleHarmonicMotion-Simulation)

Table of Contents

1. Overview	1
2. Key Features	1
3. Importing Necessary Libraries	1
How to Use This Notebook	1
4. Simple Harmonic Motion of a Pendulum	2
4.1 SHM of an Undamped Pendulum	2
Derivation of the Non-Linear Period of a Pendulum under SHM	2
Period Calculation Function	3
Pendulum Animation Code	4
4.2 Undamped SHM with Animation and Plots	9
4.3 Damped Pendulum Motion	16
Runge-Kutta Method for Solving Differential Equations	16
Equation of Motion	17
Equation of Motion	17
System of First-Order ODEs	17
RK4 Implementation	18
Damped vs Undamped Comparison Animation	19
4.4 Damped Pendulum Motion with Phase and Time Domain Plots	26
5. Mass-Spring System with Damping	35
5.1 Theory and RK4 Implementation	35
1. Physical System Overview	35
2. Equation of Motion (Newton's Second Law)	36
3. Converting to First-Order System	36
4. RK4 Implementation for Mass-Spring-Damper	36
5. Damping Classification	37
6. Underdamped Response (Most Common)	37
7. Complete RK4 Integration	38
8. Forced Oscillations	38
9. Comparison Table: Key Parameters	38
5.2 Animating the Damped Mass-Spring System	41
5.3 Animating for Multiple Damping Cases	47
5.4 Animation of Damped vs Undamped SHM with Plots	53
5.5 Phase Space Evolution of the Mass-Spring-Damper System	62
5.6 Energy Evolution of the Mass-Spring-Damper System	67
5.7 Damped Driven Mass-Spring Systems	72
5.8 Resonance and Frequency Response	84
5.9 Phase Space and Energy Analysis	95
6. Conclusion	104
7. References	105

1. Overview

This notebook provides a comprehensive exploration of **Simple Harmonic Motion (SHM)** through theory, numerical simulation, and visualization. We investigate two core oscillatory systems:

1. **Simple Pendulum** — small-angle SHM and **large-amplitude** motion, where the period increases beyond $T_0 = 2\pi\sqrt{L/g}$ and is computed accurately using **elliptic integrals** (via `ellipk`).
2. **Mass–Spring–Damper System** — free oscillations across damping regimes (undamped, underdamped, critically damped, overdamped) and **driven damped motion** under sinusoidal forcing.

We combine analytical results where possible (e.g., undamped solutions and steady-state response concepts) with **4th-order Runge–Kutta (RK4)** integration for general damped/forced dynamics. The notebook emphasizes physical interpretation using **time-domain plots**, **phase-space portraits** (x, v), and **energy diagnostics**: - conservative motion (closed loops, constant energy), - free damped decay (inward spirals, monotonic energy loss), - driven steady state (approach to limit cycles, bounded time-varying energy).

Finally, we study **resonance and frequency response** for driven systems, highlighting that: - **displacement (amplitude) resonance** peaks at a damping-dependent frequency $f_r = f_0\sqrt{1 - 2\zeta^2}$ (only if $\zeta < 1/\sqrt{2}$), - while **velocity** and **power (energy-transfer)** peaks occur at the **natural frequency** f_0 for all damping cases.

2. Key Features

- **Exact Non-Linear Period Calculation** using complete elliptic integrals of the first kind
- **Numerical Integration** with 4th-order Runge-Kutta method for accurate time evolution
- **Real-Time Animations** showing physical motion with energy, phase space, and time-domain plots
- **Comparative Analysis** of different damping regimes (undamped, underdamped, critically damped, overdamped)
- **Interactive Parameters** allowing customization of initial conditions, system properties, and simulation settings

3. Importing Necessary Libraries

We begin by importing the essential Python libraries for numerical computation, visualization, and animation:

- **NumPy**: For numerical arrays and mathematical operations
- **Matplotlib**: For creating plots and animations
- **SciPy**: For special functions (elliptic integrals)

The `%matplotlib ipynb` magic command enables interactive plotting within the notebook.

```
[ ]: import numpy as np
import matplotlib.pyplot as plt
from matplotlib.animation import FuncAnimation
import os
from matplotlib.patches import Rectangle
from scipy.special import ellipk
from matplotlib.gridspec import GridSpec

%matplotlib ipynb
```

How to Use This Notebook

1. **Run all cells sequentially**: Execute cells from top to bottom to define all functions
2. **Interactive animations**: Each animation function can be called directly or through the `main()` function
3. **Customize parameters**: Modify function arguments to explore different physical scenarios:
 - Pendulum length, initial angle, damping coefficient
 - Mass, spring constant, damping coefficient
 - Number of oscillations, frame rate
4. **Save animations**: Set `save_animation=True` to export animations as GIF files
5. **Experiment**: Try different initial conditions and compare behaviors!

4. Simple Harmonic Motion of a Pendulum

4.1 SHM of an Undamped Pendulum

Derivation of the Non-Linear Period of a Pendulum under SHM

We know that the time period of a simple pendulum undergoing small oscillations is given by:

$$T_0 = 2\pi\sqrt{\frac{l}{g}}$$

where l is the length of the pendulum and g is the acceleration due to gravity. However, for larger amplitudes, the period deviates from this simple formula. The exact period T for a pendulum with maximum angular displacement θ_0 can be expressed as:

$$T = \frac{2}{\pi} T_0 K\left(\sin^2 \frac{\theta_0}{2}\right)$$

where $K(k)$ is the complete elliptic integral of the first kind, defined as:

$$K(k) = \int_0^{\pi/2} \frac{d\phi}{\sqrt{1 - k^2 \sin^2 \phi}}$$

and $T_0 = 2\pi\sqrt{\frac{l}{g}}$ is the period for small oscillations.

Below is the derivation of the formula for the period of a pendulum undergoing simple harmonic motion (SHM) with larger amplitudes:

We start with the equation of motion for a simple pendulum:

1. Equation of Motion For a simple pendulum of length L and mass m , the restoring torque is $\tau = -mgL \sin \theta$. Using Newton's second law for rotation ($\tau = I\alpha$):

$$\begin{aligned} mL^2 \frac{d^2\theta}{dt^2} &= -mgL \sin \theta \\ \implies \frac{d^2\theta}{dt^2} + \frac{g}{L} \sin \theta &= 0 \end{aligned}$$

2. Conservation of Energy (First Integral) Multiplying both sides by $2\frac{d\theta}{dt}$ and integrating with respect to time:

$$\begin{aligned} \frac{d}{dt} \left(\frac{d\theta}{dt} \right)^2 &= 2 \frac{g}{L} \frac{d}{dt} (\cos \theta) \\ \left(\frac{d\theta}{dt} \right)^2 &= \frac{2g}{L} \cos \theta + C \end{aligned}$$

At the maximum amplitude $\theta = \theta_0$, the velocity is zero ($\frac{d\theta}{dt} = 0$). Thus, $C = -\frac{2g}{L} \cos \theta_0$.

$$\begin{aligned} \left(\frac{d\theta}{dt} \right)^2 &= \frac{2g}{L} (\cos \theta - \cos \theta_0) \\ \frac{d\theta}{dt} &= \sqrt{\frac{2g}{L} (\cos \theta - \cos \theta_0)} \end{aligned}$$

3. Integral for Time Period Rearranging for dt and integrating over a quarter period (from $\theta = 0$ to $\theta = \theta_0$):

$$dt = \sqrt{\frac{L}{2g}} \frac{d\theta}{\sqrt{\cos \theta - \cos \theta_0}}$$

Therefore, the total period T is four times this integral:

$$T = 4 \int_0^{\theta_0} dt = 4 \sqrt{\frac{L}{2g}} \int_0^{\theta_0} \frac{d\theta}{\sqrt{\cos \theta - \cos \theta_0}}$$

4. Half-Angle Substitution Using the identity $\cos \theta = 1 - 2 \sin^2(\theta/2)$:

$$\cos \theta - \cos \theta_0 = 2(\sin^2(\theta_0/2) - \sin^2(\theta/2))$$

Substituting this back into the integral:

$$\begin{aligned} T &= 4\sqrt{\frac{L}{2g}} \int_0^{\theta_0} \frac{d\theta}{\sqrt{2(\sin^2(\theta_0/2) - \sin^2(\theta/2))}} \\ &= 2\sqrt{\frac{L}{g}} \int_0^{\theta_0} \frac{d\theta}{\sqrt{\sin^2(\theta_0/2) - \sin^2(\theta/2)}} \end{aligned}$$

5. Elliptic Integral Form Let $k = \sin(\theta_0/2)$. We introduce a change of variables $\sin(\theta/2) = k \sin \phi$. Differentiating gives $d\theta = \frac{2k \cos \phi}{\sqrt{1-k^2 \sin^2 \phi}} d\phi$. The limits change: $\theta : 0 \rightarrow \theta_0$ becomes $\phi : 0 \rightarrow \pi/2$.

$$\begin{aligned} T &= 2\sqrt{\frac{L}{g}} \int_0^{\pi/2} \frac{1}{k \cos \phi} \cdot \frac{2k \cos \phi}{\sqrt{1-k^2 \sin^2 \phi}} d\phi \\ &= 4\sqrt{\frac{L}{g}} \int_0^{\pi/2} \frac{d\phi}{\sqrt{1-k^2 \sin^2 \phi}} \end{aligned}$$

The integral is the **Complete Elliptic Integral of the First Kind**, $K(k^2)$.

Final Expression:

$$T = 4\sqrt{\frac{L}{g}} K\left(\sin^2 \frac{\theta_0}{2}\right)$$

which can be expressed in terms of T_0 as:

$$T = \frac{2}{\pi} T_0 K\left(\sin^2 \frac{\theta_0}{2}\right)$$

In the code cell below we utilized the ‘elli

Period Calculation Function

The following function implements the exact period calculation for a pendulum with arbitrary initial amplitude using the complete elliptic integral of the first kind $K(k)$.

```
[2]: def calculate_nonlinear_period(theta_0, L, G):
    """
    Calculate the actual period of a nonlinear pendulum for large angles.
    Uses elliptic integral of the first kind for exact calculation.

    For small angles:  $T \approx 2\pi\sqrt{L/g}$ 
    For large angles:  $T = 4\sqrt{L/g} * K(k)$ 
    where  $K$  is the complete elliptic integral and  $k = \sin(\theta_0/2)$ 

    Parameters:
    -----
    theta_0 : float
        Initial angle in radians
    L : float
        Length of pendulum
    G : float
        Gravitational acceleration

    Returns:
    -----
    T : float
        Actual period for the given amplitude
    """
    # Small angle period (linear approximation)
    T_0 = 2 * np.pi * np.sqrt(L / G)

    # For large angles, use elliptic integral correction
```

```

# The period is  $T = T_0 * (2/\pi) * K(k^2)$  where  $k = \sin(\theta_0/2)$ 
k = np.sin(theta_0 / 2) # Modulus
K_k = ellipk(k**2) # Complete elliptic integral of the first kind

# Actual period accounting for nonlinearity
T_actual = T_0 * (2 / np.pi) * K_k

return T_actual

```

Pendulum Animation Code

This function creates an animated visualization of a simple pendulum undergoing SHM. The animation includes:

- Real-time position tracking of the pendulum bob
- Trail visualization showing recent positions
- Display of physical parameters and oscillation count
- Option to save the animation as a GIF file

In the code shell below we animated the motion of a simple pendulum undergoing SHM using the derived period formula. The code for animation is briefly summarized as follows:

- How it's animated :
 - The code computes a period T (calls `calculate_nonlinear_period`) and builds a time array `t_array` for the requested number of revolutions and fps.
 - A simple oscillatory angular law is used: the code sets an effective angular frequency `omega_actual` = $2\pi / T$ and computes

$$\theta(t) = \theta_0 \cos(\omega_{\text{actual}} t).$$

- Cartesian bob coordinates are then

$$x(t) = L \sin(\theta(t)), \quad y(t) = -L \cos(\theta(t)).$$

- Matplotlib creates the plot elements (pivot, string line, bob scatter, trail, info text) and `FuncAnimation` calls `animate_frame` for each frame to update the line, bob position, trail (last 25 points), on-screen counters and title.
- The animation is either shown with `plt.show()` or saved (attempts `anim.save(...)` into OUTPUTS/ANIMATIONS).
- Key physical properties & gotchas:
 - Small-angle period used for reference:

$$T_{\text{linear}} = 2\pi \sqrt{\frac{L}{G}}.$$

- The code obtains an actual (nonlinear) period via `calculate_nonlinear_period` and reports the period ratio T/T_{linear} (period increases with amplitude).
- The simulation is ideal: no damping or driving forces included, so energy is effectively conserved.
- Important approximation: the angular motion uses a cosine form (linear solution) but with the corrected period. For large amplitudes this is only an approximation; a full nonlinear solution requires solving the ODE (e.g. numerical integration with `solve_ivp`) or using elliptic-integral expressions for exact phase evolution.
- Saving: the code uses `ffmpeg` writer with `codec="gif"` which can sometimes fail depending on installed encoders—be prepared to switch writer/codec or install `ffmpeg`.

```

[ ]: def animate_pendulum(
    L=1.0,
    G=9.81,
    theta_0_deg=20.0,
    num_revolutions=3,
    fps=30,
    save_animation=False,
    filename=None,
):
    """
    Animate a simple pendulum with oscillation counter and time display.

    Parameters:
    -----
    L : float, default=1.0
        Length of pendulum in meters
    G : float, default=9.81

```

```

    Acceleration due to gravity in m/s^2
theta_0_deg : float, default=20.0
    Initial angle in degrees
num_revolutions : int, default=3
    Number of complete oscillations to simulate
fps : int, default=30
    Frames per second for animation
save_animation : bool, default=False
    Whether to save animation as GIF
filename : str or None, default=None
    Filename for saved GIF
"""

# Convert initial angle to radians
theta_0 = np.radians(theta_0_deg)

# Pendulum parameters
omega_linear = np.sqrt(G / L) # Natural frequency (small angle)
T_linear = 2 * np.pi / omega_linear # Period (small-angle approximation)

# Calculate actual nonlinear period
T = calculate_nonlinear_period(theta_0, L, G)
total_time = num_revolutions * T

# The effective omega based on the ACTUAL period
omega_actual = 2 * np.pi / T

# Period ratio for information
period_ratio = T / T_linear

# Time array for animation
num_frames = int(total_time * fps) + 1
t_array = np.linspace(0, total_time, num_frames)

def calculate_pendulum_position(t, L=L, theta_0=theta_0, omega=omega_actual):
    """Calculate the position of the pendulum bob at time t."""
    theta_t = theta_0 * np.cos(omega * t) # Angular position
    x = L * np.sin(theta_t) # Horizontal position
    y = -L * np.cos(theta_t) # Vertical position (negative downwards)
    return x, y, theta_t

# Setting up the figure and axis
fig, ax = plt.subplots(figsize=(8, 12))
fig.subplots_adjust(top=0.9, bottom=0.05, left=0.1, right=0.9)

max_x = L * np.sin(theta_0) * 1.2
ax.set_xlim(-max_x, max_x)
ax.set_ylim(-L * 1.2, 0.3)
ax.set_xlabel("Horizontal Position (m)", fontsize=12)
ax.set_ylabel("Vertical Position (m)", fontsize=12)

# Pendulum elements
# Pivot point (fixed)
pivot_rect = Rectangle(
    (-max_x / 4, -max_x / 16), max_x / 2, max_x / 8, color="black", zorder=10
)
ax.add_patch(pivot_rect)

# Pendulum string (will be updated)
(pendulum_line,) = ax.plot([], [], "b-", linewidth=3, alpha=0.8, label="String")

# Pendulum bob (will be updated)
bob_scatter = ax.scatter(

```

```

    [], [], s=500, c="red", edgecolor="darkred", linewidth=2, label="Bob", zorder=5
)
# Trail of the bob (will be updated)
trail_x, trail_y = [], []
(trail_line,) = ax.plot([], [], "r-", alpha=0.3, linewidth=1, label="Trail")

ax.axhline(0, color="gray", linestyle="--", alpha=0.5)
ax.axvline(0, color="gray", linestyle="--", alpha=0.5)

# Text displays for counters and information
info_text = ax.text(
    0.02,
    0.98,
    "",
    transform=ax.transAxes,
    fontsize=10,
    verticalalignment="top",
    bbox=dict(boxstyle="round,pad=0.5", facecolor="lightblue", alpha=0.8),
)

physics_text = ax.text(
    0.02,
    0.01,
    "",
    transform=ax.transAxes,
    fontsize=10,
    verticalalignment="bottom",
    bbox=dict(boxstyle="round,pad=0.3", facecolor="yellow", alpha=0.4),
)

# Physics info
physics_info = (
    f"Length: {L:.2f} m\n"
    f"Initial angle: {theta_0_deg:.1f}°\n"
    f"Period (linear): {T_linear:.3f} s\n"
    f"Period (actual): {T:.3f} s\n"
    f"Period ratio: {period_ratio:.4f}\n"
    f"Frequency: {omega_actual / (2 * np.pi):.3f} Hz"
)
physics_text.set_text(physics_info)

def animate_frame(frame):
    """Animation function called for each frame."""
    t_current = t_array[frame]

    # Calculates current position
    x_bob, y_bob, theta_current = calculate_pendulum_position(t_current)

    # Updates pendulum string (from pivot to bob)
    pendulum_line.set_data([0, x_bob], [0, y_bob])

    # Updates bob position
    bob_scatter.set_offsets([[x_bob, y_bob]])

    # Updates trail (keep last 25 points for smooth trail)
    trail_x.append(x_bob)
    trail_y.append(y_bob)
    if len(trail_x) > 25:
        trail_x.pop(0)
        trail_y.pop(0)
    trail_line.set_data(trail_x, trail_y)

    # Calculates number of completed oscillations

```

```

completed_oscillations = t_current / T
progress_pct = (frame / (num_frames - 1)) * 100 if num_frames > 1 else 100.0

# Updates information display
info_display = (
    f"Time: {t_current:.2f} s\n"
    f"Completed oscillations: {completed_oscillations:.2f}\n"
    f"Current angle: {np.degrees(theta_current):+.1f}°\n"
    f"Progress: {progress_pct:.1f}%"
)
info_text.set_text(info_display)

# Updates title with current status
ax.set_title(
    "Simple Harmonic Motion of a Pendulum\n"
    f"Oscillation: {completed_oscillations:.2f}/{num_revolutions}",
    fontsize=16,
    pad=15,
)

return pendulum_line, bob_scatter, trail_line, info_text

# Adds legend
ax.legend(
    loc="upper right",
    fancybox=True,
    edgecolor="gray",
    fontsize=10,
    labelspacing=1.2,
)

# Creates animation
anim = FuncAnimation(
    fig,
    animate_frame,
    frames=num_frames,
    interval=int(1000 / fps),
    blit=False,
    repeat=False,
)

# Saves animation if requested
if save_animation:
    if filename is None:
        filename = (
            f"pendulum_L{L}_theta{theta_0_deg:.0f}deg_{num_revolutions}rev.gif"
        )

    save_dir = "OUTPUTS/ANIMATIONS"
    os.makedirs(save_dir, exist_ok=True)
    filepath = os.path.join(save_dir, filename)

    try:
        print("Saving pendulum animation... This may take a moment.")
        anim.save(filepath, writer="ffmpeg", fps=fps, codec="gif", dpi=150)
        print(f"Animation saved as: {filepath}")
    except Exception as e:
        print(f"Error saving animation: {e}")
    plt.close(fig)
else:
    plt.show()

print("\nPendulum Simulation Summary:")

```

```

print(f"Length: {L} m")
print(f"Initial angle: {theta_0_deg:.1f}°")
print(f"Period (small-angle approx): {T_linear:.3f} s")
print(f"Period (nonlinear, actual): {T:.3f} s")
print(f"Period increase: {100 * (period_ratio - 1):.2f}%")
print(f"Total simulation time: {total_time:.2f} s")
print(f"Number of oscillations: {num_revolutions}")

return anim

def main():
    """Main function to run the pendulum animation with user inputs."""

    print("Simple Pendulum Animation Simulator")
    print("=====")

    # Default parameters
    defaults = {
        "L": 1.0,
        "G": 9.81,
        "theta_0_deg": 20.0,
        "num_revolutions": 3,
        "fps": 30,
    }

    use_defaults = input("Use default parameters? (y/n): ").strip().lower()
    if use_defaults == "y":
        params = defaults.copy()
    else:
        params = {}
        try:
            params["L"] = float(
                input(f"Enter length of pendulum in meters [{defaults['L']}]: ")
                or defaults["L"]
            )
            params["G"] = float(
                input(f"Enter gravitational acceleration in m/s² [{defaults['G']}]: ")
                or defaults["G"]
            )
            params["theta_0_deg"] = float(
                input(f"Enter initial angle in degrees [{defaults['theta_0_deg']}]: ")
                or defaults["theta_0_deg"]
            )
            params["num_revolutions"] = int(
                input(
                    f"Enter number of oscillations to simulate ↵
↳ [{defaults['num_revolutions']}]: "
                )
                or defaults["num_revolutions"]
            )
            params["fps"] = int(
                input(f"Enter frames per second [{defaults['fps']}]: ")
                or defaults["fps"]
            )
        except ValueError:
            print("Invalid input, using default parameters.")
            params = defaults.copy()

    save_anim = input("Save animation as GIF? (y/n): ").strip().lower() == "y"
    filename = None
    if save_anim:
        filename = input("Enter filename (or press Enter for auto-generated): ").strip()

```

```

    if not filename.endswith(".gif") and filename != "":
        filename += ".gif"
    if not filename:
        filename = None

print("\nRunning simulation with parameters:")
for key, value in params.items():
    print(f"  {key}: {value}")

animation = animate_pendulum(
    L=params["L"],
    G=params["G"],
    theta_0_deg=params["theta_0_deg"],
    num_revolutions=params["num_revolutions"],
    fps=params["fps"],
    save_animation=save_anim,
    filename=filename,
)

return animation

if __name__ == "__main__":
    anim = main()

```

See the animation in the GitHub repository: [Animation 1](#)

4.2 Undamped SHM with Animation and Plots

This section extends the basic pendulum animation by adding real-time plots showing: - **Angular position vs time**: The oscillatory motion in the time domain - **Angular velocity vs time**: The rate of change of angular displacement - **Phase space diagram**: A plot of velocity versus position, forming a characteristic ellipse for conservative systems

```

[ ]: def animate_pendulum_with_plots(
    L=1.0,
    G=9.81,
    theta_0_deg=20.0,
    num_revolutions=3,
    fps=30,
    save_animation=False,
    filename=None,
):
    """
    Animate a simple pendulum with oscillation counter and time display.

    Parameters:
    -----
    L : float, default=1.0
        Length of pendulum in meters
    G : float, default=9.81
        Acceleration due to gravity in m/s^2
    theta_0_deg : float, default=20.0
        Initial angle in degrees
    num_revolutions : int, default=3
        Number of complete oscillations to simulate
    fps : int, default=30
        Frames per second for animation
    save_animation : bool, default=False
        Whether to save animation as GIF
    filename : str or None, default=None
        Filename for saved GIF
    """

```

```

# Convert initial angle to radians
theta_0 = np.radians(theta_0_deg)

# Pendulum parameters
omega_linear = np.sqrt(G / L) # Natural frequency (small angle)
T_linear = 2 * np.pi / omega_linear # Period (small-angle approximation)

# Calculate actual nonlinear period
T = calculate_nonlinear_period(theta_0, L, G)
total_time = num_revolutions * T

# Actual angular frequency based on actual period
omega_actual = 2 * np.pi / T

# Period ratio for information
period_ratio = T / T_linear

num_frames = int(total_time * fps)
t_array = np.linspace(0, total_time, num_frames)

def calculate_pendulum_position(t, L=L, theta_0=theta_0, omega=omega_actual):
    """Calculate the position of the pendulum bob at time t."""
    theta_t = theta_0 * np.cos(omega * t)
    x = L * np.sin(theta_t)
    y = -L * np.cos(theta_t)
    return x, y, theta_t

def calculate_velocity(t, L=L, theta_0=theta_0, omega=omega_actual):
    """Calculate the velocity of the pendulum bob at time t."""
    theta_t = theta_0 * np.cos(omega * t)
    theta_dot = -theta_0 * omega * np.sin(omega * t)

    # Cartesian velocities
    vx = L * theta_dot * np.cos(theta_t)
    vy = L * theta_dot * np.sin(theta_t)

    return vx, vy, theta_dot

all_positions = [calculate_pendulum_position(t) for t in t_array]
all_velocities = [calculate_velocity(t) for t in t_array]

theta_angles = np.array([pos[2] for pos in all_positions])
theta_dots = np.array([vel[2] for vel in all_velocities])

# Setting up the figure with GridSpec layout (3 rows, 2 columns)
fig = plt.figure(figsize=(12, 10))
gs = GridSpec(
    3,
    2,
    figure=fig,
    hspace=0.3,
    wspace=0.3,
    left=0.08,
    right=0.95,
    top=0.9,
    bottom=0.06,
)

# Left column: Pendulum animation
ax_pendulum = fig.add_subplot(gs[:, 0])

# Right column: Three plots
ax_position = fig.add_subplot(gs[0, 1]) # Position vs time

```

```

ax_velocity = fig.add_subplot(gs[1, 1]) # Velocity vs time
ax_phase = fig.add_subplot(gs[2, 1]) # Phase space

# Configure pendulum axis
max_x = L * np.sin(theta_0) * 1.2
ax_pendulum.set_xlim(-max_x, max_x)
ax_pendulum.set_ylim(-L * 1.2, 0.3)
ax_pendulum.set_xlabel("Horizontal Position (m)", fontsize=11)
ax_pendulum.set_ylabel("Vertical Position (m)", fontsize=11)
ax_pendulum.set_title(
    "Undamped motion of a Pendulum", fontsize=12, fontweight="bold"
)
ax_pendulum.set_aspect("equal")

# Configure position vs time plot
ax_position.set_xlim(0, 1.1 * total_time)
ax_position.set_ylim(-theta_0_deg * 1.2, theta_0_deg * 1.2)
ax_position.set_xlabel("Time (s)", fontsize=10)
ax_position.set_ylabel("Angle  $\theta$  (°)", fontsize=10)
ax_position.set_title("Angular Position vs Time", fontsize=11, fontweight="bold")
ax_position.grid(True, alpha=0.3)

# Configure velocity vs time plot
max_omega = np.max(np.abs(theta_dots))
ax_velocity.set_xlim(0, 1.1 * total_time)
ax_velocity.set_ylim(-max_omega * 1.2, max_omega * 1.2)
ax_velocity.set_xlabel("Time (s)", fontsize=10)
ax_velocity.set_ylabel("Angular Velocity  $\omega$  (rad/s)", fontsize=10)
ax_velocity.set_title("Angular Velocity vs Time", fontsize=11, fontweight="bold")
ax_velocity.grid(True, alpha=0.3)

# Configure phase space plot
ax_phase.set_xlim(-theta_0_deg * 1.2, theta_0_deg * 1.2)
ax_phase.set_ylim(-max_omega * 1.2, max_omega * 1.2)
ax_phase.set_xlabel("Angle  $\theta$  (°)", fontsize=10)
ax_phase.set_ylabel("Angular Velocity  $\omega$  (rad/s)", fontsize=10)
ax_phase.set_title(
    "Phase Space ( $\omega$  vs  $\theta$ )", fontsize=11, fontweight="bold"
)
ax_phase.grid(True, alpha=0.3)

# Pendulum elements
pivot_rect = Rectangle(
    (-max_x / 4, -max_x / 16), max_x / 2, max_x / 8, color="black", zorder=10
)
ax_pendulum.add_patch(pivot_rect)

(pendulum_line,) = ax_pendulum.plot(
    [], [], "b-", linewidth=3, alpha=0.8, label="String"
)

bob_scatter = ax_pendulum.scatter(
    [], [], s=500, c="red", edgecolor="darkred", linewidth=2, label="Bob", zorder=5
)

trail_x, trail_y = [], []
(trail_line,) = ax_pendulum.plot(
    [], [], "r-", alpha=0.3, linewidth=1, label="Trail"
)

ax_pendulum.axhline(0, color="gray", linestyle="--", alpha=0.5)
ax_pendulum.axvline(0, color="gray", linestyle="--", alpha=0.5)

```

```

# Initialize plot elements for right column
# Position plot
(line_position,) = ax_position.plot([], [], "b-", linewidth=2, label=" $\theta(t)$ ")
point_position = ax_position.scatter([], [], s=80, c="red", zorder=5)

# Velocity plot
(line_velocity,) = ax_velocity.plot([], [], "g-", linewidth=2, label=" $\omega(t)$ ")
point_velocity = ax_velocity.scatter([], [], s=80, c="red", zorder=5)

# Phase space plot
(line_phase,) = ax_phase.plot(
    [], [], "purple", linewidth=1.5, alpha=0.6, label="Trajectory"
)
point_phase = ax_phase.scatter([], [], s=100, c="red", zorder=5, label="Current")

# plot legends
ax_pendulum.legend(
    loc="upper right",
    fancybox=True,
    edgecolor="gray",
    fontsize=9,
    labelspacing=1.2,
)
ax_position.legend(loc="upper right", fontsize=9)
ax_velocity.legend(loc="upper right", fontsize=9)
ax_phase.legend(loc="upper right", fontsize=8)

# Text displays for counters and information
info_text = ax_pendulum.text(
    0.03,
    0.98,
    "",
    transform=ax_pendulum.transAxes,
    fontsize=10,
    verticalalignment="top",
    bbox=dict(boxstyle="round,pad=0.5", facecolor="lightblue", alpha=0.8),
)

physics_text = ax_pendulum.text(
    0.02,
    0.01,
    "",
    transform=ax_pendulum.transAxes,
    fontsize=9,
    verticalalignment="bottom",
    bbox=dict(boxstyle="round,pad=0.3", facecolor="yellow", alpha=0.4),
)

physics_info = (
    f"Length: {L:.2f} m\n"
    f"Initial angle: {theta_0_deg:.1f}°\n"
    f"Period (linear): {T_linear:.3f} s\n"
    f"Period (actual): {T:.3f} s\n"
    f"Period ratio: {period_ratio:.4f}\n"
    f"Frequency: {omega_actual / (2 * np.pi):.3f} Hz"
)
physics_text.set_text(physics_info)

def animate_frame(frame):
    """Animation function called for each frame."""
    t_current = t_array[frame]

    # Calculates current position and velocity

```

```

x_bob, y_bob, theta_current = calculate_pendulum_position(t_current)
vx, vy, theta_dot_current = calculate_velocity(t_current)

# Updates pendulum string (from pivot to bob)
pendulum_line.set_data([0, x_bob], [0, y_bob])

# Updates bob position
bob_scatter.set_offsets([[x_bob, y_bob]])

# Updates trail (keeps last 25 points for smooth trail)
trail_x.append(x_bob)
trail_y.append(y_bob)
if len(trail_x) > 25:
    trail_x.pop(0)
    trail_y.pop(0)
trail_line.set_data(trail_x, trail_y)

# Update position vs time plot
t_data = t_array[: frame + 1]
theta_data = np.degrees(theta_angles[: frame + 1])
line_position.set_data(t_data, theta_data)
point_position.set_offsets([[t_current, np.degrees(theta_current)]])

# Update velocity vs time plot
omega_data = theta_dots[: frame + 1]
line_velocity.set_data(t_data, omega_data)
point_velocity.set_offsets([[t_current, theta_dot_current]])

# Update phase space plot
theta_phase_data = np.degrees(theta_angles[: frame + 1])
omega_phase_data = theta_dots[: frame + 1]
line_phase.set_data(theta_phase_data, omega_phase_data)
point_phase.set_offsets([[np.degrees(theta_current), theta_dot_current]])

# Calculates number of completed oscillations
completed_oscillations = t_current / T

progress_pct = (frame / (num_frames - 1)) * 100 if num_frames > 1 else 100.0

info_display = (
    f"Time: {t_current:.2f} s\n"
    f"Oscillations: {completed_oscillations:.2f}\n"
    f"Angle: {np.degrees(theta_current):+.2f}°\n"
    f"Velocity: {theta_dot_current:+.3f} rad/s\n"
    f"Progress: {progress_pct:.1f}%"
)
info_text.set_text(info_display)

# Updates main title
fig.suptitle(
    "Analysis of Simple Harmonic Motion of a Pendulum\n"
    f"Oscillation: {completed_oscillations:.2f}/{num_revolutions}",
    fontsize=14,
    fontweight="bold",
)

return (
    pendulum_line,
    bob_scatter,
    trail_line,
    info_text,
    line_position,
    point_position,

```

```

        line_velocity,
        point_velocity,
        line_phase,
        point_phase,
    )

    # Creates animation
    anim = FuncAnimation(
        fig,
        animate_frame,
        frames=num_frames,
        interval=int(1000 / fps),
        blit=True,
        repeat=False,
    )

    if save_animation:
        if filename is None:
            filename = f"pendulum_analysis_L{L}_theta{theta_0_deg:.
→0f}deg_{num_revolutions}rev.gif"

        save_dir = "OUTPUTS/ANIMATIONS"
        os.makedirs(save_dir, exist_ok=True)
        filepath = os.path.join(save_dir, filename)

        try:
            print("Saving pendulum animation... This may take a moment.")
            anim.save(filepath, writer="pillow", dpi=150)
            print(f"Animation saved as: {filepath}")
        except Exception as e:
            print(f"Error saving animation: {e}")
        plt.close()
    else:
        plt.show()

    print("\nPendulum Simulation Summary:")
    print(f"Length: {L} m")
    print(f"Initial angle: {theta_0_deg:.1f}°")
    print(f"Period (small-angle approx): {T_linear:.3f} s")
    print(f"Period (nonlinear, actual): {T:.3f} s")
    print(f"Period increase: {100 * (period_ratio - 1):.2f}%")
    print(f"Total simulation time: {total_time:.2f} s")
    print(f"Number of oscillations: {num_revolutions}")

    return anim

def main():
    """Main function to run the pendulum animation with user inputs."""

    print("Simple Pendulum Animation Simulator")
    print("=====")

    # Default parameters
    defaults = {
        "L": 1.0,
        "G": 9.81,
        "theta_0_deg": 20.0,
        "num_revolutions": 3,
        "fps": 30,
    }

    use_defaults = input("Use default parameters? (y/n): ").strip().lower()

```

```

if use_defaults == "y":
    params = defaults.copy()
else:
    params = {}
    try:
        params["L"] = float(
            input(f"Enter length of pendulum in meters [{defaults['L']}] : ")
            or defaults["L"]
        )
        params["G"] = float(
            input(f"Enter gravitational acceleration in m/s2 [{defaults['G']}] : ")
            or defaults["G"]
        )
        params["theta_0_deg"] = float(
            input(f"Enter initial angle in degrees [{defaults['theta_0_deg']}] : ")
            or defaults["theta_0_deg"]
        )
        params["num_revolutions"] = int(
            input(
                f"Enter number of oscillations to simulate,
↪ [{defaults['num_revolutions']}] : "
            )
            or defaults["num_revolutions"]
        )
        params["fps"] = int(
            input(f"Enter frames per second [{defaults['fps']}] : ")
            or defaults["fps"]
        )
    except ValueError:
        print("Invalid input, using default parameters.")
        params = defaults.copy()

save_anim = input("Save animation as GIF? (y/n) : ").strip().lower() == "y"
filename = None
if save_anim:
    filename = input("Enter filename (or press Enter for auto-generated) : ").strip()
    if not filename.endswith(".gif") and filename != "":
        filename += ".gif"
    if not filename:
        filename = None

print("\nRunning simulation with parameters:")
for key, value in params.items():
    print(f"  {key}: {value}")

animation = animate_pendulum_with_plots(
    L=params["L"],
    G=params["G"],
    theta_0_deg=params["theta_0_deg"],
    num_revolutions=params["num_revolutions"],
    fps=params["fps"],
    save_animation=save_anim,
    filename=filename,
)

return animation

if __name__ == "__main__":
    anim = main()

```

4.3 Damped Pendulum Motion

Runge-Kutta Method for Solving Differential Equations

For solving the ODE related to the damped pendulum motion, we used the Runge-Kutta method of order 4. Below we have briefly discussed about the RK4 method and how is this implemented in our case.

1. Introduction to Differential Equations

We want to solve initial value problems (IVP) of the form:

$$\frac{dy}{dt} = f(t, y), \quad y(t_0) = y_0$$

For a pendulum, we have a **system of first-order ODEs**:

$$\begin{aligned}\frac{d\theta}{dt} &= \omega \\ \frac{d\omega}{dt} &= f(\theta, \omega, t)\end{aligned}$$

where $\omega = \frac{d\theta}{dt}$ is the angular velocity, and f depends on the forces acting on the pendulum.

2. The Runge-Kutta 4th Order (RK4) Method

The RK4 method approximates the solution by evaluating the slope (derivative) at **four strategic points** within each time step.

Derivation from Taylor Series

The exact solution over a time step $h = \Delta t$ is:

$$y(t+h) = y(t) + \int_t^{t+h} f(s, y(s)) ds$$

The RK4 method approximates this integral using Simpson's rule with four weighted evaluations:

$$y_{n+1} = y_n + \frac{h}{6}(k_1 + 2k_2 + 2k_3 + k_4)$$

where the slopes are computed at:

Step 1 (at current position):

$$k_1 = f(t_n, y_n)$$

Step 2 (at midpoint, using k_1):

$$k_2 = f\left(t_n + \frac{h}{2}, y_n + \frac{h}{2}k_1\right)$$

Step 3 (at midpoint, using k_2):

$$k_3 = f\left(t_n + \frac{h}{2}, y_n + \frac{h}{2}k_2\right)$$

Step 4 (at next position, using k_3):

$$k_4 = f(t_n + h, y_n + h \cdot k_3)$$

Weighting The coefficients $\{1, 2, 2, 1\}$ divided by 6 give higher weight to the midpoint evaluations: - Endpoints (k_1, k_4): weight $\frac{1}{6}$ each - Midpoints (k_2, k_3): weight $\frac{1}{3}$ each

This provides **4th-order accuracy**: local truncation error $O(h^5)$ and global error $O(h^4)$.

3. Application to Undamped Pendulum

Equation of Motion

$$\frac{d^2\theta}{dt^2} = -\frac{g}{L} \sin \theta$$

We convert this to a **system of first-order equations**:

$$\frac{d\theta}{dt} = \omega$$

$$\frac{d\omega}{dt} = -\frac{g}{L} \sin \theta$$

RK4 Implementation

Let $y_1 = \theta$ and $y_2 = \omega$. The RK4 update becomes:

$$\begin{aligned} k_1^{(\theta)} &= \omega_n \\ k_1^{(\omega)} &= -\frac{g}{L} \sin(\theta_n) \end{aligned}$$

$$\begin{aligned} k_2^{(\theta)} &= \omega_n + \frac{h}{2} k_1^{(\omega)} \\ k_2^{(\omega)} &= -\frac{g}{L} \sin\left(\theta_n + \frac{h}{2} k_1^{(\theta)}\right) \end{aligned}$$

$$\begin{aligned} k_3^{(\theta)} &= \omega_n + \frac{h}{2} k_2^{(\omega)} \\ k_3^{(\omega)} &= -\frac{g}{L} \sin\left(\theta_n + \frac{h}{2} k_2^{(\theta)}\right) \end{aligned}$$

$$\begin{aligned} k_4^{(\theta)} &= \omega_n + h \cdot k_3^{(\omega)} \\ k_4^{(\omega)} &= -\frac{g}{L} \sin(\theta_n + h \cdot k_3^{(\theta)}) \end{aligned}$$

Update equations:

$$\begin{aligned} \theta_{n+1} &= \theta_n + \frac{h}{6} \left(k_1^{(\theta)} + 2k_2^{(\theta)} + 2k_3^{(\theta)} + k_4^{(\theta)} \right) \\ \omega_{n+1} &= \omega_n + \frac{h}{6} \left(k_1^{(\omega)} + 2k_2^{(\omega)} + 2k_3^{(\omega)} + k_4^{(\omega)} \right) \end{aligned}$$

4. Application to Damped Pendulum

Equation of Motion

$$\frac{d^2\theta}{dt^2} = -\frac{g}{L} \sin \theta - \gamma \frac{d\theta}{dt}$$

where $\gamma = \frac{c}{m}$ is the **damping coefficient** (with c being the viscous damping constant).

This represents viscous damping proportional to angular velocity.

System of First-Order ODEs

$$\frac{d\theta}{dt} = \omega$$

$$\frac{d\omega}{dt} = -\frac{g}{L} \sin \theta - \gamma \omega$$

RK4 Implementation

The only change from the undamped case is in the ω derivatives:

$$k_1^{(\omega)} = -\frac{g}{L} \sin(\theta_n) - \gamma \omega_n$$

$$k_2^{(\omega)} = -\frac{g}{L} \sin\left(\theta_n + \frac{h}{2} k_1^{(\theta)}\right) - \gamma\left(\omega_n + \frac{h}{2} k_1^{(\omega)}\right)$$

$$k_3^{(\omega)} = -\frac{g}{L} \sin\left(\theta_n + \frac{h}{2} k_2^{(\theta)}\right) - \gamma\left(\omega_n + \frac{h}{2} k_2^{(\omega)}\right)$$

$$k_4^{(\omega)} = -\frac{g}{L} \sin(\theta_n + h \cdot k_3^{(\theta)}) - \gamma(\omega_n + h \cdot k_3^{(\omega)})$$

The update equations for θ and ω remain the same.

5. Comparison: Undamped vs. Damped

Property	Undamped	Damped
Equation	$\ddot{\theta} + \frac{g}{L} \sin \theta = 0$	$\ddot{\theta} + \gamma \dot{\theta} + \frac{g}{L} \sin \theta = 0$
Energy	Conserved	Decreases exponentially
Amplitude	Constant	Decays as $e^{-\gamma t/2}$ (for small angles)
Frequency	$\omega_0 = \sqrt{g/L}$	$\omega_d = \sqrt{\omega_0^2 - (\gamma/2)^2}$ (underdamped)
Long-term behavior	Periodic oscillation	Oscillates and approaches equilibrium ($\theta \rightarrow 0$)

6. RK4 Code Implementation

Here's how this is implemented:

```
def rk4_damped_pendulum(theta, omega, dt, L, G, gamma):
    """RK4 solver for damped pendulum"""

    def derivatives(theta, omega):
        dtheta_dt = omega
        domega_dt = -(G / L) * np.sin(theta) - gamma * omega
        return dtheta_dt, domega_dt

    # Four slope evaluations
    k1_theta, k1_omega = derivatives(theta, omega)

    k2_theta, k2_omega = derivatives(
        theta + 0.5 * dt * k1_theta,
        omega + 0.5 * dt * k1_omega
    )

    k3_theta, k3_omega = derivatives(
        theta + 0.5 * dt * k2_theta,
        omega + 0.5 * dt * k2_omega
    )

    k4_theta, k4_omega = derivatives(
        theta + dt * k3_theta,
        omega + dt * k3_omega
    )

    # Weighted average
    theta_new = theta + (dt / 6.0) * (k1_theta + 2*k2_theta + 2*k3_theta + k4_theta)
    omega_new = omega + (dt / 6.0) * (k1_omega + 2*k2_omega + 2*k3_omega + k4_omega)

    return theta_new, omega_new
```

7. Local vs. Global Truncation Error

Local Truncation Error: $e_n = O(h^5)$ - Error introduced at a single step

Global Truncation Error: $E_N = O(h^4)$ - Accumulated error over N steps where $N \approx T/h$ - As h decreases, global error improves quadratically relative to Euler's method

This makes RK4 ideal for simulating pendulum dynamics over multiple oscillation periods.

Damped vs Undamped Comparison Animation

This animation simultaneously displays multiple pendulums with different damping coefficients, allowing direct comparison of their behaviors:

- **Undamped** ($\gamma = 0$): Oscillates indefinitely with constant amplitude
- **Underdamped** ($\gamma < \omega_n$): Oscillations with exponentially decreasing amplitude
- **Critically damped** ($\gamma = 2\omega_n$): Returns to equilibrium without oscillating (fastest return)
- **Overdamped** ($\gamma > 2\omega_n$): Slowly approaches equilibrium without oscillating

```
[ ]: def rk4_damped_pendulum(theta, omega, dt, L, G, gamma):  
    """  
    RK4 solver for damped pendulum equation of motion.  
     $d^2\theta/dt^2 = -(g/L)\sin(\theta) - \gamma(d\theta/dt)$   
  
    Parameters:  
    -----  
    theta : float  
        Current angle  
    omega : float  
        Current angular velocity  
    dt : float  
        Time step  
    L : float  
        Length of pendulum  
    G : float  
        Gravitational acceleration  
    gamma : float  
        Damping coefficient  
  
    Returns:  
    -----  
    theta_new, omega_new : tuple  
        Updated angle and angular velocity  
    """  
  
    def derivatives(theta, omega):  
        dtheta_dt = omega  
        domega_dt = -(G / L) * np.sin(theta) - gamma * omega  
        return dtheta_dt, domega_dt  
  
    # RK4 steps  
    k1_theta, k1_omega = derivatives(theta, omega)  
    k2_theta, k2_omega = derivatives(  
        theta + 0.5 * dt * k1_theta, omega + 0.5 * dt * k1_omega  
    )  
    k3_theta, k3_omega = derivatives(  
        theta + 0.5 * dt * k2_theta, omega + 0.5 * dt * k2_omega  
    )  
    k4_theta, k4_omega = derivatives(theta + dt * k3_theta, omega + dt * k3_omega)  
  
    theta_new = theta + (dt / 6.0) * (k1_theta + 2 * k2_theta + 2 * k3_theta + k4_theta)  
    omega_new = omega + (dt / 6.0) * (k1_omega + 2 * k2_omega + 2 * k3_omega + k4_omega)  
  
    return theta_new, omega_new
```

```

def animate_damped_vs_undamped_pendulum(
    L=1.0,
    G=9.81,
    theta_0_deg=20.0,
    gamma_list=[0.0, 0.25, 0.5],
    num_periods=5,
    fps=30,
    save_animation=False,
    filename=None,
):
    """
    Animate multiple pendulums with different damping coefficients.
    All pendulums start from the same initial position.

    Parameters:
    -----
    L : float, default=1.0
        Length of pendulum in meters
    G : float, default=9.81
        Acceleration due to gravity in m/s^2
    theta_0_deg : float, default=20.0
        Initial angle in degrees
    gamma_list : list of float, default=[0.0, 0.25, 0.5]
        List of damping coefficients for pendulums (gamma = c/m)
        First value should be 0.0 for undamped pendulum
    num_periods : int, default=5
        Number of complete periods to simulate
    fps : int, default=30
        Frames per second for animation
    save_animation : bool, default=False
        Whether to save animation as GIF
    filename : str or None, default=None
        Filename for saved GIF
    """

    # Convert initial angle to radians
    theta_0 = np.radians(theta_0_deg)

    # Pendulum parameters
    omega_0 = np.sqrt(G / L) # Natural frequency (small angle approximation)
    T_linear = 2 * np.pi / omega_0 # Period using small-angle approximation

    # Calculate actual nonlinear period for the given amplitude
    T = calculate_nonlinear_period(theta_0, L, G)
    total_time = num_periods * T

    # Calculate period ratio for information
    period_ratio = T / T_linear

    # Time array for animation
    num_frames = int(total_time * fps) + 1
    t_array = np.linspace(0, total_time, num_frames)
    dt = t_array[1] - t_array[0]

    # Initialize arrays for all pendulums
    num_pendulums = len(gamma_list)
    theta_all = np.zeros((num_pendulums, num_frames))
    omega_all = np.zeros((num_pendulums, num_frames))

    # Set initial conditions for all pendulums
    for j in range(num_pendulums):
        theta_all[j, 0] = theta_0

```

```

    omega_all[j, 0] = 0.0

# Calculate motion for all pendulums using RK4
for j in range(num_pendulums):
    for i in range(num_frames - 1):
        theta_all[j, i + 1], omega_all[j, i + 1] = rk4_damped_pendulum(
            theta_all[j, i], omega_all[j, i], dt, L, G, gamma_list[j]
        )

# Convert to Cartesian coordinates for all pendulums
x_all = L * np.sin(theta_all)
y_all = -L * np.cos(theta_all)

# Setting up the figure - single plot
fig, ax_pendulum = plt.subplots(figsize=(10, 12))
fig.subplots_adjust(top=0.9, bottom=0.08, left=0.1, right=0.9)

# Calculate plot limits
max_x = L * np.sin(theta_0) * 1.2

# Configure pendulum axis
ax_pendulum.set_xlim(-max_x, max_x)
ax_pendulum.set_ylim(-L * 1.2, 0.3)
ax_pendulum.set_xlabel("Horizontal Position (m)", fontsize=13)
ax_pendulum.set_ylabel("Vertical Position (m)", fontsize=13)
ax_pendulum.axhline(0, color="gray", linestyle="--", alpha=0.4)
ax_pendulum.axvline(0, color="gray", linestyle="--", alpha=0.4)
ax_pendulum.grid(True, alpha=0.2)

# Main title
gamma_str = ", ".join([f"{g:.2f}" for g in gamma_list])
fig.suptitle(
    f"Undamped vs Damped Motion of a Pendulum\n"
    f"$L$={L}m, $\theta_0$={theta_0_deg}$^\circ$, $g$={G}m/s$^2$, $\gamma$=[{gamma_str}]",
    fontsize=16,
    fontweight="bold",
)

# Pivot point
pivot_width = max_x / 4
pivot_height = max_x / 16

pivot_rect = Rectangle(
    (-pivot_width, -pivot_height),
    2 * pivot_width,
    2 * pivot_height,
    color="black",
    zorder=10,
)

ax_pendulum.add_patch(pivot_rect)

# colors for different pendulums
colors = ["blue", "green", "red", "violet", "orange", "magenta", "gray"]

# Create pendulum elements for all pendulums
lines = []
bobs = []
trails_x = []
trails_y = []
trail_lines = []

for j in range(num_pendulums):
    color = colors[j % len(colors)]

```

```

gamma_val = gamma_list[j]
edgecolor = f"dark{color}"

# Label for legend
if gamma_val == 0.0:
    label = "Undamped ( $\gamma=0$ )"
else:
    label = f"Damped ( $\gamma={\text{gamma\_val:.2f}}$ )"

# Create line, bob, and trail for pendulum animation
(line,) = ax_pendulum.plot(
    [], [], color=color, linewidth=3, alpha=0.7, label=label
)
bob = ax_pendulum.scatter(
    [],
    [],
    s=500,
    c=color,
    edgecolor=edgecolor,
    linewidth=2,
    zorder=5,
)
trail_x, trail_y = [], []
(trail_line,) = ax_pendulum.plot([], [], color=color, alpha=0.25, linewidth=1.5)

lines.append(line)
bobs.append(bob)
trails_x.append(trail_x)
trails_y.append(trail_y)
trail_lines.append(trail_line)

# Add legend
ax_pendulum.legend(
    loc="upper left",
    fontsize=10,
    framealpha=0.7,
    fancybox=True,
    ncol=1 if num_pendulums <= 4 else 2,
)

# Text displays - only show if 3 or fewer pendulums
show_info = num_pendulums <= 3
info_texts = []

if show_info:
    info_colors = ["lightblue", "lightgreen", "lightcoral"]
    y_positions = [0.98, 0.84, 0.7]

    for j in range(num_pendulums):
        info_text = ax_pendulum.text(
            0.78,
            y_positions[j],
            "",
            transform=ax_pendulum.transAxes,
            fontsize=10,
            va="top",
            ha="left",
            bbox=dict(
                boxstyle="round,pad=0.5", facecolor=info_colors[j], alpha=0.75
            ),
        )
        info_texts.append(info_text)

```

```

physics_text = ax_pendulum.text(
    0.02,
    0.88,
    "",
    transform=ax_pendulum.transAxes,
    fontsize=10,
    va="top",
    ha="left",
    bbox=dict(boxstyle="round,pad=0.4", facecolor="yellow", alpha=0.4),
)

physics_text.set_text(
    f"Pendulum Properties:\n"
    f"Length: {L:.2f} m\n"
    f"Initial angle: {theta_0_deg:.1f}°\n"
    f"Period (linear): {T_linear:.3f} s\n"
    f"Period (actual): {T:.3f} s\n"
    f"Period ratio: {period_ratio:.4f}\n"
    f"$\\omega_0$: {omega_0:.3f} rad/s"
)

# Calculate energy for pendulums
def calculate_energy(theta, omega, m=1.0):
    """Calculate total mechanical energy"""
    kinetic = 0.5 * m * (L * omega) ** 2
    potential = m * G * L * (1 - np.cos(theta))
    return kinetic + potential

initial_energy = calculate_energy(theta_0, 0.0)

def animate_frame(frame):
    """Animation function called for each frame."""
    t_current = t_array[frame]
    completed_periods = t_current / T

    artists = []

    # Update all pendulums
    for j in range(num_pendulums):
        x = x_all[j, frame]
        y = y_all[j, frame]
        theta = theta_all[j, frame]
        omega = omega_all[j, frame]

        # Update line and bob
        lines[j].set_data(np.array([0, x]), np.array([0, y]))
        bobs[j].set_offsets([x, y])

        # Update trail
        trails_x[j].append(x)
        trails_y[j].append(y)
        if len(trails_x[j]) > 25:
            trails_x[j].pop(0)
            trails_y[j].pop(0)
        trail_lines[j].set_data(np.array(trails_x[j]), np.array(trails_y[j]))

        # Add to artists list
        artists.extend([lines[j], bobs[j], trail_lines[j]])

        # Update info text if shown
        if show_info:
            energy = calculate_energy(theta, omega)
            gamma_val = gamma_list[j]

```

```

        if gamma_val == 0.0:
            title = "UNDAMPED ($\\gamma$=0)"
        else:
            title = f"DAMPED ($\\gamma$={gamma_val:.2f})"

        info_display = (
            f"{title}\\n"
            f"{'-' * 18}\\n"
            f"$\\theta$: {np.degrees(theta):+.2f}$^{\circ}$\\n"
            f"$\\omega$: {omega:+.3f} rad/s\\n"
            f"$E$: {energy:.3f} J\\n"
            f"$E/E_0$: {energy / initial_energy:.4f}"
        )
        info_texts[j].set_text(info_display)
        artists.append(info_texts[j])

    # Update title with time and period info
    ax_pendulum.set_title(
        f"Time: {t_current:.2f} s | Period: {completed_periods:.2f}/{num_periods}",
        fontsize=14,
        fontweight="bold",
        pad=10,
    )

    return tuple(artists)

anim = FuncAnimation(
    fig,
    animate_frame,
    frames=num_frames,
    interval=int(1000 / fps),
    blit=False,
    repeat=False,
)

if save_animation:
    if filename is None:
        gamma_str_file = "_".join([f"{g:.2f}" for g in gamma_list])
        filename = (
            f"multi_pendulum_L{L}_theta{theta_0_deg:.0f}deg_"
            f"gamma{gamma_str_file}_{num_periods}periods.gif"
        )

    save_dir = "OUTPUTS/ANIMATIONS"
    os.makedirs(save_dir, exist_ok=True)
    filepath = os.path.join(save_dir, filename)

    try:
        print("Saving animation... This may take a moment.")
        anim.save(filepath, writer="ffmpeg", fps=fps, codec="gif", dpi=80)
        print(f"Animation saved as: {filepath}")
    except Exception as e:
        print(f"Error saving animation: {e}")

    plt.close(fig)
else:
    plt.show()

print("\nMulti-Pendulum Comparison Summary:")
print("=" * 60)
print(f"Number of pendulums: {num_pendulums}")
print(f"Length: {L} m")

```

```

print(f"Initial angle: {theta_0_deg:.1f}°")
print(f"Period (small-angle approx): {T_linear:.3f} s")
print(f"Period (nonlinear, actual): {T:.3f} s")
print(f"Period increase due to nonlinearity: {100 * (period_ratio - 1):.2f}%")
print(f"Total simulation time: {total_time:.2f} s")
print(f"Number of periods: {num_periods}")
print(f"\nDamping coefficients:  $\gamma$  = {gamma_list}")
print("\nFinal Results for Each Pendulum:")
print("-" * 60)
for j in range(num_pendulums):
    gamma_val = gamma_list[j]
    final_angle = np.degrees(theta_all[j, -1])
    energy_final = calculate_energy(theta_all[j, -1], omega_all[j, -1])
    energy_retention = 100 * energy_final / initial_energy

    print(f"Pendulum {j + 1} ( $\gamma$ = {gamma_val:.3f}):")
    print(f"  Final angle: {final_angle:+.2f}° (initial: {theta_0_deg:.1f}°)")
    print(f"  Energy retention: {energy_retention:.1f}%")

return anim

def main():
    """Main function to run the multi-pendulum animation with user inputs."""

    print("Multi-Pendulum Damping Analysis Animation")
    print("=" * 50)

    # Default parameters
    defaults = {
        "L": 1.0,
        "G": 9.81,
        "theta_0_deg": 20.0,
        "gamma_list": [0.0, 0.25, 0.5],
        "num_periods": 5,
        "fps": 30,
    }

    use_defaults = input("Use default parameters? (y/n): ").strip().lower()

    if use_defaults == "y":
        params = defaults.copy()
    else:
        params = {}
        try:
            params["L"] = float(
                input(f"Enter pendulum length in meters [{defaults['L']}]: ")
                or defaults["L"]
            )
            params["G"] = float(
                input(f"Enter gravitational acceleration in m/s² [{defaults['G']}]: ")
                or defaults["G"]
            )
            params["theta_0_deg"] = float(
                input(f"Enter initial angle in degrees [{defaults['theta_0_deg']}]: ")
                or defaults["theta_0_deg"]
            )

            # Get gamma list
            gamma_input = input(
                "Enter damping coefficients as comma-separated values (e.g., 0.0,0.
→25,0.5) "

```

```

        f"[', '.join(map(str, defaults['gamma_list']))]": "
    )
).strip()
if gamma_input:
    params["gamma_list"] = [
        float(g.strip()) for g in gamma_input.split(",")
    ]
else:
    params["gamma_list"] = defaults["gamma_list"]

params["num_periods"] = int(
    input(
        f"Enter number of periods to simulate [{defaults['num_periods']}]: "
    )
    or defaults["num_periods"]
)
params["fps"] = int(
    input(f"Enter frames per second [{defaults['fps']}]: ")
    or defaults["fps"]
)
except ValueError:
    print("Invalid input, using default parameters.")
    params = defaults.copy()

save_anim = input("Save animation as GIF? (y/n): ").strip().lower() == "y"
filename = None
if save_anim:
    filename = input("Enter filename (or press Enter for auto-generated): ").strip()
    if filename and not filename.endswith(".gif"):
        filename += ".gif"
    if not filename:
        filename = None

print("\nRunning simulation with parameters:")
for key, value in params.items():
    print(f"  {key}: {value}")
print(f"  save_animation: {save_anim}")

animation = animate_damped_vs_undamped_pendulum(
    L=params["L"],
    G=params["G"],
    theta_0_deg=params["theta_0_deg"],
    gamma_list=params["gamma_list"],
    num_periods=params["num_periods"],
    fps=params["fps"],
    save_animation=save_anim,
    filename=filename,
)
return animation

if __name__ == "__main__":
    anim = main()

```

See the animation in the GitHub repository: [Animation 2](#)

4.4 Damped Pendulum Motion with Phase and Time Domain Plots

```

[ ]: def animate_damped_vs_undamped_pendulum_with_plots(
    L=1.0,
    G=9.81,
    theta_0_deg=20.0,
    gamma_list=[0.0, 0.25, 0.5],

```

```

num_periods=5,
fps=30,
save_animation=False,
filename=None,
):
    """
    Animate multiple pendulums with different damping coefficients.
    All pendulums start from the same initial position.

    Parameters:
    -----
    L : float, default=1.0
        Length of pendulum in meters
    G : float, default=9.81
        Acceleration due to gravity in m/s^2
    theta_0_deg : float, default=20.0
        Initial angle in degrees
    gamma_list : list of float, default=[0.0, 0.25, 0.5]
        List of damping coefficients for pendulums (gamma = c/m)
        First value should be 0.0 for undamped pendulum
    num_periods : int, default=5
        Number of complete periods to simulate
    fps : int, default=30
        Frames per second for animation
    save_animation : bool, default=False
        Whether to save animation as GIF
    filename : str or None, default=None
        Filename for saved GIF
    """

    theta_0 = np.radians(theta_0_deg)

    # Pendulum parameters
    omega_0 = np.sqrt(G / L)
    T_linear = 2 * np.pi / omega_0

    # Calculate actual nonlinear period for the given amplitude
    T = calculate_nonlinear_period(theta_0, L, G)
    total_time = num_periods * T

    # Calculate period ratio for information
    period_ratio = T / T_linear

    num_frames = int(total_time * fps) + 1
    t_array = np.linspace(0, total_time, num_frames)
    dt = t_array[1] - t_array[0]

    # Initialize arrays for all pendulums
    num_pendulums = len(gamma_list)
    theta_all = np.zeros((num_pendulums, num_frames))
    omega_all = np.zeros((num_pendulums, num_frames))

    # Set initial conditions for all pendulums
    for j in range(num_pendulums):
        theta_all[j, 0] = theta_0
        omega_all[j, 0] = 0.0

    # Calculate motion for all pendulums using RK4
    for j in range(num_pendulums):
        for i in range(num_frames - 1):
            theta_all[j, i + 1], omega_all[j, i + 1] = rk4_damped_pendulum(
                theta_all[j, i], omega_all[j, i], dt, L, G, gamma_list[j]
            )

```

```

# Convert to Cartesian coordinates for all pendulums
x_all = L * np.sin(theta_all)
y_all = -L * np.cos(theta_all)

# Plot setup with GridSpec
fig = plt.figure(figsize=(14, 12))
gs = GridSpec(
    3,
    2,
    figure=fig,
    hspace=0.32,
    wspace=0.28,
    left=0.06,
    right=0.95,
    top=0.9,
    bottom=0.05,
)

# Left column: Pendulum animation
ax_pendulum = fig.add_subplot(gs[:, 0])

# Right column: Three plots
ax_position = fig.add_subplot(gs[0, 1]) # Position vs time
ax_velocity = fig.add_subplot(gs[1, 1]) # Velocity vs time
ax_phase = fig.add_subplot(gs[2, 1]) # Phase space

max_x = L * np.sin(theta_0) * 1.2

# Configure pendulum axis
ax_pendulum.set_xlim(-max_x, max_x)
ax_pendulum.set_ylim(-L * 1.2, 0.3)
ax_pendulum.set_xlabel("Horizontal Position (m)", fontsize=11)
ax_pendulum.set_ylabel("Vertical Position (m)", fontsize=11)
ax_pendulum.axhline(0, color="gray", linestyle="--", alpha=0.4)
ax_pendulum.axvline(0, color="gray", linestyle="--", alpha=0.4)
ax_pendulum.grid(True, alpha=0.2)

# Configure position vs time plot
ax_position.set_xlim(0, 1.1 * total_time)
ax_position.set_ylim(-theta_0_deg * 1.15, theta_0_deg * 1.2)
ax_position.set_xlabel("Time (s)", fontsize=11)
ax_position.set_ylabel("Angle  $\theta$  (°)", fontsize=11)
ax_position.set_title("Angular Position vs Time", fontsize=12, fontweight="bold")
ax_position.grid(True, alpha=0.3)

# Configure velocity vs time plot
max_omega = np.max(np.abs(omega_all))
ax_velocity.set_xlim(0, 1.1 * total_time)
ax_velocity.set_ylim(-max_omega * 1.15, max_omega * 1.2)
ax_velocity.set_xlabel("Time (s)", fontsize=11)
ax_velocity.set_ylabel("Angular Velocity  $\omega$  (rad/s)", fontsize=11)
ax_velocity.set_title("Angular Velocity vs Time", fontsize=12, fontweight="bold")
ax_velocity.grid(True, alpha=0.3)

# Configure phase space plot
ax_phase.set_xlim(-theta_0_deg * 1.15, theta_0_deg * 1.15)
ax_phase.set_ylim(-max_omega * 1.15, max_omega * 1.2)
ax_phase.set_xlabel("Angle  $\theta$  (°)", fontsize=11)
ax_phase.set_ylabel("Angular Velocity  $\omega$  (rad/s)", fontsize=11)
ax_phase.set_title(
    "Phase Space ( $\omega$  vs  $\theta$ )", fontsize=12, fontweight="bold"
)

```

```

ax_phase.grid(True, alpha=0.3)

# Main title
gamma_str = ", ".join([f"{g:.2f}" for g in gamma_list])
fig.suptitle(
    f"Analysis of Undamped and Damped motion of Pendulums\n"
    f"$L$={L}m, $\theta_0$={theta_0_deg}°, $g$={G}m/s², $\gamma$={gamma_str}",
    fontsize=16,
    fontweight="bold",
)

# Pivot point
pivot_width = max_x / 4
pivot_height = max_x / 16

pivot_rect = Rectangle(
    (-pivot_width, -pivot_height),
    2 * pivot_width,
    2 * pivot_height,
    color="black",
    zorder=10,
)

ax_pendulum.add_patch(pivot_rect)

# Colors for different pendulums
colors = ["blue", "green", "red", "magenta", "orange", "violet", "gray"]

# Create pendulum elements for all pendulums
lines = []
bobs = []
trails_x = []
trails_y = []
trail_lines = []

# Create plot elements for right column
lines_pos = []
lines_vel = []
lines_phase = []
points_pos = []
points_vel = []
points_phase = []

for j in range(num_pendulums):
    color = colors[j % len(colors)]
    gamma_val = gamma_list[j]
    edgecolor = f"dark{color}"

    # Label for legend
    if gamma_val == 0.0:
        label = "Undamped ($\gamma=0$)"
    else:
        label = f"Damped ($\gamma$={gamma_val:.2f})"

    # Create line, bob, and trail for pendulum animation
    (line,) = ax_pendulum.plot(
        [], [], color=color, linewidth=3, alpha=0.7, label=label
    )
    bob = ax_pendulum.scatter(
        [],
        [],
        s=500,
        c=color,
        edgecolor=edgecolor,

```

```

        linewidth=2,
        zorder=5,
    )
    trail_x, trail_y = [], []
    (trail_line,) = ax_pendulum.plot([], [], color=color, alpha=0.25, linewidth=1.5)

    lines.append(line)
    bobs.append(bob)
    trails_x.append(trail_x)
    trails_y.append(trail_y)
    trail_lines.append(trail_line)

    # Create elements for position plot
    (line_pos,) = ax_position.plot(
        [], [], color=color, linewidth=2, alpha=0.7, label=label
    )
    point_pos = ax_position.scatter(
        [],
        [],
        s=50,
        c=color,
        zorder=5,
        edgecolor=edgecolor,
    )
    lines_pos.append(line_pos)
    points_pos.append(point_pos)

    # Create elements for velocity plot
    (line_vel,) = ax_velocity.plot(
        [], [], color=color, linewidth=2, alpha=0.7, label=label
    )
    point_vel = ax_velocity.scatter(
        [],
        [],
        s=50,
        c=color,
        zorder=5,
        edgecolor=edgecolor,
    )
    lines_vel.append(line_vel)
    points_vel.append(point_vel)

    # Create elements for phase space plot
    (line_phase,) = ax_phase.plot(
        [], [], color=color, linewidth=1.5, alpha=0.6, label=label
    )
    point_phase = ax_phase.scatter(
        [],
        [],
        s=60,
        c=color,
        zorder=5,
        edgecolor=edgecolor,
    )
    lines_phase.append(line_phase)
    points_phase.append(point_phase)

    # Add legends
    ax_pendulum.legend(
        loc="upper left",
        fontsize=9,
        framealpha=0.7,
        ncol=1 if num_pendulums <= 4 else 2,

```

```

)

# Text displays - only show if 3 or fewer pendulums
show_info = num_pendulums <= 3
info_texts = []

if show_info:
    info_colors = ["lightblue", "lightgreen", "lightcoral"]
    y_positions = [0.98, 0.86, 0.74]

    for j in range(num_pendulums):
        info_text = ax_pendulum.text(
            0.69,
            y_positions[j],
            "",
            transform=ax_pendulum.transAxes,
            fontsize=9,
            va="top",
            ha="left",
            bbox=dict(
                boxstyle="round,pad=0.5", facecolor=info_colors[j], alpha=0.75
            ),
        )
        info_texts.append(info_text)

physics_text = ax_pendulum.text(
    0.02,
    0.91,
    "",
    transform=ax_pendulum.transAxes,
    fontsize=9,
    va="top",
    ha="left",
    bbox=dict(boxstyle="round,pad=0.4", facecolor="yellow", alpha=0.4),
)

physics_text.set_text(
    f"Pendulum Properties:\n"
    f"Length: {L:.2f} m\n"
    f"Initial angle: {theta_0_deg:.1f}°\n"
    f"Period (linear): {T_linear:.3f} s\n"
    f"Period (actual): {T:.3f} s\n"
    f"Period ratio: {period_ratio:.4f}\n"
    f" $\omega_0$ : {omega_0:.3f} rad/s"
)

# Calculate energy for pendulums
def calculate_energy(theta, omega, m=1.0):
    """Calculate total mechanical energy"""
    kinetic = 0.5 * m * (L * omega) ** 2
    potential = m * G * L * (1 - np.cos(theta))
    return kinetic + potential

initial_energy = calculate_energy(theta_0, 0.0)

def animate_frame(frame):
    """Animation function called for each frame."""
    t_current = t_array[frame]
    completed_periods = t_current / T

    artists = []

    # Update all pendulums

```

```

for j in range(num_pendulums):
    x = x_all[j, frame]
    y = y_all[j, frame]
    theta = theta_all[j, frame]
    omega = omega_all[j, frame]

    # Update line and bob
    lines[j].set_data(np.array([0, x]), np.array([0, y]))
    bobs[j].set_offsets([[x, y]])

    # Update trail
    trails_x[j].append(x)
    trails_y[j].append(y)
    if len(trails_x[j]) > 25:
        trails_x[j].pop(0)
        trails_y[j].pop(0)
    trail_lines[j].set_data(np.array(trails_x[j]), np.array(trails_y[j]))

    # Add to artists list
    artists.extend([lines[j], bobs[j], trail_lines[j]])

    # Update position vs time plots
    t_data = t_array[: frame + 1]
    theta_data = np.degrees(theta_all[j, : frame + 1])
    lines_pos[j].set_data(t_data, theta_data)
    points_pos[j].set_offsets([[t_current, np.degrees(theta)]])
    artists.extend([lines_pos[j], points_pos[j]])

    # Update velocity vs time plots
    omega_data = omega_all[j, : frame + 1]
    lines_vel[j].set_data(t_data, omega_data)
    points_vel[j].set_offsets([[t_current, omega]])
    artists.extend([lines_vel[j], points_vel[j]])

    # Update phase space plots
    lines_phase[j].set_data(theta_data, omega_data)
    points_phase[j].set_offsets([[np.degrees(theta), omega]])
    artists.extend([lines_phase[j], points_phase[j]])

    # Update info text if shown
    if show_info:
        energy = calculate_energy(theta, omega)
        gamma_val = gamma_list[j]

        if gamma_val == 0.0:
            title = "UNDAMPED ($\\gamma=0$)"
        else:
            title = f"DAMPED ($\\gamma={gamma_val:.2f}$)"

        info_display = (
            f"{title}\\n"
            f"{'-' * 18}\\n"
            f"$\\theta$: {np.degrees(theta):+.2f}$^\\circ\\n"
            f"$\\omega$: {omega:+.3f} rad/s\\n"
            f"$E$: {energy:.3f} J\\n"
            f"$E/E_0$: {energy / initial_energy:.4f}"
        )
        info_texts[j].set_text(info_display)
        artists.append(info_texts[j])

ax_pendulum.set_title(
    f"Time: {t_current:.2f} s | Period: {completed_periods:.2f}/{num_periods}",
    fontsize=12,

```

```

        pad=10,
        fontweight="bold",
    )

    return tuple(artists)

anim = FuncAnimation(
    fig,
    animate_frame,
    frames=num_frames,
    interval=int(1000 / fps),
    blit=False,
    repeat=False,
)

# Save animation if requested
if save_animation:
    if filename is None:
        gamma_str_file = "_".join([f"{g:.2f}" for g in gamma_list])
        filename = (
            f"multi_pendulum_analysis_L{L}_theta{theta_0_deg:.0f}"
            f"deg_gamma{gamma_str_file}_{num_periods}periods.gif"
        )

    save_dir = "OUTPUTS/ANIMATIONS"
    os.makedirs(save_dir, exist_ok=True)
    filepath = os.path.join(save_dir, filename)

    try:
        print("Saving animation... This may take a moment.")
        anim.save(filepath, writer="ffmpeg", fps=fps, codec="gif", dpi=80)
        print(f"Animation saved as: {filepath}")
    except Exception as e:
        print(f"Error saving animation: {e}")

    plt.close(fig)
else:
    plt.show()

print("\nMulti-Pendulum Comparison Summary:")
print("=" * 60)
print(f"Number of pendulums: {num_pendulums}")
print(f"Length: {L} m")
print(f"Initial angle: {theta_0_deg:.1f}°")
print(f"Period (small-angle approx): {T_linear:.3f} s")
print(f"Period (nonlinear, actual): {T:.3f} s")
print(f"Period increase due to nonlinearity: {100 * (period_ratio - 1):.2f}%")
print(f"Total simulation time: {total_time:.2f} s")
print(f"Number of periods: {num_periods}")
print(f"\nDamping coefficients:  $\gamma$  = {gamma_list}")
print("\nFinal Results for Each Pendulum:")
print("-" * 60)
for j in range(num_pendulums):
    gamma_val = gamma_list[j]
    final_angle = np.degrees(theta_all[j, -1])
    energy_final = calculate_energy(theta_all[j, -1], omega_all[j, -1])
    energy_retention = 100 * energy_final / initial_energy

    print(f"Pendulum {j + 1} ( $\gamma$ = {gamma_val:.3f}):")
    print(f"  Final angle: {final_angle:+.2f}° (initial: {theta_0_deg:.1f}°)")
    print(f"  Energy retention: {energy_retention:.1f}%")

return anim

```

```

def main():
    """Main function to run the multi-pendulum animation with user inputs."""

    print("Multi-Pendulum Damping Analysis Animation")
    print("=" * 50)

    # Default parameters
    defaults = {
        "L": 1.0,
        "G": 9.81,
        "theta_0_deg": 20.0,
        "gamma_list": [0.0, 0.25, 0.5],
        "num_periods": 5,
        "fps": 30,
    }

    use_defaults = input("Use default parameters? (y/n): ").strip().lower()

    if use_defaults == "y":
        params = defaults.copy()
    else:
        params = {}
        try:
            params["L"] = float(
                input(f"Enter pendulum length in meters [{defaults['L']}]: ")
                or defaults["L"]
            )
            params["G"] = float(
                input(f"Enter gravitational acceleration in m/s2 [{defaults['G']}]: ")
                or defaults["G"]
            )
            params["theta_0_deg"] = float(
                input(f"Enter initial angle in degrees [{defaults['theta_0_deg']}]: ")
                or defaults["theta_0_deg"]
            )

            # Get gamma list
            gamma_input = input(
                (
                    "Enter damping coefficients as comma-separated values (e.g., 0.0,0.
↪25,0.5) "
                    f"[{','.join(map(str, defaults['gamma_list']))}]: "
                )
            ).strip()
            if gamma_input:
                params["gamma_list"] = [
                    float(g.strip()) for g in gamma_input.split(",")
                ]
            else:
                params["gamma_list"] = defaults["gamma_list"]

            params["num_periods"] = int(
                input(
                    f"Enter number of periods to simulate [{defaults['num_periods']}]: "
                )
                or defaults["num_periods"]
            )
            params["fps"] = int(
                input(f"Enter frames per second [{defaults['fps']}]: ")
                or defaults["fps"]
            )

```

```

except ValueError:
    print("Invalid input, using default parameters.")
    params = defaults.copy()

save_anim = input("Save animation as GIF? (y/n): ").strip().lower() == "y"
filename = None
if save_anim:
    filename = input("Enter filename (or press Enter for auto-generated): ").strip()
    if filename and not filename.endswith(".gif"):
        filename += ".gif"
    if not filename:
        filename = None

print("\nRunning simulation with parameters:")
for key, value in params.items():
    print(f"  {key}: {value}")
print(f"  save_animation: {save_anim}")

animation = animate_damped_vs_undamped_pendulum_with_plots(
    L=params["L"],
    G=params["G"],
    theta_0_deg=params["theta_0_deg"],
    gamma_list=params["gamma_list"],
    num_periods=params["num_periods"],
    fps=params["fps"],
    save_animation=save_anim,
    filename=filename,
)

return animation

if __name__ == "__main__":
    anim = main()

```

See the animation in the GitHub repository: [Animation 3](#)

5. Mass-Spring System with Damping

The **mass-spring-damper system** is a fundamental model in classical mechanics, widely used in engineering and physics to study: - Mechanical vibrations - Vehicle suspension systems - Seismic isolation in buildings - Control systems and signal processing

The system consists of three components:

1. **Mass** (m): Provides inertia
2. **Spring** (k): Provides restoring force
3. **Damper** (c): Provides resistance proportional to velocity

5.1 Theory and RK4 Implementation

1. Physical System Overview

A **mass-spring-damper system** consists of three fundamental components:

1. **Mass** (m): Inertial resistance to acceleration
2. **Spring** (k): Restoring force proportional to displacement
3. **Damper** (c): Dissipative force proportional to velocity

This is one of the most important systems in physics and engineering, used to model: - Vehicle suspensions - Building seismic isolation - Mechanical vibrations - Control systems

2. Equation of Motion (Newton's Second Law)

The forces acting on the mass are:

$$\begin{aligned}F_{\text{spring}} &= -kx & (\text{Hooke's Law}) \\F_{\text{damper}} &= -c\dot{x} & (\text{Viscous damping}) \\F_{\text{external}} &= F(t) & (\text{Applied force})\end{aligned}$$

Applying Newton's second law ($\sum F = ma$):

$$m\ddot{x} + c\dot{x} + kx = F(t)$$

Or in standard form:

$$\ddot{x} + 2\zeta\omega_n\dot{x} + \omega_n^2x = \frac{F(t)}{m}$$

where: - $\omega_n = \sqrt{\frac{k}{m}}$ is the **natural frequency** (undamped) - $\zeta = \frac{c}{2\sqrt{km}}$ is the **damping ratio**

3. Converting to First-Order System

To apply RK4, we must convert the 2nd-order ODE to a **system of first-order ODEs**:

Let:

$$\begin{aligned}x_1 &= x & (\text{displacement}) \\x_2 &= \dot{x} & (\text{velocity})\end{aligned}$$

Then:

$$\begin{aligned}\frac{dx_1}{dt} &= x_2 \\ \frac{dx_2}{dt} &= \frac{F(t) - cx_2 - kx_1}{m}\end{aligned}$$

In vector form with state vector $\mathbf{y} = \begin{bmatrix} x \\ v \end{bmatrix}$:

$$\frac{d\mathbf{y}}{dt} = \mathbf{f}(t, \mathbf{y}) = \begin{bmatrix} v \\ \frac{F(t) - cv - kx}{m} \end{bmatrix}$$

This is implemented in the code as:

```
def mass_spring_damper_system(t, state, m, c, k, forcing_func):
    x, v = state
    F_t = forcing_func(t)
    dxdt = v
    dvdt = (F_t - c * v - k * x) / m
    return np.array([dxdt, dvdt])
```

4. RK4 Implementation for Mass-Spring-Damper

The RK4 step formula for the coupled system is:

$$\begin{aligned}\mathbf{k}_1 &= h \cdot \mathbf{f}(t_n, \mathbf{y}_n) \\ \mathbf{k}_2 &= h \cdot \mathbf{f}\left(t_n + \frac{h}{2}, \mathbf{y}_n + \frac{\mathbf{k}_1}{2}\right) \\ \mathbf{k}_3 &= h \cdot \mathbf{f}\left(t_n + \frac{h}{2}, \mathbf{y}_n + \frac{\mathbf{k}_2}{2}\right) \\ \mathbf{k}_4 &= h \cdot \mathbf{f}(t_n + h, \mathbf{y}_n + \mathbf{k}_3)\end{aligned}$$

The final update is given by:

$$\mathbf{y}_{n+1} = \mathbf{y}_n + \frac{1}{6}(\mathbf{k}_1 + 2\mathbf{k}_2 + 2\mathbf{k}_3 + \mathbf{k}_4)$$

Breaking down the components:

$$\begin{aligned}
k_{1,x} &= h \cdot v_n \\
k_{1,v} &= h \cdot \frac{F(t_n) - cv_n - kx_n}{m} \\
k_{2,x} &= h \cdot \left(v_n + \frac{k_{1,v}}{2} \right) \\
k_{2,v} &= h \cdot \frac{F(t_n + h/2) - c(v_n + k_{1,v}/2) - k(x_n + k_{1,x}/2)}{m}
\end{aligned}$$

Similarly for k_3 and k_4 .

Final update:

$$\begin{aligned}
x_{n+1} &= x_n + \frac{h}{6}(k_{1,x} + 2k_{2,x} + 2k_{3,x} + k_{4,x}) \\
v_{n+1} &= v_n + \frac{h}{6}(k_{1,v} + 2k_{2,v} + 2k_{3,v} + k_{4,v})
\end{aligned}$$

Code implementation:

```
def rk4_step(t, y, h, m, c, k, forcing_func):
    k1 = h * mass_spring_damper_system(t, y, m, c, k, forcing_func)
    k2 = h * mass_spring_damper_system(t + h/2, y + k1/2, m, c, k, forcing_func)
    k3 = h * mass_spring_damper_system(t + h/2, y + k2/2, m, c, k, forcing_func)
    k4 = h * mass_spring_damper_system(t + h, y + k3, m, c, k, forcing_func)
    return y + (k1 + 2*k2 + 2*k3 + k4) / 6
```

5. Damping Classification

The **damping ratio** ζ determines the system's behavior:

ζ Value	Classification	Behavior	Equation
$\zeta = 0$	Undamped	Oscillates indefinitely	$x(t) = A \cos(\omega_n t + \phi)$
$0 < \zeta < 1$	Underdamped	Oscillates with decay	$x(t) = Ae^{-\zeta\omega_n t} \cos(\omega_d t + \phi)$
$\zeta = 1$	Critically Damped	No oscillation, fastest decay	$x(t) = (A + Bt)e^{-\omega_n t}$
$\zeta > 1$	Overdamped	Slow decay, no oscillation	$x(t) = A_1 e^{r_1 t} + A_2 e^{r_2 t}$

where $\omega_d = \omega_n \sqrt{1 - \zeta^2}$ is the **damped frequency** (underdamped case).

Critical damping coefficient:

$$c_{\text{crit}} = 2\sqrt{km}$$

Your code calculates this:

```
omega_n = np.sqrt(k / m)          # Natural frequency
zeta = c / (2 * np.sqrt(k * m))   # Damping ratio

if zeta < 1:
    omega_d = omega_n * np.sqrt(1 - zeta**2)  # Damped frequency
```

6. Underdamped Response (Most Common)

For $0 < \zeta < 1$, the solution is:

$$x(t) = Ae^{-\zeta\omega_n t} \cos(\omega_d t + \phi)$$

where:

1. **Envelope:** $A(t) = A_0 e^{-\zeta\omega_n t}$ (exponential decay)
2. **Damped frequency:** $\omega_d = \omega_n \sqrt{1 - \zeta^2}$
3. **Decay time constant:** $\tau = \frac{1}{\zeta\omega_n}$

Energy dissipation:

$$E(t) = \frac{1}{2}mv^2 + \frac{1}{2}kx^2 = E_0 e^{-2\zeta\omega_n t}$$

7. Complete RK4 Integration

The `rk4_mass_spring_damper` function integrates over time:

```
def rk4_mass_spring_damper(m, c, k, x0, v0, t0, tf, h, forcing_func):
    # Initialize time and state arrays
    n_steps = int((tf - t0) / h) + 1
    t_vals = np.linspace(t0, tf, n_steps)
    x_vals = np.zeros(n_steps)
    v_vals = np.zeros(n_steps)

    # Set initial conditions
    x_vals[0] = x0
    v_vals[0] = v0
    state = np.array([x0, v0])

    # Step through time
    for i in range(n_steps - 1):
        state = rk4_step(t_vals[i], state, h, m, c, k, forcing_func)
        x_vals[i + 1] = state[0]
        v_vals[i + 1] = state[1]

    return t_vals, x_vals, v_vals
```

This iteratively advances the solution using the RK4 formula at each time step.

8. Forced Oscillations

The code supports external forcing via `forcing_func`:

Sinusoidal forcing:

$$F(t) = F_0 \sin(\omega_f t)$$

```
def sinusoidal_forcing(t, amplitude=1.0, frequency=1.0):
    return amplitude * np.sin(2 * np.pi * frequency * t)
```

Step forcing:

$$F(t) = \begin{cases} 0 & t < t_s \\ F_0 & t \geq t_s \end{cases}$$

Impulse forcing:

$$F(t) = \begin{cases} F_0 & t_0 \leq t \leq t_0 + \Delta t \\ 0 & \text{otherwise} \end{cases}$$

The RK4 method naturally incorporates these via the `forcing_func` parameter in the derivative calculation.

9. Comparison Table: Key Parameters

Parameter	Symbol	Formula	Physical Meaning
Natural frequency	ω_n	$\sqrt{k/m}$	Oscillation rate (undamped)
Damping ratio	ζ	$c/(2\sqrt{km})$	Relative damping strength
Damped frequency	ω_d	$\omega_n \sqrt{1 - \zeta^2}$	Oscillation rate (damped)
Period (undamped)	T_0	$2\pi/\omega_n$	Time per undamped cycle
Period (damped)	T_d	$2\pi/\omega_d$	Time per damped cycle
Critical damping	c_c	$2\sqrt{km}$	Boundary between oscillation and non-oscillation
Time constant	τ	$1/(\zeta\omega_n)$	Envelope decay rate

RK4 Solver for Mass-Spring-Damper System

The following functions implement: 1. The RK4 numerical solver for the mass-spring-damper equation of motion 2. Analysis functions to determine system properties (natural frequency, damping ratio, damping type) 3. External forcing functions (step, sinusoidal, impulse)

```
[10]: def mass_spring_damper_system(t, state, m, c, k, forcing_func):
    """
    Define the mass-spring-damper system as first-order ODEs.

    The second-order equation:  $m * \frac{d^2x}{dt^2} + c * \frac{dx}{dt} + k * x = F(t)$ 
    is converted to first-order system:
     $\frac{dx}{dt} = v$ 
     $\frac{dv}{dt} = (F(t) - c*v - k*x) / m$ 

    Args:
        t: Current time
        state: State vector [x, v] where x is position, v is velocity
        m: Mass coefficient
        c: Damping coefficient
        k: Stiffness coefficient
        forcing_func: External forcing function F(t)

    Returns:
        Derivative vector [dx/dt, dv/dt]
    """
    x, v = state
    F_t = forcing_func(t)

    dxdt = v
    dvdt = (F_t - c * v - k * x) / m

    return np.array([dxdt, dvdt])

def rk4_step(t, y, h, m, c, k, forcing_func):
    """
    Perform one RK4 step for the mass-spring-damper system.

    Args:
        t: Current time
        y: Current state [x, v]
        h: Step size
        m, c, k: System parameters
        forcing_func: External forcing function

    Returns:
        Next state [x_next, v_next]
    """
    k1 = h * mass_spring_damper_system(t, y, m, c, k, forcing_func)
    k2 = h * mass_spring_damper_system(t + h / 2, y + k1 / 2, m, c, k, forcing_func)
    k3 = h * mass_spring_damper_system(t + h / 2, y + k2 / 2, m, c, k, forcing_func)
    k4 = h * mass_spring_damper_system(t + h, y + k3, m, c, k, forcing_func)

    return y + (k1 + 2 * k2 + 2 * k3 + k4) / 6

def rk4_mass_spring_damper(m, c, k, x0, v0, t0, tf, h, forcing_func):
    """
    Solve mass-spring-damper system using RK4 method.

    Args:
        m, c, k: System parameters
        x0, v0: Initial conditions
        t0, tf: Time span
        h: Step size
        forcing_func: External forcing function

    Returns:
```

```

    Tuple of (time_array, position_array, velocity_array)
    """
    # Initialize arrays
    n_steps = int((tf - t0) / h) + 1
    t_vals = np.linspace(t0, tf, n_steps)
    x_vals = np.zeros(n_steps)
    v_vals = np.zeros(n_steps)

    # Set initial conditions
    x_vals[0] = x0
    v_vals[0] = v0
    state = np.array([x0, v0])

    # Integrate using RK4
    for i in range(n_steps - 1):
        state = rk4_step(t_vals[i], state, h, m, c, k, forcing_func)
        x_vals[i + 1] = state[0]
        v_vals[i + 1] = state[1]

    return t_vals, x_vals, v_vals

def no_forcing(t):
    """No external forcing function."""
    return 0.0

def step_forcing(t, step_time=0.0, amplitude=1.0):
    """Step function forcing."""
    return amplitude if t >= step_time else 0.0

def sinusoidal_forcing(t, amplitude=1.0, frequency=1.0):
    """Sinusoidal forcing function."""
    return amplitude * np.sin(2 * np.pi * frequency * t)

def impulse_forcing(t, impulse_time=0.0, amplitude=10.0, duration=0.1):
    """Impulse forcing function."""
    if impulse_time <= t <= impulse_time + duration:
        return amplitude
    return 0.0

def analyze_system_properties(m, c, k):
    """
    Analyze system properties: natural frequency, damping ratio, etc.

    Args:
        m, c, k: System parameters

    Returns:
        Dictionary with system properties
    """
    omega_n = np.sqrt(k / m) # Natural frequency
    zeta = c / (2 * np.sqrt(k * m)) # Damping ratio

    properties = {
        "omega_n": omega_n,
        "zeta": zeta,
        "period": 2 * np.pi / omega_n if omega_n > 0 else float("inf"),
    }

```

```

# Classify damping
if zeta < 1:
    properties["damping_type"] = "underdamped"
    properties["omega_d"] = omega_n * np.sqrt(1 - zeta**2) # Damped frequency

# Treat values very close to 1 as critically damped (within tolerance)
tol = 1e-3
if abs(zeta - 1.0) <= tol:
    properties["damping_type"] = "critically damped"
    # If omega_d was set in the underdamped branch, remove it for clarity
    properties.pop("omega_d", None)
elif zeta > 1.0:
    properties["damping_type"] = "overdamped"
else:
    # zeta < 1 (and not near 1) - already marked as underdamped above
    pass

return properties

```

5.2 Animating the Damped Mass-Spring System

This animation compares undamped and damped horizontal mass-spring systems side by side: - The **undamped system** oscillates with constant amplitude following $x(t) = A \cos(\omega_n t)$ - The **damped system** is solved numerically using RK4, showing the effects of energy dissipation - Both systems include realistic spring visualization with dynamic coil spacing

```

[ ]: def animate_mass_spring_shm(
    m=1.0,
    c=0.5,
    k=10.0,
    x0=0.1,
    v0=0.0,
    oscillations=5.0,
    fps=30,
    save_animation=False,
    filename=None,
    num_coils=8,
    vertical_separation=0.15,
):
    """
    Animate undamped and damped horizontal mass-spring systems simultaneously.

    The undamped system oscillates with  $x(t) = x0 \cdot \cos(\omega_n t)$ 
    The damped system is solved using RK4 method

    Parameters:
        m: Mass
        c: Damping coefficient
        k: Stiffness
        x0: Initial displacement for damped system
        v0: Initial velocity for damped system
        oscillations: Number of oscillations to show
        fps: Frames per second
        save_animation: Whether to save as GIF
        filename: Output filename
        num_coils: Number of coils in spring visualization
        vertical_separation: Vertical distance between the two systems
    """

    # System properties
    omega_n = np.sqrt(k / m)
    zeta = c / (2 * np.sqrt(k * m))
    T = 2 * np.pi / omega_n

```

```

# Time parameters
duration = oscillations * T
num_frames = int(round(duration * fps)) + 1
t_array = np.linspace(0.0, duration, num_frames, endpoint=True)

# Undamped system:  $x(t) = A \cos(\omega_n t)$ 
x_undamped = x0 * np.cos(omega_n * t_array)

# Damped system: solve using RK4
h = duration / (num_frames - 1)
_, x_damped, _ = rk4_mass_spring_damper(
    m, c, k, x0, v0, 0.0, duration, h, no_forcing
)

# Analyze damping type
properties = analyze_system_properties(m, c, k)
damping_type = properties["damping_type"]

# Create figure
fig, ax = plt.subplots(figsize=(12, 7))
fig.subplots_adjust(top=0.85, bottom=0.1, left=0.05, right=0.8)

# Update supptitle
fig.suptitle(
    f"Undamped vs Damped Mass-Spring Systems\n"
    f"$m$={m} kg, $k$={k} N/m, $c$={c} N·s/m | $\omega_0$={omega_n:.2f} rad/s, "
    f"$\zeta$={zeta:.3f} ({damping_type})",
    fontsize=15,
    fontweight="bold",
)

# Set up coordinate system
wall_x = -1.5 * x0

ax.set_xlim(wall_x - 0.02, 1.6 * abs(x0))
ax.set_ylim(-vertical_separation - 0.15, 0.15)
ax.set_xlabel("Position (m)", fontsize=12)
ax.set_ylabel("")
ax.set_yticks([])

# Draw wall
ax.axvline(wall_x, color="black", linewidth=15, zorder=4)

# Draw equilibrium lines
ax.axvline(0, color="gray", linestyle="--", alpha=0.3, linewidth=1)
ax.axvline(x0, color="gray", linestyle="--", alpha=0.3, linewidth=1)
ax.axvline(-x0, color="gray", linestyle="--", alpha=0.3, linewidth=1)
ax.axhline(0, color="gray", alpha=0.3, linewidth=0.5)
ax.axhline(-vertical_separation, color="gray", alpha=0.3, linewidth=0.5)

# Initialize artists for undamped system
undamped_mass = ax.scatter(
    [],
    [],
    s=400,
    c="blue",
    marker="s",
    edgecolor="darkblue",
    linewidth=2,
    label="Undamped",
    zorder=5,
)

(undamped_spring,) = ax.plot([], [], color="blue", lw=2, alpha=0.8, zorder=3)

```

```

# Initialize artists for damped system
damped_mass = ax.scatter(
    [],
    [],
    s=400,
    c="red",
    marker="s",
    edgecolor="darkred",
    linewidth=2,
    label=f"Damped (c={c})",
    zorder=5,
)
(damped_spring,) = ax.plot([], [], color="red", lw=2, alpha=0.8, zorder=3)

# Add legend
ax.legend(
    loc="upper right",
    fontsize=10,
    fancybox=True,
    shadow=True,
    facecolor="white",
    edgecolor="darkgray",
    markerscale=0.6,
    labelspacing=1.25,
)

# Text boxes for displaying system information
undamped_text = ax.text(
    1.02,
    0.98,
    "",
    transform=ax.transAxes,
    fontsize=9,
    va="top",
    ha="left",
    bbox=dict(
        boxstyle="round,pad=0.5",
        facecolor="lightblue",
        alpha=0.8,
        edgecolor="darkblue",
    ),
    family="monospace",
)

damped_text = ax.text(
    1.02,
    0.78,
    "",
    transform=ax.transAxes,
    fontsize=9,
    va="top",
    ha="left",
    bbox=dict(
        boxstyle="round,pad=0.5",
        facecolor="lightcoral",
        alpha=0.8,
        edgecolor="darkred",
    ),
    family="monospace",
)

def draw_spring_horizontal(start_x, end_x, num_coils, radius=0.03, y_center=0.0):

```

```

"""Draw a horizontal spring from start_x to end_x"""
if abs(end_x - start_x) < 1e-9:
    return np.array([start_x]), np.array([y_center])

t = np.linspace(0, num_coils * 2 * np.pi, 300)
x_spring = np.linspace(start_x, end_x, 300)
y_spring = y_center + radius * np.sin(t)

return x_spring, y_spring

def init():
    """Initialize animation"""
    undamped_mass.set_offsets(np.array([[np.nan, 0.0]]))
    undamped_spring.set_data([], [])
    damped_mass.set_offsets(np.array([[np.nan, -vertical_separation]]))
    damped_spring.set_data([], [])
    return undamped_mass, undamped_spring, damped_mass, damped_spring

def update(frame):
    """Update animation for each frame"""
    # Current time
    t = t_array[frame]
    num_osc = min(t / T, oscillations)

    # Undamped system
    x_u = x_undamped[frame]
    x_spring_u, y_spring_u = draw_spring_horizontal(
        wall_x, x_u, num_coils, radius=0.03, y_center=0
    )
    undamped_spring.set_data(x_spring_u, y_spring_u)
    undamped_mass.set_offsets([[x_u, 0]])

    # Calculate undamped spring deformation
    spring_deform_u = abs(x_u)

    # Damped system
    x_d = x_damped[frame]
    x_spring_d, y_spring_d = draw_spring_horizontal(
        wall_x, x_d, num_coils, radius=0.03, y_center=-vertical_separation
    )
    damped_spring.set_data(x_spring_d, y_spring_d)
    damped_mass.set_offsets([[x_d, -vertical_separation]])

    # Calculate damped spring deformation
    spring_deform_d = abs(x_d)

    # Update title with only time and oscillation
    ax.set_title(
        f"Time: {t:.2f} s | Oscillation: {num_osc:.2f}/{oscillations}",
        fontsize=12,
        fontweight="bold",
        pad=10,
    )

    # Update undamped text box
    undamped_info = (
        "UNDAMPED SYSTEM\n"
        f"{'-' * 25}\n"
        f"Position:      {x_u:+.4f} m\n"
        f"Spring def:    {spring_deform_u:.4f} m\n"
        f"Amplitude:     {x0:.4f} m\n"
        f"Frequency:     {omega_n:.2f} rad/s"
    )

```

```

undamped_text.set_text(undamped_info)

# Update damped text box
damped_info = (
    "DAMPED SYSTEM\n"
    f"{'-' * 25}\n"
    f"Position:      {x_d:+.4f} m\n"
    f"Spring def:     {spring_deform_d:.4f} m\n"
    f"Damping:        {c} N·s/m\n"
    f"Damp. ratio:    {zeta:.3f}"
)
damped_text.set_text(damped_info)

return (
    undamped_mass,
    undamped_spring,
    damped_mass,
    damped_spring,
    undamped_text,
    damped_text,
)

# Create animation
anim = FuncAnimation(
    fig,
    update,
    frames=num_frames,
    init_func=init,
    blit=False,
    interval=int(1000.0 / fps),
    repeat=False,
)

# Save or show
if save_animation:
    if filename is None:
        filename = f"undamped_vs_damped_m{m}_k{k}_c{c}.gif"

    save_dir = "OUTPUTS/ANIMATIONS"
    os.makedirs(save_dir, exist_ok=True)
    filepath = os.path.join(save_dir, filename)

    try:
        print(f"Saving animation to {os.path.abspath(filepath)}...")
        anim.save(filepath, writer="ffmpeg", fps=fps, codec="gif", dpi=150)
        print(f"Animation saved as: {filepath}")
    except Exception as e:
        print(f"Error saving animation: {e}")
    plt.close(fig)
else:
    plt.show()

return anim

def main():
    """Main function to run the mass-spring-damper animation with user inputs."""

    print("\n" + "=" * 60)
    print("  Mass-Spring-Damper Animation: Undamped vs Damped")
    print("=" * 60)
    print("\nThis animation compares undamped and damped oscillations")
    print("of a horizontal mass-spring system.\n")

```

```

defaults = {
    "m": 1.0,
    "k": 10.0,
    "c": 0.5,
    "x0": 0.1,
    "oscillations": 5.0,
    "fps": 30,
}

use_defaults = input("Use default parameters? (y/n): ").strip().lower()

if use_defaults == "y":
    params = defaults.copy()
else:
    params = {}
    try:
        params["m"] = float(
            input(f"Enter mass (kg) [{defaults['m']}]: ") or defaults["m"]
        )
        params["k"] = float(
            input(f"Enter stiffness (N/m) [{defaults['k']}]: ") or defaults["k"]
        )
        params["c"] = float(
            input(f"Enter damping coefficient (N·s/m) [{defaults['c']}]: ")
            or defaults["c"]
        )
        params["x0"] = float(
            input(f"Enter initial displacement (m) [{defaults['x0']}]: ")
            or defaults["x0"]
        )
        params["oscillations"] = float(
            input(
                f"Enter number of periods to display [{defaults['oscillations']}]: "
            )
            or defaults["oscillations"]
        )
        params["fps"] = int(
            input(f"Enter frames per second [{defaults['fps']}]: ")
            or defaults["fps"]
        )
    except ValueError:
        print("Invalid input, using default parameters.")
        params = defaults.copy()

save_anim = input("Save animation as GIF? (y/n): ").strip().lower() == "y"
filename = None
if save_anim:
    filename = input("Enter filename (or press Enter for auto-generated): ").strip()
    if filename and not filename.endswith(".gif"):
        filename += ".gif"
    if not filename:
        filename = None

print("\nRunning simulation with parameters:")
for key, value in params.items():
    print(f"  {key}: {value}")
print(f"  save_animation: {save_anim}")

animation = animate_mass_spring_shm(
    m=params["m"],
    c=params["c"],
    k=params["k"],

```

```

        x0=params["x0"],
        oscillations=params["oscillations"],
        fps=params["fps"],
        save_animation=save_anim,
        filename=filename,
    )

    return animation

if __name__ == "__main__":
    anim = main()

```

See the animation in the GitHub repository: [Animation 4](#)

5.3 Animating for Multiple Damping Cases

This comprehensive animation displays multiple mass-spring systems with different damping coefficients simultaneously, enabling direct comparison of all damping regimes:

Damping Regimes: - **Undamped** ($\zeta = 0$): Pure harmonic oscillation - **Underdamped** ($0 < \zeta < 1$): Oscillatory with exponential decay - **Critically Damped** ($\zeta = 1$): Optimal return to equilibrium (no overshoot, fastest settling) - **Overdamped** ($\zeta > 1$): Slow, non-oscillatory return to equilibrium

where $\zeta = \frac{c}{2\sqrt{km}}$ is the damping ratio.

```

[ ]: def animate_multiple_damping_cases(
    m=1.0,
    c_values=[0, 0.5],
    k=10.0,
    x0=0.1,
    v0=0.0,
    oscillations=5.0,
    fps=30,
    save_animation=False,
    filename=None,
    num_coils=8,
    vertical_separation=0.15,
    forcing_func=None,
    forcing_name="none",
):
    """
    Animate multiple damping cases (undamped, underdamped, critically damped, overdamped)
    ↪ simultaneously.

    Parameters:
        m: Mass
        c_values: List of damping coefficients (0 for undamped, then underdamped,
        ↪ critical, overdamped)
        k: Stiffness
        x0: Initial displacement
        v0: Initial velocity
        oscillations: Number of oscillations to show
        fps: Frames per second
        save_animation: Whether to save as GIF
        filename: Output filename
        num_coils: Number of coils in spring visualization
        vertical_separation: Vertical distance between systems
        forcing_func: External forcing function
        forcing_name: Name of forcing function for display
    """

    if forcing_func is None:
        forcing_func = no_forcing

```

```

# System properties
omega_n = np.sqrt(k / m)
T = 2 * np.pi / omega_n
c_critical = 2 * np.sqrt(k * m)

# Time parameters
duration = oscillations * T
num_frames = int(round(duration * fps)) + 1
t_array = np.linspace(0.0, duration, num_frames, endpoint=True)
h = duration / (num_frames - 1)

# Solve for each damping case
num_cases = len(c_values)
x_solutions = []
case_labels = []
case_colors = ["blue", "green", "orange", "red", "magenta", "violet"]

for i, c in enumerate(c_values):
    if c == 0:
        # Undamped case: analytical solution
        x_vals = x0 * np.cos(omega_n * t_array)
        case_labels.append("Undamped (c=0)")
    else:
        # Damped cases: numerical solution
        _, x_vals, _ = rk4_mass_spring_damper(
            m, c, k, x0, v0, 0.0, duration, h, forcing_func
        )
        props = analyze_system_properties(m, c, k)
        case_labels.append(f"{props['damping_type'].title()} (c={c:.2f})")

    x_solutions.append(x_vals)

fig_height = 4 + num_cases * 1.5
fig, ax = plt.subplots(figsize=(12, fig_height))
fig.subplots_adjust(top=0.88, bottom=0.08, left=0.05, right=0.78)

c_str = ", ".join([str(c) for c in c_values])
fig.suptitle(
    f"Mass-Spring-Damper System: Multiple Damping Cases\n"
    f"$m$={m} kg, $k$={k} N/m, $c_{crit}$={c_critical:.2f} N.s/m, "
    f"$\omega_0$={omega_n:.2f} rad/s, Forcing: {forcing_name}",
    fontsize=14,
    fontweight="bold",
)

# Set up coordinate system
wall_x = -1.5 * x0
total_height = (num_cases - 1) * vertical_separation

ax.set_xlim(wall_x - 0.02, 1.6 * abs(x0))
ax.set_ylim(-total_height - 0.15, 0.15)
ax.set_xlabel("Position (m)", fontsize=12)
ax.set_ylabel("")
ax.set_yticks([])

ax.axvline(wall_x, color="black", linewidth=15, zorder=4)

ax.axvline(0, color="gray", linestyle="--", alpha=0.3, linewidth=1)
ax.axvline(x0, color="gray", linestyle="--", alpha=0.3, linewidth=1)
ax.axvline(-x0, color="gray", linestyle="--", alpha=0.3, linewidth=1)

for i in range(num_cases):

```

```

        y_pos = -i * vertical_separation
        ax.axhline(y_pos, color="gray", alpha=0.3, linewidth=0.5)

masses = []
springs = []
text_boxes = []

for i, (c, label) in enumerate(zip(c_values, case_labels)):
    color = case_colors[i % len(case_colors)]
    edge_color = f"dark{color}"

    # Mass
    mass = ax.scatter(
        [],
        [],
        s=400,
        c=color,
        marker="s",
        edgecolor=edge_color,
        linewidth=2,
        label=label,
        zorder=5,
    )
    masses.append(mass)

    # Spring
    (spring,) = ax.plot([], [], color=color, lw=2, alpha=0.8, zorder=3)
    springs.append(spring)

    # Text box
    y_text_pos = 0.98 - i * 0.18
    text = ax.text(
        1.02,
        y_text_pos,
        "",
        transform=ax.transAxes,
        fontsize=9,
        va="top",
        ha="left",
        bbox=dict(
            boxstyle="round,pad=0.4",
            facecolor=color,
            alpha=0.25,
            edgecolor="black",
            linewidth=1.5,
        ),
        family="monospace",
    )
    text_boxes.append(text)

# Legend
ax.legend(
    loc="upper right",
    fontsize=9,
    fancybox=True,
    shadow=True,
    facecolor="white",
    edgecolor="darkgray",
    markerscale=0.5,
    labelspacing=1.1,
)

def draw_spring_horizontal(start_x, end_x, num_coils, radius=0.03, y_center=0.0):

```

```

"""Draw a horizontal spring from start_x to end_x"""
if abs(end_x - start_x) < 1e-9:
    return np.array([start_x]), np.array([y_center])

t = np.linspace(0, num_coils * 2 * np.pi, 300)
x_spring = np.linspace(start_x, end_x, 300)
y_spring = y_center + radius * np.sin(t)

return x_spring, y_spring

def init():
    """Initialize animation"""
    for i, (mass, spring) in enumerate(zip(masses, springs)):
        y_pos = -i * vertical_separation
        mass.set_offsets(np.array([[np.nan, y_pos]]))
        spring.set_data([], [])
    return tuple(masses + springs + text_boxes)

def update(frame):
    """Update animation for each frame"""
    # Current time
    t = t_array[frame]
    num_osc = min(t / T, oscillations)

    # Update title
    ax.set_title(
        f"Time: {t:.2f} s | Oscillation: {num_osc:.2f}/{oscillations}",
        fontsize=12,
        fontweight="bold",
        pad=10,
    )

    # Update each system
    for i, (c, x_vals, mass, spring, text) in enumerate(
        zip(c_values, x_solutions, masses, springs, text_boxes)
    ):
        y_pos = -i * vertical_separation
        x_current = x_vals[frame]

        # Update spring and mass
        x_spring, y_spring = draw_spring_horizontal(
            wall_x, x_current, num_coils, radius=0.03, y_center=y_pos
        )
        spring.set_data(x_spring, y_spring)
        mass.set_offsets([[x_current, y_pos]])

        # Calculate spring deformation
        spring_deform = abs(x_current)

        # Calculate damping ratio
        zeta = c / c_critical if c > 0 else 0

        # Update text box
        info = (
            f"{case_labels[i]}\n"
            f"{'-' * 25}\n"
            f"Position: {x_current:+.4f} m\n"
            f"Spring def: {spring_deform:.4f} m\n"
            f"Damping: {c:.3f} N·s/m\n"
            f"Damp ratio: {zeta:.3f}"
        )
        text.set_text(info)

```

```

        return tuple(masses + springs + text_boxes)

# Create animation
anim = FuncAnimation(
    fig,
    update,
    frames=num_frames,
    init_func=init,
    blit=False,
    interval=int(1000.0 / fps),
    repeat=False,
)

# Save or show
if save_animation:
    if filename is None:
        c_str = "_".join([f"c{c}" for c in c_values])
        filename = f"multi_damping_m{m}_k{k}_{c_str}.gif"

    save_dir = "OUTPUTS/ANIMATIONS"
    os.makedirs(save_dir, exist_ok=True)
    filepath = os.path.join(save_dir, filename)

    try:
        print(f"Saving animation to {os.path.abspath(filepath)}...")
        anim.save(filepath, writer="ffmpeg", fps=fps, codec="gif", dpi=120)
        print(f"Animation saved as: {filepath}")
    except Exception as e:
        print(f"Error saving animation: {e}")
    plt.close(fig)
else:
    plt.show()

return anim

def main():
    """Main function to run the multi-damping animation with user inputs."""

    print("\n" + "=" * 70)
    print("  Mass-Spring-Damper Animation: Multiple Damping Cases")
    print("=" * 70)
    print("\nThis animation compares multiple damping cases:")
    print("  - Undamped ( $c=0$ )")
    print("  - Underdamped ( $0 < c < c_{critical}$ )")
    print("  - Critically damped ( $c = c_{critical}$ )")
    print("  - Overdamped ( $c > c_{critical}$ )\n")

    # Default parameters
    defaults = {
        "m": 1.0,
        "k": 10.0,
        "c_values": [0, 0.5, 6.32, 7],
        "x0": 0.1,
        "v0": 0.0,
        "oscillations": 5.0,
        "fps": 30,
    }

    use_defaults = input("Use default parameters? (y/n) [y]: ").strip().lower()

    if use_defaults == "n":
        try:

```

```

print("\n--- System Parameters ---")
m = float(input("Mass (m) [1.0 kg]: ") or 1.0)
k = float(input("Stiffness (k) [10.0 N/m]: ") or 10.0)

# Calculate critical damping
c_critical = 2 * np.sqrt(k * m)
print(f"\nCritical damping coefficient: c_crit = {c_critical:.3f} N·s/m")

# Input damping coefficients
print("\n--- Damping Coefficients ---")
print("Enter damping coefficients (comma-separated).")
print("Include 0 for undamped case.")
print(
    f"Suggestions: 0 (undamped), {c_critical * 0.2:.2f} (underdamped), ↵
↵{c_critical:.2f} (critical), {c_critical * 1.5:.2f} (overdamped)"
)
c_input = input(
    f"c values [0, {c_critical * 0.2:.2f}, {c_critical:.2f}, {c_critical * 1.5:
↵.2f}]: "
).strip()
if c_input:
    c_values = [float(c.strip()) for c in c_input.split(",")]
else:
    c_values = [0, c_critical * 0.2, c_critical, c_critical * 1.5]

# Sort c_values to ensure proper ordering
c_values = sorted(c_values)

# Initial conditions
print("\n--- Initial Conditions ---")
x0 = float(input("Initial displacement/amplitude (x0) [0.1 m]: ") or 0.1)
v0 = float(input("Initial velocity v0 [0.0 m/s]: ") or 0.0)

print("\n--- Animation Parameters ---")
oscillations = float(input("Number of periods to display [5.0]: ") or 5.0)
fps = int(input("Frames per second [30]: ") or 30)

except ValueError as e:
    print(f"\nInvalid input: {e}")
    print("Using default parameters.")
    m = defaults["m"]
    k = defaults["k"]
    c_values = defaults["c_values"]
    x0 = defaults["x0"]
    v0 = defaults["v0"]
    oscillations = defaults["oscillations"]
    fps = defaults["fps"]
else:
    m = defaults["m"]
    k = defaults["k"]
    c_values = defaults["c_values"]
    x0 = defaults["x0"]
    v0 = defaults["v0"]
    oscillations = defaults["oscillations"]
    fps = defaults["fps"]

# Validate parameters
if m <= 0:
    print("Warning: Mass must be positive, setting to 1.0")
    m = 1.0
if k <= 0:
    print("Warning: Stiffness must be positive, setting to 10.0")
    k = 10.0

```

```

if any(c < 0 for c in c_values):
    print("Warning: Damping coefficients must be non-negative")
    c_values = [max(0, c) for c in c_values]

# Setting forcing function to None for non-driven case
forcing_func = None
forcing_name = "none"

save_anim = input("\nSave animation as GIF? (y/n) [n]: ").strip().lower() == "y"
filename = None
if save_anim:
    filename_input = input("Enter filename (or press Enter for default): ").strip()
    if filename_input:
        if not filename_input.endswith(".gif"):
            filename_input += ".gif"
        filename = filename_input

print("\n" + "-" * 70)
print("Animating with:")
print(f"  Mass (m):           {m} kg")
print(f"  Stiffness (k):       {k} N/m")
print(
    f"  Damping values:     {[float(np.round(c, 3)) for c in c_values if c > 0]} N·s/m"
)
print(f"  Initial disp (x0):  {x0} m")
print(f"  Initial vel (v0):   {v0} m/s")
print(f"  Periods:            {oscillations}")
print(f"  FPS:                {fps}")
print(f"  Forcing:            {forcing_name}")
print("-" * 70 + "\n")

anim = animate_multiple_damping_cases(
    m=m,
    c_values=c_values,
    k=k,
    x0=x0,
    v0=v0,
    oscillations=oscillations,
    fps=fps,
    save_animation=save_anim,
    filename=filename,
    forcing_func=forcing_func,
    forcing_name=forcing_name,
)

print("\nAnimation completed!")
return anim

if __name__ == "__main__":
    animation = main()

```

See the animation in the GitHub repository: [Animation 5](#)

5.4 Animation of Damped vs Undamped SHM with Plots

This is the most comprehensive visualization combining: - **Left panel**: Animated mass-spring systems for multiple damping cases - **Right panels**: Three real-time plots 1. **Position vs Time**: Shows how displacement varies over time 2. **Phase Space** (v vs x): Velocity-position trajectories 3. **Total Energy vs Time**: Demonstrates energy conservation (undamped) or dissipation (damped)

This visualization provides complete insight into the dynamics, energetics, and phase space behavior of oscillatory systems.

```
[ ]: def animate_multiple_damping_cases_with_plots(
    m=1.0,
    c_values=[0, 0.5],
    k=10.0,
    x0=0.1,
    v0=0.0,
    oscillations=5.0,
    fps=30,
    save_animation=False,
    filename=None,
    num_coils=8,
    vertical_separation=0.15,
    forcing_func=None,
    forcing_name="none",
):
    """
    Animate multiple damping cases (undamped, underdamped, critically damped, overdamped)
    ↪ simultaneously.

    Parameters:
        m: Mass
        c_values: List of damping coefficients (0 for undamped, then underdamped,
    ↪ critical, overdamped)
        k: Stiffness
        x0: Initial displacement
        v0: Initial velocity
        oscillations: Number of oscillations to show
        fps: Frames per second
        save_animation: Whether to save as GIF
        filename: Output filename
        num_coils: Number of coils in spring visualization
        vertical_separation: Vertical distance between systems
        forcing_func: External forcing function
        forcing_name: Name of forcing function for display
    """
    if forcing_func is None:
        forcing_func = no_forcing

    # System properties
    omega_n = np.sqrt(k / m)
    T = 2 * np.pi / omega_n
    c_critical = 2 * np.sqrt(k * m)

    # Time parameters
    duration = oscillations * T
    num_frames = int(round(duration * fps)) + 1
    t_array = np.linspace(0.0, duration, num_frames, endpoint=True)
    h = duration / (num_frames - 1)

    # Solve for each damping case
    num_cases = len(c_values)
    x_solutions = []
    v_solutions = []
    case_labels = []
    case_colors = ["blue", "green", "orange", "red", "magenta", "violet"]

    for i, c in enumerate(c_values):
        if c == 0:
            # Undamped case: analytical solution
            x_vals = x0 * np.cos(omega_n * t_array)
            v_vals = -x0 * omega_n * np.sin(omega_n * t_array)
            case_labels.append("Undamped (c=0)")
        else:

```

```

        # Damped cases: numerical solution
        _, x_vals, v_vals = rk4_mass_spring_damper(
            m, c, k, x0, v0, 0.0, duration, h, forcing_func
        )
        props = analyze_system_properties(m, c, k)
        case_labels.append(f"{props['damping_type'].title()} (c={c:.2f})")

    x_solutions.append(x_vals)
    v_solutions.append(v_vals)

# Calculate energy for all cases
def calculate_energy(x, v, mass=m, stiffness=k):
    """Calculate total mechanical energy"""
    kinetic = 0.5 * mass * v**2
    potential = 0.5 * stiffness * x**2
    return kinetic + potential

# Calculate energy arrays for all cases
energy_solutions = []
for x_vals, v_vals in zip(x_solutions, v_solutions):
    E_vals = calculate_energy(x_vals, v_vals)
    energy_solutions.append(E_vals)

# Create figure with GridSpec layout (3 rows, 2 columns)
fig = plt.figure(figsize=(14, 10))
gs = GridSpec(
    3,
    2,
    figure=fig,
    hspace=0.35,
    wspace=0.55,
    left=0.05,
    right=0.95,
    top=0.88,
    bottom=0.1,
    width_ratios=[1.5, 1],
)

# Left column: Mass-spring animation (spans all 3 rows)
ax = fig.add_subplot(gs[:, 0])

# Right column: Three plots
ax_position = fig.add_subplot(gs[0, 1]) # Position vs time
ax_phase = fig.add_subplot(gs[1, 1]) # Phase space
ax_energy = fig.add_subplot(gs[2, 1]) # Energy vs time

# Update supptitle
c_str = ", ".join([str(c) for c in c_values])
fig.suptitle(
    f"Mass-Spring-Damper System Analysis: Multiple Damping Cases\n"
    f"$m$={m} kg, $k$={k} N/m, $c_{crit}$={c_critical:.2f} N·s/m, "
    f"$\\omega_0$={omega_n:.2f} rad/s, Forcing: {forcing_name}",
    fontsize=14,
    fontweight="bold",
)

# Set up coordinate system
wall_x = -1.5 * x0
total_height = (num_cases - 1) * vertical_separation

ax.set_xlim(wall_x - 0.02, 1.6 * abs(x0))
ax.set_ylim(-total_height - 0.15, 0.15)
ax.set_xlabel("Position (m)", fontsize=12)

```

```

ax.set_ylabel("")
ax.set_yticks([])

# Draw wall
ax.axvline(wall_x, color="black", linewidth=15, zorder=4)

# Draw equilibrium lines and horizontal lines for each system
ax.axvline(0, color="gray", linestyle="--", alpha=0.3, linewidth=1)
ax.axvline(x0, color="gray", linestyle="--", alpha=0.3, linewidth=1)
ax.axvline(-x0, color="gray", linestyle="--", alpha=0.3, linewidth=1)

for i in range(num_cases):
    y_pos = -i * vertical_separation
    ax.axhline(y_pos, color="gray", alpha=0.3, linewidth=0.5)

# Initialize artists for each system
masses = []
springs = []
text_boxes = []

# Plot elements for right column
lines_pos = []
points_pos = []
lines_phase = []
points_phase = []
lines_energy = []
points_energy = []

for i, (c, label) in enumerate(zip(c_values, case_labels)):
    color = case_colors[i % len(case_colors)]
    edge_color = f"dark{color}"

    # Mass
    mass = ax.scatter(
        [],
        [],
        s=400,
        c=color,
        marker="s",
        edgecolor=edge_color,
        linewidth=2,
        label=label,
        zorder=5,
    )
    masses.append(mass)

    # Spring
    (spring,) = ax.plot([], [], color=color, lw=2, alpha=0.8, zorder=3)
    springs.append(spring)

    # Text box
    y_text_pos = 0.98 - i * 0.18
    text = ax.text(
        1.02,
        y_text_pos,
        "",
        transform=ax.transAxes,
        fontsize=8,
        va="top",
        ha="left",
        bbox=dict(
            boxstyle="round,pad=0.4",
            facecolor=color,

```

```

        alpha=0.25,
        edgecolor="black",
        linewidth=1.5,
    ),
    family="monospace",
)
text_boxes.append(text)

# Create elements for position plot
(line_pos,) = ax_position.plot(
    [], [], color=color, linewidth=2, alpha=0.7, label=label
)
point_pos = ax_position.scatter(
    [], [], s=50, c=color, zorder=5, edgecolor=edge_color
)
lines_pos.append(line_pos)
points_pos.append(point_pos)

# Create elements for phase space plot
(line_phase,) = ax_phase.plot(
    [], [], color=color, linewidth=1.5, alpha=0.6, label=label
)
point_phase = ax_phase.scatter(
    [], [], s=60, c=color, zorder=5, edgecolor=edge_color
)
lines_phase.append(line_phase)
points_phase.append(point_phase)

# Create elements for energy plot
(line_energy,) = ax_energy.plot(
    [], [], color=color, linewidth=2, alpha=0.7, label=label
)
point_energy = ax_energy.scatter(
    [], [], s=50, c=color, zorder=5, edgecolor=edge_color
)
lines_energy.append(line_energy)
points_energy.append(point_energy)

# Add legend
ax.legend(
    loc="upper right",
    fontsize=9,
    fancybox=True,
    shadow=True,
    facecolor="white",
    edgecolor="darkgray",
    markerscale=0.5,
    labelspacing=1.1,
)

# Configure position vs time plot
max_x = max(np.max(np.abs(x)) for x in x_solutions)
ax_position.set_xlim(0, 1.1 * duration)
ax_position.set_ylim(-max_x * 1.15, max_x * 1.2)
ax_position.set_xlabel("Time (s)", fontsize=11)
ax_position.set_ylabel("Position  $x$  (m)", fontsize=11)
ax_position.set_title("Position vs Time", fontsize=12, fontweight="bold")
ax_position.grid(True, alpha=0.3)

# Configure phase space plot
max_v = max(np.max(np.abs(v)) for v in v_solutions)
ax_phase.set_xlim(-max_x * 1.15, max_x * 1.15)
ax_phase.set_ylim(-max_v * 1.15, max_v * 1.2)

```

```

ax_phase.set_xlabel("Position  $x$  (m)", fontsize=11)
ax_phase.set_ylabel("Velocity  $v$  (m/s)", fontsize=11)
ax_phase.set_title("Phase Space ( $v$  vs  $x$ )", fontsize=12, fontweight="bold")
ax_phase.grid(True, alpha=0.3)

# Configure energy vs time plot
max_E = max(np.max(E) for E in energy_solutions)
ax_energy.set_xlim(0, 1.1 * duration)
ax_energy.set_ylim(-0.05 * max_E, max_E * 1.2)
ax_energy.set_xlabel("Time (s)", fontsize=11)
ax_energy.set_ylabel("Energy  $E$  (J)", fontsize=11)
ax_energy.set_title("Total Energy vs Time", fontsize=12, fontweight="bold")
ax_energy.grid(True, alpha=0.3)

def draw_spring_horizontal(start_x, end_x, num_coils, radius=0.03, y_center=0.0):
    """Draw a horizontal spring from start_x to end_x"""
    if abs(end_x - start_x) < 1e-9:
        return np.array([start_x]), np.array([y_center])

    t = np.linspace(0, num_coils * 2 * np.pi, 300)
    x_spring = np.linspace(start_x, end_x, 300)
    y_spring = y_center + radius * np.sin(t)

    return x_spring, y_spring

def init():
    """Initialize animation"""
    for i, (mass, spring) in enumerate(zip(masses, springs)):
        y_pos = -i * vertical_separation
        mass.set_offsets(np.array([[np.nan, y_pos]]))
        spring.set_data([], [])
    for line_pos, line_phase, line_energy in zip(
        lines_pos, lines_phase, lines_energy
    ):
        line_pos.set_data([], [])
        line_phase.set_data([], [])
        line_energy.set_data([], [])
    return tuple(
        masses
        + springs
        + text_boxes
        + lines_pos
        + points_pos
        + lines_phase
        + points_phase
        + lines_energy
        + points_energy
    )

def update(frame):
    """Update animation for each frame"""
    # Current time
    t = t_array[frame]
    num_osc = min(t / T, oscillations)

    # Update title
    ax.set_title(
        f"Time: {t:.2f} s | Oscillation: {num_osc:.2f}/{oscillations}",
        fontsize=12,
        fontweight="bold",
        pad=10,
    )

```

```

artists = []

# Update each system
for i, (c, x_vals, v_vals, E_vals) in enumerate(
    zip(c_values, x_solutions, v_solutions, energy_solutions)
):
    y_pos = -i * vertical_separation
    x_current = x_vals[frame]
    v_current = v_vals[frame]
    E_current = E_vals[frame]

    # Update spring and mass animation
    x_spring, y_spring = draw_spring_horizontal(
        wall_x, x_current, num_coils, radius=0.03, y_center=y_pos
    )
    springs[i].set_data(x_spring, y_spring)
    masses[i].set_offsets([[x_current, y_pos]])

    # Calculate spring deformation
    spring_deform = abs(x_current)

    # Calculate damping ratio
    zeta = c / c_critical if c > 0 else 0

    # Update text box
    info = (
        f"{case_labels[i]}\n"
        f"{'-' * 22}\n"
        f"Position:   {x_current:+.4f} m\n"
        f"Velocity:    {v_current:+.4f} m/s\n"
        f"Energy:      {E_current:.4f} J\n"
        f"Spring def:  {spring_deform:.4f} m\n"
        f"Damping:     {c:.3f} N·s/m\n"
        f"Damp ratio:  {zeta:.3f}"
    )
    text_boxes[i].set_text(info)

    # Update position vs time plot
    t_data = t_array[: frame + 1]
    x_data = x_vals[: frame + 1]
    lines_pos[i].set_data(t_data, x_data)
    points_pos[i].set_offsets([[t, x_current]])

    # Update phase space plot
    v_data = v_vals[: frame + 1]
    lines_phase[i].set_data(x_data, v_data)
    points_phase[i].set_offsets([[x_current, v_current]])

    # Update energy vs time plot
    E_data = E_vals[: frame + 1]
    lines_energy[i].set_data(t_data, E_data)
    points_energy[i].set_offsets([[t, E_current]])

    artists.extend(
        [
            masses[i],
            springs[i],
            text_boxes[i],
            lines_pos[i],
            points_pos[i],
            lines_phase[i],
            points_phase[i],
            lines_energy[i],

```

```

        points_energy[i],
    ]
)

return tuple(artists)

# Create animation
anim = FuncAnimation(
    fig,
    update,
    frames=num_frames,
    init_func=init,
    blit=False,
    interval=int(1000.0 / fps),
    repeat=False,
)

# Save or show
if save_animation:
    if filename is None:
        c_str = "_".join([f"c{c}" for c in c_values])
        filename = f"multi_damping_analysis_m{m}_k{k}_{c_str}.gif"

    save_dir = "OUTPUTS/ANIMATIONS"
    os.makedirs(save_dir, exist_ok=True)
    filepath = os.path.join(save_dir, filename)

    try:
        print(f"Saving animation to {os.path.abspath(filepath)}...")
        anim.save(filepath, writer="ffmpeg", fps=fps, codec="gif", dpi=120)
        print(f"Animation saved as: {filepath}")
    except Exception as e:
        print(f"Error saving animation: {e}")
    plt.close(fig)
else:
    plt.show()

return anim

def main():
    """Main function to run the multi-damping animation with user inputs."""

    print("\n" + "=" * 70)
    print("  Mass-Spring-Damper Animation: Multiple Damping Cases")
    print("=" * 70)
    print("\nThis animation compares multiple damping cases:")
    print("  - Undamped ( $c=0$ )")
    print("  - Underdamped ( $0 < c < c_{critical}$ )")
    print("  - Critically damped ( $c = c_{critical}$ )")
    print("  - Overdamped ( $c > c_{critical}$ )\n")

    # Default parameters
    defaults = {
        "m": 1.0,
        "k": 10.0,
        "c_values": [0, 0.5, 6.32, 7],
        "x0": 0.1,
        "v0": 0.0,
        "oscillations": 5.0,
        "fps": 30,
    }

```

```

use_defaults = input("Use default parameters? (y/n) [y]: ").strip().lower()

if use_defaults == "n":
    try:
        print("\n--- System Parameters ---")
        m = float(input("Mass (m) [1.0 kg]: ") or 1.0)
        k = float(input("Stiffness (k) [10.0 N/m]: ") or 10.0)

        # Calculate critical damping
        c_critical = 2 * np.sqrt(k * m)
        print(f"\nCritical damping coefficient: c_crit = {c_critical:.3f} N·s/m")

        # Input damping coefficients
        print("\n--- Damping Coefficients ---")
        print("Enter damping coefficients (comma-separated).")
        print("Include 0 for undamped case.")
        print(
            f"Suggestions: 0 (undamped), {c_critical * 0.2:.2f} (underdamped), ↵
↵{c_critical:.2f} (critical), {c_critical * 1.5:.2f} (overdamped)"
        )
        c_input = input(
            f"c values [0, {c_critical * 0.2:.2f}, {c_critical:.2f}, {c_critical * 1.5:
↵.2f}]: "
        ).strip()

        if c_input:
            c_values = [float(c.strip()) for c in c_input.split(",")]
        else:
            c_values = [0, c_critical * 0.2, c_critical, c_critical * 1.5]

        # Sort c_values to ensure proper ordering
        c_values = sorted(c_values)

        # Initial conditions
        x0 = float(input("Initial displacement/amplitude (x0) [0.1 m]: ") or 0.1)
        v0 = float(input("Initial velocity v0 [0.0 m/s]: ") or 0.0)

        print("\n--- Animation Parameters ---")
        oscillations = float(input("Number of periods to display [5.0]: ") or 5.0)
        fps = int(input("Frames per second [30]: ") or 30)

    except ValueError as e:
        print(f"\nInvalid input: {e}")
        print("Using default parameters.")
        m = defaults["m"]
        k = defaults["k"]
        c_values = defaults["c_values"]
        x0 = defaults["x0"]
        v0 = defaults["v0"]
        oscillations = defaults["oscillations"]
        fps = defaults["fps"]
    else:
        m = defaults["m"]
        k = defaults["k"]
        c_values = defaults["c_values"]
        x0 = defaults["x0"]
        v0 = defaults["v0"]
        oscillations = defaults["oscillations"]
        fps = defaults["fps"]

    # Validate parameters
    if m <= 0:
        print("Warning: Mass must be positive, setting to 1.0")

```

```

    m = 1.0
if k <= 0:
    print("Warning: Stiffness must be positive, setting to 10.0")
    k = 10.0
if any(c < 0 for c in c_values):
    print("Warning: Damping coefficients must be non-negative")
    c_values = [max(0, c) for c in c_values]

# Setting the forcing function to none for non-driven case
forcing_func = no_forcing
forcing_name = "None"

save_anim = input("\nSave animation as GIF? (y/n) [n]: ").strip().lower() == "y"
filename = None
if save_anim:
    filename_input = input("Enter filename (or press Enter for default): ").strip()
    if filename_input:
        if not filename_input.endswith(".gif"):
            filename_input += ".gif"
        filename = filename_input

print("\n" + "-" * 70)
print("Animating with:")
print(f"  Mass (m):           {m} kg")
print(f"  Stiffness (k):       {k} N/m")
print(f"  Damping values:      {[float(np.round(c, 2)) for c in c_values if c > 0]}")
print(f"  Initial disp (x0):   {x0} m")
print(f"  Initial vel (v0):    {v0} m/s")
print(f"  Periods:             {oscillations}")
print(f"  FPS:                 {fps}")
print("-" * 70 + "\n")

anim = animate_multiple_damping_cases_with_plots(
    m=m,
    c_values=c_values,
    k=k,
    x0=x0,
    v0=v0,
    oscillations=oscillations,
    fps=fps,
    save_animation=save_anim,
    filename=filename,
    forcing_func=forcing_func,
    forcing_name=forcing_name,
)

print("\nAnimation completed!")
return anim

if __name__ == "__main__":
    animation = main()

```

See the animation in the GitHub repository: [Animation 6](#)

5.5 Phase Space Evolution of the Mass-Spring-Damper System

Phase Space (Mass-Spring) - Phase space is the 2D state plane (x, v) where x is displacement and $v = \dot{x}$ is velocity. The dynamics become a first-order system:

$$\dot{x} = v, \quad \dot{v} = \ddot{x}$$

Undamped SHM ($c = 0$)

- Equation of motion:

$$m\ddot{x} + kx = 0, \quad \omega_n = \sqrt{k/m}.$$

- State-space form:

$$\dot{x} = v, \quad \dot{v} = -\omega_n^2 x$$

- Solve (one convenient form):

$$x(t) = A \cos(\omega_n t) + B \sin(\omega_n t).$$

Then

$$v(t) = \dot{x} = -A\omega_n \sin(\omega_n t) + B\omega_n \cos(\omega_n t)$$

- Eliminate t to get the phase-curve (an ellipse):

$$x^2 + \left(\frac{v}{\omega_n}\right)^2 = A^2 + B^2 = X_0^2$$

So trajectories are closed loops (a “center”) around the origin; energy is constant:

$$E = \frac{1}{2}mv^2 + \frac{1}{2}kx^2 = \text{const.}$$

Damped SHM ($c > 0$)

- Equation of motion:

$$m\ddot{x} + c\dot{x} + kx = 0.$$

- State-space form (linear system):

$$\begin{bmatrix} \dot{x} \\ \dot{v} \end{bmatrix} = \underbrace{\begin{bmatrix} 0 & 1 \\ -k/m & -c/m \end{bmatrix}}_A \begin{bmatrix} x \\ v \end{bmatrix}.$$

Eigenvalues (characteristic equation $mr^2 + cr + k = 0$):

$$r_{1,2} = \frac{-c \pm \sqrt{c^2 - 4mk}}{2m}.$$

Using $\omega_n = \sqrt{k/m}$ and $\zeta = \frac{c}{2\sqrt{km}}$

- **Underdamped** ($0 < \zeta < 1$):

$$r = -\zeta\omega_n \pm i\omega_d, \quad \omega_d = \omega_n\sqrt{1 - \zeta^2}.$$

$$x(t) = e^{-\zeta\omega_n t} (C_1 \cos(\omega_d t) + C_2 \sin(\omega_d t)).$$

This exponential envelope makes the phase portrait an inward spiral toward $(0, 0)$ (a stable spiral/focus).

- **Critical** ($\zeta = 1$):

$$x(t) = (C_1 + C_2 t)e^{-\omega_n t}$$

(no oscillation; fastest return). Phase portrait is a stable node with repeated eigenvalue.

- **Overdamped** ($\zeta > 1$):

$$x(t) = C_1 e^{r_1 t} + C_2 e^{r_2 t}, \quad r_{1,2} < 0$$

(no oscillation). Phase portrait is a stable node (approach without spiraling).

- Energy decay (derivation): with $E = \frac{1}{2}mv^2 + \frac{1}{2}kx^2$,

$$\frac{dE}{dt} = mv\dot{v} + kx\dot{x} = v(m\dot{v} + kx) = v(-cv) = -cv^2 \leq 0,$$

explaining why damped trajectories collapse into the origin.

- In the code cell below we first integrate over the first-order system $\dot{x} = v$, $\dot{v} = (F(t) - cv - kx)/m$ in `mass_spring_damper_system`, with `no_forcing(t)=0` for free oscillation (so $F(t) = 0$), using an RK4 time-stepper `rk4_mass_spring_damper`.
- For each damping value `c` in `c_values`, it computes arrays `(x(t), v(t))` via RK4, then plots **velocity vs position** by calling `ax.plot(x, v, ...)`, which directly draws the phase trajectory in the (x, v) plane.
- Labels are generated from the damping ratio $\zeta = c/c_{\text{crit}}$ with $c_{\text{crit}} = 2\sqrt{km}$, and the damping regime is classified by `analyze_system_properties`.

```
[14]: def _case_label(m, c, k, c_critical):
    if abs(c) < 1e-15:
        return "Undamped\n(c=0,  $\zeta$ =0.00)"

    props = analyze_system_properties(m, c, k)
    damping_type = props.get("damping_type", "damped").title()
    zeta = c / c_critical
    return f"{damping_type}\n(c={c:.3f},  $\zeta$ ={zeta:.2f})"

def plot_phase_space_free_oscillation(
    m=1.0,
    k=10.0,
    c_values=None,
    x0=0.1,
    v0=0.0,
    duration_periods=5.0,
    points_per_period=250,
    save_figure=False,
    filename=None,
):
    """Plot phase space (v vs x) for free oscillation (no forcing) across multiple damping_
    ↪ values."""

    omega_n = np.sqrt(k / m)
    T_n = 2 * np.pi / omega_n
    c_critical = 2 * np.sqrt(k * m)

    if c_values is None:
        c_values = [0.0, 0.2 * c_critical, c_critical, 2.0 * c_critical]

    # Ensure we always compare against the undamped oscillator
    if all(abs(c) > 1e-15 for c in c_values):
        c_values = [0.0] + list(c_values)

    duration = duration_periods * T_n
    h = T_n / float(points_per_period)

    fig, ax = plt.subplots(figsize=(12, 6))
    fig.subplots_adjust(top=0.9, bottom=0.1, left=0.08, right=0.83)
    colors = ["black", "tab:blue", "tab:orange", "tab:green", "tab:red", "tab:purple"]

    for idx, c in enumerate(c_values):
        t, x, v = rk4_mass_spring_damper(m, c, k, x0, v0, 0.0, duration, h, no_forcing)

        label = _case_label(m, c, k, c_critical)
        if abs(c) < 1e-15:
            ax.plot(
                x,
                v,
                linewidth=2.5,
                alpha=0.85,
                color=colors[0],
                label=label,
            )
        else:
            ax.plot(
                x,
                v,
                linewidth=2.0,
                alpha=0.85,
                color=colors[(idx % (len(colors) - 1)) + 1],
                label=label,
            )
```

```

    )

    ax.set_xlabel("Position  $x$  (m)")
    ax.set_ylabel("Velocity  $v$  (m/s)")
    ax.set_title(
        "Phase Space: Free Mass-Spring-Damper", fontsize=14, fontweight="bold", pad=10
    )
    ax.grid(True, alpha=0.3)
    ax.axhline(0, color="black", linewidth=0.8, alpha=0.25)
    ax.axvline(0, color="black", linewidth=0.8, alpha=0.25)
    ax.legend(loc="upper right", bbox_to_anchor=(1.2, 1), fontsize=9)

    if save_figure:
        if filename is None:
            c_str = "_".join([f"c{float(np.round(c, 3))}" for c in c_values])
            filename = f"phase_space_free_oscillation_m{m}_k{k}_{c_str}.png"

            save_dir = "OUTPUTS/PLOTS"
            os.makedirs(save_dir, exist_ok=True)
            filepath = os.path.join(save_dir, filename)

            try:
                print(f"Saving figure to {os.path.abspath(filepath)}...")
                fig.savefig(filepath, dpi=300, bbox_inches="tight")
                print(f"Figure saved as: {filepath}")
            except Exception as e:
                print(f"Error saving figure: {e}")

    plt.show()

def main():
    defaults = {
        "m": 1.0,
        "k": 10.0,
        "x0": 0.1,
        "v0": 0.0,
        "duration_periods": 5.0,
        "points_per_period": 250,
    }

    use_defaults = input("Use default parameters? (y/n) [y]: ").strip().lower() or "y"

    if use_defaults == "n":
        try:
            m = float(input(f"Mass m [{defaults['m']} kg]: ") or defaults["m"])
            k = float(input(f"Stiffness k [{defaults['k']} N/m]: ") or defaults["k"])
            x0 = float(
                input(f"Initial displacement x0 [{defaults['x0']} m]: ")
                or defaults["x0"]
            )
            v0 = float(
                input(f"Initial velocity v0 [{defaults['v0']} m/s]: ")
                or defaults["v0"]
            )
            duration_periods = float(
                input(
                    f"Duration (in natural periods) [{defaults['duration_periods']}]: "
                )
                or defaults["duration_periods"]
            )
            points_per_period = int(
                input(f"Points per period [{defaults['points_per_period']}]: ")
                or defaults["points_per_period"]
            )

```

```

except ValueError:
    print("Invalid input. Using defaults.")
    m = defaults["m"]
    k = defaults["k"]
    x0 = defaults["x0"]
    v0 = defaults["v0"]
    duration_periods = defaults["duration_periods"]
    points_per_period = defaults["points_per_period"]
else:
    m = defaults["m"]
    k = defaults["k"]
    x0 = defaults["x0"]
    v0 = defaults["v0"]
    duration_periods = defaults["duration_periods"]
    points_per_period = defaults["points_per_period"]

if m <= 0:
    print("Warning: m must be positive; using m=1.0")
    m = 1.0
if k <= 0:
    print("Warning: k must be positive; using k=10.0")
    k = 10.0
if duration_periods <= 0:
    print("Warning: duration must be positive; using 5 periods")
    duration_periods = 5.0
if points_per_period < 50:
    print("Warning: points_per_period too low; using 250")
    points_per_period = 250

c_critical = 2 * np.sqrt(k * m)
default_c_values = [0.0, 0.2 * c_critical, c_critical, 2.0 * c_critical]
default_str = ", ".join(f"{c:.3f}" for c in default_c_values)

print("\n--- Damping suggestions ---")
print(f"Critical damping: c_crit = {c_critical:.3f} N.s/m")
print("Suggested c_values (comma-separated):")
print(f" - Default set: {default_str}")
print(
    " - Interpretation: c=0 (undamped), c<c_crit (underdamped), c=c_crit (critical),
    ↪c>c_crit (overdamped)"
)

c_input = input(f"Enter c_values [{default_str}]: ").strip()
if c_input:
    try:
        c_values = [float(s.strip()) for s in c_input.split(",") if s.strip()]
    except ValueError:
        print("Invalid c_values. Using defaults.")
        c_values = default_c_values
else:
    c_values = default_c_values

# Force non-negative damping values
c_values = [max(0.0, c) for c in c_values]

save_fig = input("\nSave figures? (y/n) [n]: ").strip().lower() == "y"
filename = None
if save_fig:
    filename_input = input(
        "Enter base filename (or press Enter for default): "
    ).strip()
    if filename_input:

```

```

    if not filename_input.endswith(".png"):
        filename_input += ".png"
    filename = filename_input

    plot_phase_space_free_oscillation(
        m=m,
        k=k,
        c_values=c_values,
        x0=x0,
        v0=v0,
        duration_periods=duration_periods,
        points_per_period=points_per_period,
        save_figure=save_fig,
        filename=filename,
    )

if __name__ == "__main__":
    main()

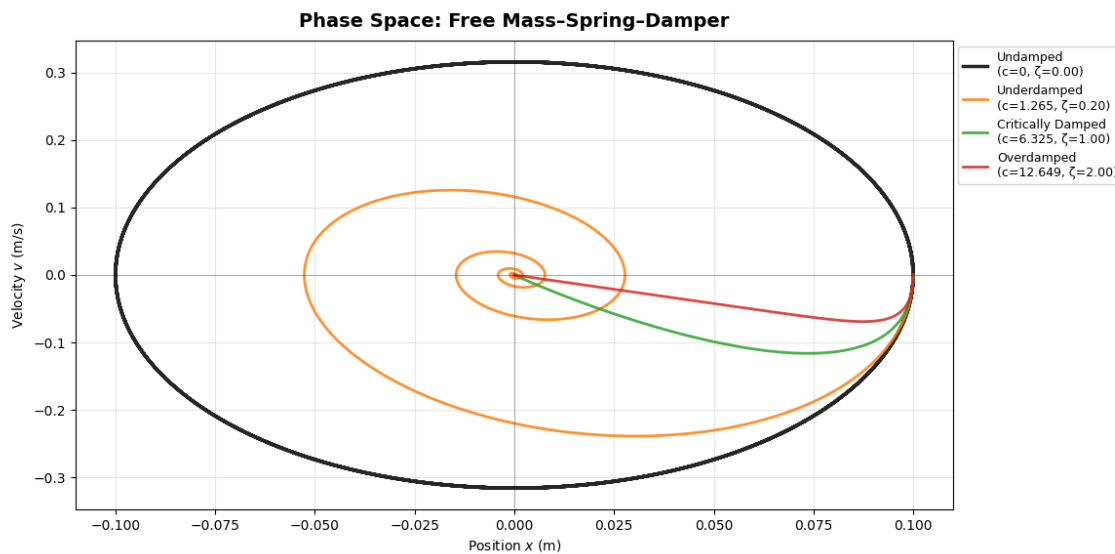
```

--- Damping suggestions ---

Critical damping: $c_{\text{crit}} = 6.325 \text{ N}\cdot\text{s/m}$

Suggested c_{values} (comma-separated):

- Default set: 0.000, 1.265, 6.325, 12.649
- Interpretation: $c=0$ (undamped), $c < c_{\text{crit}}$ (underdamped), $c=c_{\text{crit}}$ (critical), $c > c_{\text{crit}}$ (overdamped)



5.6 Energy Evolution of the Mass-Spring-Damper System

Energy in Undamped SHM (Constant)

- For a mass-spring system (no damping, no forcing),

$$m\ddot{x} + kx = 0, \quad v = \dot{x}.$$

- Total mechanical energy:

$$E(t) = T + U = \frac{1}{2}mv^2 + \frac{1}{2}kx^2.$$

- Differentiate and use the equation of motion:

$$\frac{dE}{dt} = mv\dot{v} + kx\dot{x} = v(m\ddot{x} + kx) = v(0) = 0.$$

Hence

$$E(t) = \text{constant}.$$

Physically: no dissipation, so the phase-space curve stays a closed ellipse.

Energy in Damped SHM (Decreases Monotonically)

- For viscous damping (still no forcing),

$$m\ddot{x} + c\dot{x} + kx = 0.$$

- Using the same energy definition:

$$\frac{dE}{dt} = m\dot{v}\dot{v} + kx\dot{x} = v(m\ddot{x} + kx) = v(-c\dot{x}) = -cv^2 \leq 0.$$

So energy is **not conserved**; it continuously decreases and is dissipated as heat. Equality holds only when $v = 0$ instantaneously.

- In the **underdamped** case ($0 < \zeta < 1$), the displacement amplitude decays as $e^{-\zeta\omega_n t}$, so the energy envelope decays approximately as

$$E(t) \propto e^{-2\zeta\omega_n t}.$$

This matches the phase portrait: trajectories spiral inward as energy drains.

```
[15]: def plot_energy_evolution_free_oscillation(
    m=1.0,
    k=10.0,
    c_values=None,
    x0=0.1,
    v0=0.0,
    duration_periods=5.0,
    points_per_period=250,
    save_figure=False,
    filename=None,
):
    """Plot energy vs time for free oscillation (no forcing) across multiple damping
    values."""

    omega_n = np.sqrt(k / m)
    T_n = 2 * np.pi / omega_n
    c_critical = 2 * np.sqrt(k * m)

    if c_values is None:
        c_values = [0.0, 0.2 * c_critical, c_critical, 2.0 * c_critical]

    if all(abs(c) > 1e-15 for c in c_values):
        c_values = [0.0] + list(c_values)

    duration = duration_periods * T_n
    h = T_n / float(points_per_period)

    fig, ax = plt.subplots(figsize=(10, 5))
    colors = ["black", "tab:blue", "tab:orange", "tab:green", "tab:red", "tab:purple"]

    for idx, c in enumerate(c_values):
        t, x, v = rk4_mass_spring_damper(m, c, k, x0, v0, 0.0, duration, h, no_forcing)
        E = 0.5 * m * (v**2) + 0.5 * k * (x**2)
        label = _case_label(m, c, k, c_critical)

        if abs(c) < 1e-15:
            ax.plot(
                t,
                E,
                linestyle="--",
                linewidth=2.5,
```

```

        alpha=0.85,
        color=colors[0],
        label=label,
    )
else:
    ax.plot(
        t,
        E,
        linewidth=2.0,
        alpha=0.85,
        color=colors[(idx % (len(colors) - 1)) + 1],
        label=label,
    )

ax.set_xlabel("Time $t$ (s)")
ax.set_ylabel("Energy $E$ (J)")
ax.set_title(
    "Energy Evolution: Free Mass-Spring-Damper ",
    fontsize=14,
    fontweight="bold",
    pad=10,
)
ax.grid(True, alpha=0.3)
ax.legend(fontsize=9)
ax.set_xlim(0, duration)

if save_figure:
    if filename is None:
        c_str = "_".join([f"c{float(np.round(c, 3))}" for c in c_values])
        filename = f"energy_evolution_free_oscillation_m{m}_k{k}_{c_str}.png"

        save_dir = "OUTPUTS/PLOTS"
        os.makedirs(save_dir, exist_ok=True)
        filepath = os.path.join(save_dir, filename)

        try:
            print(f"Saving figure to {os.path.abspath(filepath)}...")
            fig.savefig(filepath, dpi=300, bbox_inches="tight")
            print(f"Figure saved as: {filepath}")
        except Exception as e:
            print(f"Error saving figure: {e}")

plt.tight_layout()
plt.show()

def main():
    defaults = {
        "m": 1.0,
        "k": 10.0,
        "x0": 0.1,
        "v0": 0.0,
        "duration_periods": 5.0,
        "points_per_period": 250,
    }

    use_defaults = input("Use default parameters? (y/n) [y]: ").strip().lower() or "y"

    if use_defaults == "n":
        try:
            m = float(input(f"Mass m [{defaults['m']} kg]: ") or defaults["m"])
            k = float(input(f"Stiffness k [{defaults['k']} N/m]: ") or defaults["k"])
            x0 = float(

```

```

        input(f"Initial displacement x0 [{defaults['x0']} m]: ")
        or defaults["x0"]
    )
    v0 = float(
        input(f"Initial velocity v0 [{defaults['v0']} m/s]: ") or defaults["v0"]
    )
    duration_periods = float(
        input(
            f"Duration (in natural periods) [{defaults['duration_periods']}]: "
        )
        or defaults["duration_periods"]
    )
    points_per_period = int(
        input(f"Points per period [{defaults['points_per_period']}]: ")
        or defaults["points_per_period"]
    )

except ValueError:
    print("Invalid input. Using defaults.")
    m = defaults["m"]
    k = defaults["k"]
    x0 = defaults["x0"]
    v0 = defaults["v0"]
    duration_periods = defaults["duration_periods"]
    points_per_period = defaults["points_per_period"]
else:
    m = defaults["m"]
    k = defaults["k"]
    x0 = defaults["x0"]
    v0 = defaults["v0"]
    duration_periods = defaults["duration_periods"]
    points_per_period = defaults["points_per_period"]

if m <= 0:
    print("Warning: m must be positive; using m=1.0")
    m = 1.0
if k <= 0:
    print("Warning: k must be positive; using k=10.0")
    k = 10.0
if duration_periods <= 0:
    print("Warning: duration must be positive; using 5 periods")
    duration_periods = 5.0
if points_per_period < 50:
    print("Warning: points_per_period too low; using 250")
    points_per_period = 250

c_critical = 2 * np.sqrt(k * m)
default_c_values = [0.0, 0.2 * c_critical, c_critical, 2.0 * c_critical]
default_str = ", ".join(f"{c:.3f}" for c in default_c_values)

print("\n--- Damping suggestions ---")
print(f"Critical damping: c_crit = {c_critical:.3f} N.s/m")
print("Suggested c_values (comma-separated):")
print(f" - Default set: {default_str}")
print(
    " - Interpretation: c=0 (undamped), c<c_crit (underdamped), c=c_crit (critical), c>c_crit (overdamped)"
)

c_input = input(f"Enter c_values [{default_str}]: ").strip()
if c_input:
    try:
        c_values = [float(s.strip()) for s in c_input.split(",") if s.strip()]

```

```

except ValueError:
    print("Invalid c_values. Using defaults.")
    c_values = default_c_values
else:
    c_values = default_c_values

# Force non-negative damping values
c_values = [max(0.0, c) for c in c_values]

save_fig = input("\nSave figures? (y/n) [n]: ").strip().lower() == "y"
filename = None
if save_fig:
    filename_input = input(
        "Enter base filename (or press Enter for default): "
    ).strip()
    if filename_input:
        if not filename_input.endswith(".png"):
            filename_input += ".png"
        filename = filename_input

plot_energy_evolution_free_oscillation(
    m=m,
    k=k,
    c_values=c_values,
    x0=x0,
    v0=v0,
    duration_periods=duration_periods,
    points_per_period=points_per_period,
    save_figure=save_fig,
    filename=filename,
)

if __name__ == "__main__":
    main()

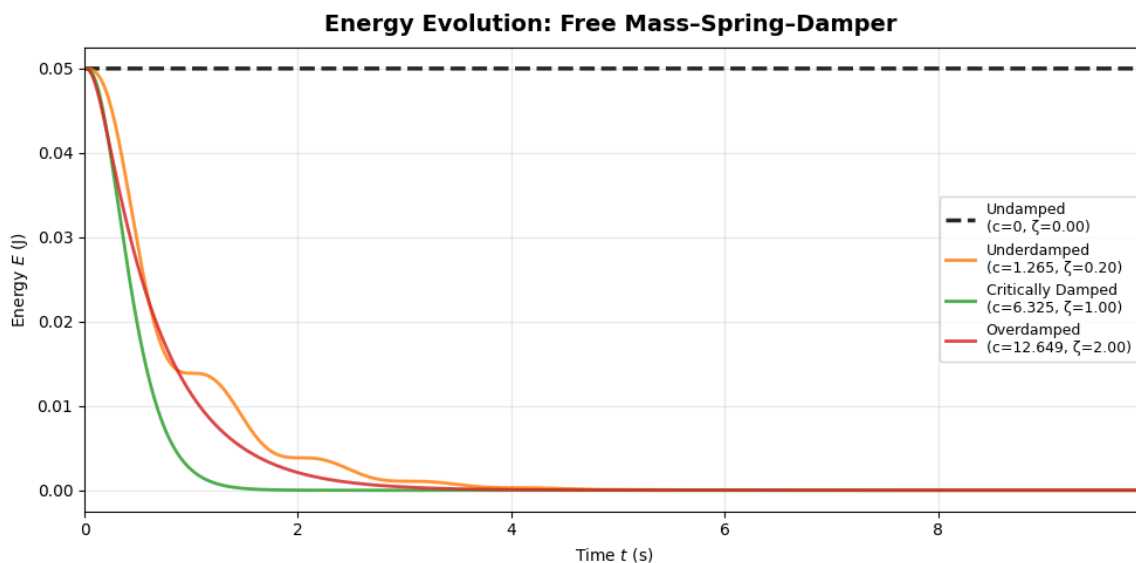
```

--- Damping suggestions ---

Critical damping: $c_{\text{crit}} = 6.325 \text{ N}\cdot\text{s/m}$

Suggested c_{values} (comma-separated):

- Default set: 0.000, 1.265, 6.325, 12.649
- Interpretation: $c=0$ (undamped), $c < c_{\text{crit}}$ (underdamped), $c = c_{\text{crit}}$ (critical), $c > c_{\text{crit}}$ (overdamped)



5.7 Damped Driven Mass-Spring Systems

For a mass-spring-damper driven by a sinusoidal force,

$$m\ddot{x} + c\dot{x} + kx = F_0 \cos(\omega t).$$

Define the standard parameters

$$\omega_0 = \sqrt{\frac{k}{m}}, \quad \beta = \frac{c}{2m}, \quad \zeta = \frac{c}{2\sqrt{km}} = \frac{\beta}{\omega_0}.$$

Write it in normalized form:

$$\ddot{x} + 2\beta\dot{x} + \omega_0^2 x = \frac{F_0}{m} \cos(\omega t).$$

Steady-State Solution (Explicit Derivation) The total response is

$$x(t) = x_h(t) + x_p(t),$$

where x_h solves the homogeneous equation (transients) and x_p is the forced steady-state.

Use the complex method for the particular solution. Assume

$$x_p(t) = \Re\{Xe^{i\omega t}\}, \quad F(t) = \Re\{F_0 e^{i\omega t}\}.$$

Then $\dot{x}_p = i\omega X e^{i\omega t}$ and $\ddot{x}_p = -\omega^2 X e^{i\omega t}$. Substitute:

$$m(-\omega^2 X) + c(i\omega X) + kX = F_0.$$

So the complex amplitude is

$$X = \frac{F_0}{k - m\omega^2 + i c \omega}.$$

Therefore the steady-state displacement can be written as

$$x_p(t) = A(\omega) \cos(\omega t - \phi),$$

with amplitude and phase lag

$$A(\omega) = |X| = \frac{F_0}{\sqrt{(k - m\omega^2)^2 + (c\omega)^2}} = \frac{F_0/m}{\sqrt{(\omega_0^2 - \omega^2)^2 + (2\beta\omega)^2}},$$

$$\phi(\omega) = \tan^{-1}\left(\frac{c\omega}{k - m\omega^2}\right) = \tan^{-1}\left(\frac{2\beta\omega}{\omega_0^2 - \omega^2}\right).$$

What Resonance Means (and When It Happens) In driven oscillations, “resonance” typically means a **peak** in steady-state response as you vary the driving frequency (large response because the drive is most effective near the system’s natural timescale).

1) Undamped case ($c = 0$): true (unbounded) amplitude resonance

- If $c = 0$,

$$A(\omega) = \frac{F_0/m}{|\omega_0^2 - \omega^2|}.$$

- As $\omega \rightarrow \omega_0$, the denominator $\rightarrow 0$ and $A(\omega) \rightarrow \infty$ (ideal mathematical resonance). Physically, with no dissipation, the driver can add energy coherently each cycle without limit.

2) Damped case ($c > 0$): finite amplitude peak

- Damping keeps the amplitude finite because energy input is balanced by dissipation.

Condition for displacement (amplitude) resonance To find the peak of $A(\omega)$, maximize $A(\omega)$ or equivalently minimize

$$D(\omega) = (\omega_0^2 - \omega^2)^2 + (2\beta\omega)^2.$$

Differentiate and set to zero:

$$\frac{dD}{d\omega} = -4\omega(\omega_0^2 - \omega^2) + 8\beta^2\omega = 4\omega(\omega^2 - \omega_0^2 + 2\beta^2) = 0.$$

Ignoring the trivial $\omega = 0$, the resonance (peak-amplitude) frequency is

$$\omega_r = \sqrt{\omega_0^2 - 2\beta^2} = \omega_0 \sqrt{1 - 2\zeta^2}.$$

This is real only if

$$\omega_0^2 - 2\beta^2 > 0 \iff \zeta < \frac{1}{\sqrt{2}}.$$

Why resonance is not possible for critically damped/overdamped cases

- Critically damped: $\zeta = 1$
- Overdamped: $\zeta > 1$

Both satisfy $\zeta \geq 1/\sqrt{2}$, so ω_r becomes imaginary: there is **no nonzero frequency where $A(\omega)$ has a true peak**. Practically:

- The system's free motion has no sustained oscillatory character (no ringing frequency).
- The frequency response $A(\omega)$ becomes a smooth, monotonic “low-pass” shape (largest at very low frequency, approaching the static deflection $A(0) = F_0/k$), with **no resonant bump**.

Important nuance: even when **displacement resonance** disappears, **velocity** and **power** peaks still occur at the natural frequency: Velocity amplitude $A_v(\omega) = \omega A(\omega)$ peaks at $\omega = \omega_0$. Average absorbed power peaks at $\omega = \omega_0$.

The theory is implemented in the notebook as follows: Sinusoidal forcing is defined as $F(t) = F_0 \sin(2\pi ft)$ in `sinusoidal_forcing`. The ODE is integrated in first-order form:

$$\dot{x} = v, \quad \dot{v} = \frac{F(t) - cv - kx}{m}$$

using RK4 (`rk4_mass_spring_damper`). The “displacement resonance exists only if $\zeta < 1/\sqrt{2}$ ” logic is implemented explicitly. It computes $c_{\text{crit}} = 2\sqrt{km}$, $\zeta = c/c_{\text{crit}}$. If $\zeta < 1/\sqrt{2}$, it uses $\omega_r = \omega_0 \sqrt{1 - 2\zeta^2}$, otherwise it falls back to ω_0 as a reference. For the frequency-response curves, we estimate the **steady-state** numerically by simulating many cycles, discarding the transient part, then measuring amplitude as $(\max - \min)/2$ and computing average power $\langle P \rangle = \langle Fv \rangle$. The plotting code marks the displacement-resonance peak only when $\zeta < 1/\sqrt{2}$, and marks velocity/power peaks at f_0 (natural frequency) for all damped cases.

```
[ ]: def animate_damped_driven_oscillation_with_plots(
    m=1.0,
    c_values=[0, 0.5],
    k=10.0,
    x0=0.1,
    v0=0.0,
    oscillations=5.0,
    fps=30,
    save_animation=False,
    filename=None,
    file_format="gif",
    num_coils=8,
    vertical_separation=0.15,
    forcing_func=None,
    forcing_name=None,
    resonant_freq=None,
):
    """
    Animate multiple damping cases (undamped, underdamped, critically damped, overdamped)
    ↪ simultaneously.

    Parameters:
        m: Mass
        c_values: List of damping coefficients (0 for undamped, then underdamped,
    ↪ critical, overdamped)
        k: Stiffness
        x0: Initial displacement/amplitude
        v0: Initial velocity
        oscillations: Number of oscillations to show
        fps: Frames per second
```

```

    save_animation: Whether to save as GIF
    filename: Output filename
    file_format: Output file format ("gif" or "mp4")
    num_coils: Number of coils in spring visualization
    vertical_separation: Vertical distance between systems
    forcing_func: External forcing function (single function or list of functions for_
→ each case)
    forcing_name: Name of forcing function for display (single string, list, or None)
    resonant_freq: Resonant frequency to display in title (optional)
"""
# Handle forcing_func as single function or list
if forcing_func is None:
    forcing_func = [no_forcing] * len(c_values)
elif not isinstance(forcing_func, list):
    forcing_func = [forcing_func] * len(c_values)

# System properties
omega_n = np.sqrt(k / m)
T = 2 * np.pi / omega_n
c_critical = 2 * np.sqrt(k * m)

# Time parameters
duration = oscillations * T
num_frames = int(round(duration * fps)) + 1
t_array = np.linspace(0.0, duration, num_frames, endpoint=True)
h = duration / (num_frames - 1)

# Solve for each damping case
num_cases = len(c_values)
x_solutions = []
v_solutions = []
case_labels = []
box_labels = []
case_colors = ["blue", "green", "orange", "red", "magenta", "violet"]

# Handle forcing_name as single string, list, or None
if forcing_name is None:
    forcing_name_list = ["none"] * len(c_values)
elif not isinstance(forcing_name, list):
    forcing_name_list = [forcing_name] * len(c_values)
else:
    forcing_name_list = forcing_name

for i, c in enumerate(c_values):
    if c == 0:
        # Undamped case: analytical solution
        x_vals = x0 * np.cos(omega_n * t_array)
        v_vals = -x0 * omega_n * np.sin(omega_n * t_array)
        case_labels.append(f"Undamped ({forcing_name_list[i]})")
        box_labels.append("Undamped, c = 0")
    else:
        # Damped cases: numerical solution
        _, x_vals, v_vals = rk4_mass_spring_damper(
            m, c, k, x0, v0, 0.0, duration, h, forcing_func[i]
        )
        props = analyze_system_properties(m, c, k)
        case_labels.append(
            f"{props['damping_type'].title()} - {forcing_name_list[i]}"
        )
        box_labels.append(
            f"{props['damping_type'].title()}, c = {c:.2f}\n{forcing_name_list[i]}"
        )

```

```

x_solutions.append(x_vals)
v_solutions.append(v_vals)

# Calculate energy for all cases
def calculate_energy(x, v, mass=m, stiffness=k):
    """Calculate total mechanical energy"""
    kinetic = 0.5 * mass * v**2
    potential = 0.5 * stiffness * x**2
    return kinetic + potential

# Calculate energy arrays for all cases
energy_solutions = []
for x_vals, v_vals in zip(x_solutions, v_solutions):
    E_vals = calculate_energy(x_vals, v_vals)
    energy_solutions.append(E_vals)

# Figure setup
fig = plt.figure(figsize=(14, 10))
gs = GridSpec(
    3,
    2,
    figure=fig,
    hspace=0.35,
    wspace=0.55,
    left=0.05,
    right=0.95,
    top=0.88,
    bottom=0.1,
    width_ratios=[1.5, 1],
)

# Left column: Mass-spring animation
ax = fig.add_subplot(gs[:, 0])

# Right column: Three plots
ax_position = fig.add_subplot(gs[0, 1])
ax_phase = fig.add_subplot(gs[1, 1])
ax_energy = fig.add_subplot(gs[2, 1])

# Update supptitle
c_str = ", ".join([str(c) for c in c_values])
resonant_str = (
    f",  $f_{{res}} = {{resonant_freq:.2f}}$  Hz" if resonant_freq is not None else ""
)
fig.suptitle(
    f"Mass-Spring-Damper System Analysis: Driven Damped Motion\n"
    f" $m = {{m}}$  kg,  $k = {{k}}$  N/m,  $c_{{crit}} = {{c\_critical:.2f}}$  N.s/m, "
    f" $\omega_0 = {{omega_n:.2f}}$  rad/s{{resonant_str}}",
    fontsize=14,
    fontweight="bold",
)

# Set up coordinate system based on actual maximum amplitudes
max_amplitude = max(np.max(np.abs(x)) for x in x_solutions)
max_amplitude = max(max_amplitude, abs(x0)) * 1.2 # Add margin
wall_x = -max_amplitude * 1.2
spring_length = abs(wall_x) # Natural length of spring
total_height = (num_cases - 1) * vertical_separation

ax.set_xlim(wall_x - 0.02, max_amplitude * 1.1)
ax.set_ylim(-total_height - 0.15, 0.15)
ax.set_xlabel("Position (m)", fontsize=12)
ax.set_ylabel("")

```

```

ax.set_yticks([])

ax.axvline(wall_x, color="black", linewidth=15, zorder=4)
ax.axvline(0, color="gray", linestyle="--", alpha=0.3, linewidth=1)
ax.axvline(x0, color="gray", linestyle="--", alpha=0.3, linewidth=1)
ax.axvline(-x0, color="gray", linestyle="--", alpha=0.3, linewidth=1)

for i in range(num_cases):
    y_pos = -i * vertical_separation
    ax.axhline(y_pos, color="gray", alpha=0.3, linewidth=0.5)

# Initialize artists for each system
masses = []
springs = []
text_boxes = []

# Plot elements for right column
lines_pos = []
points_pos = []
lines_phase = []
points_phase = []
lines_energy = []
points_energy = []

for i, (c, label) in enumerate(zip(c_values, case_labels)):
    color = case_colors[i % len(case_colors)]
    edge_color = f"dark{color}"

    # Mass
    mass = ax.scatter(
        [],
        [],
        s=400,
        c=color,
        marker="s",
        edgecolor=edge_color,
        linewidth=2,
        label=label,
        zorder=5,
    )
    masses.append(mass)

    # Spring
    (spring,) = ax.plot([], [], color=color, lw=2, alpha=0.8, zorder=3)
    springs.append(spring)

    # Text box
    y_text_pos = 0.98 - i * 0.18
    text = ax.text(
        1.02,
        y_text_pos,
        "",
        transform=ax.transAxes,
        fontsize=8,
        va="top",
        ha="left",
        bbox=dict(
            boxstyle="round,pad=0.4",
            facecolor=color,
            alpha=0.25,
            edgecolor="black",
            linewidth=1.5,
        ),
    ),

```

```

        family="monospace",
    )
    text_boxes.append(text)

    # Create elements for position plot
    (line_pos,) = ax_position.plot(
        [], [], color=color, linewidth=2, alpha=0.7, label=label
    )
    point_pos = ax_position.scatter(
        [], [], s=50, c=color, zorder=5, edgecolor=edge_color
    )
    lines_pos.append(line_pos)
    points_pos.append(point_pos)

    # Create elements for phase space plot
    (line_phase,) = ax_phase.plot(
        [], [], color=color, linewidth=1.5, alpha=0.6, label=label
    )
    point_phase = ax_phase.scatter(
        [], [], s=60, c=color, zorder=5, edgecolor=edge_color
    )
    lines_phase.append(line_phase)
    points_phase.append(point_phase)

    # Create elements for energy plot
    (line_energy,) = ax_energy.plot(
        [], [], color=color, linewidth=2, alpha=0.7, label=label
    )
    point_energy = ax_energy.scatter(
        [], [], s=50, c=color, zorder=5, edgecolor=edge_color
    )
    lines_energy.append(line_energy)
    points_energy.append(point_energy)

# Add legend
ax.legend(
    loc="upper right",
    fontsize=9,
    fancybox=True,
    shadow=True,
    facecolor="white",
    edgecolor="darkgray",
    markerscale=0.5,
    labelspring=1.1,
)

# Configure position vs time plot
max_x = max(np.max(np.abs(x)) for x in x_solutions)
ax_position.set_xlim(0, 1.1 * duration)
ax_position.set_ylim(-max_x * 1.15, max_x * 1.2)
ax_position.set_xlabel("Time (s)", fontsize=11)
ax_position.set_ylabel("Position  $x$  (m)", fontsize=11)
ax_position.set_title("Position vs Time", fontsize=12, fontweight="bold")
ax_position.grid(True, alpha=0.3)

# Configure phase space plot
max_v = max(np.max(np.abs(v)) for v in v_solutions)
ax_phase.set_xlim(-max_x * 1.15, max_x * 1.15)
ax_phase.set_ylim(-max_v * 1.15, max_v * 1.2)
ax_phase.set_xlabel("Position  $x$  (m)", fontsize=11)
ax_phase.set_ylabel("Velocity  $v$  (m/s)", fontsize=11)
ax_phase.set_title("Phase Space ( $v$  vs  $x$ )", fontsize=12, fontweight="bold")
ax_phase.grid(True, alpha=0.3)

```

```

# Configure energy vs time plot
max_E = max(np.max(E) for E in energy_solutions)
ax_energy.set_xlim(0, 1.1 * duration)
ax_energy.set_ylim(-0.05 * max_E, max_E * 1.2)
ax_energy.set_xlabel("Time (s)", fontsize=11)
ax_energy.set_ylabel("Energy  $E$  (J)", fontsize=11)
ax_energy.set_title("Total Energy vs Time", fontsize=12, fontweight="bold")
ax_energy.grid(True, alpha=0.3)

def draw_spring_horizontal(start_x, end_x, num_coils, radius=0.03, y_center=0.0):
    """Draw a horizontal spring from start_x to end_x"""
    if abs(end_x - start_x) < 1e-9:
        return np.array([start_x]), np.array([y_center])

    t = np.linspace(0, num_coils * 2 * np.pi, 300)
    x_spring = np.linspace(start_x, end_x, 300)
    y_spring = y_center + radius * np.sin(t)

    return x_spring, y_spring

def init():
    """Initialize animation"""
    for i, (mass, spring) in enumerate(zip(masses, springs)):
        y_pos = -i * vertical_separation
        mass.set_offsets(np.array([[np.nan, y_pos]]))
        spring.set_data([], [])
    for line_pos, line_phase, line_energy in zip(
        lines_pos, lines_phase, lines_energy
    ):
        line_pos.set_data([], [])
        line_phase.set_data([], [])
        line_energy.set_data([], [])
    return tuple(
        masses
        + springs
        + text_boxes
        + lines_pos
        + points_pos
        + lines_phase
        + points_phase
        + lines_energy
        + points_energy
    )

def update(frame):
    """Update animation for each frame"""
    # Current time
    t = t_array[frame]
    num_osc = min(t / T, oscillations)

    # Update title
    ax.set_title(
        f"Time: {t:.2f} s | Oscillation: {num_osc:.2f}/{oscillations}",
        fontsize=12,
        fontweight="bold",
        pad=10,
    )

    artists = []

    # Update each system
    for i, (c, x_vals, v_vals, E_vals) in enumerate(

```

```

zip(c_values, x_solutions, v_solutions, energy_solutions)
):
    y_pos = -i * vertical_separation
    x_current = x_vals[frame]
    v_current = v_vals[frame]
    E_current = E_vals[frame]

    # Update spring and mass animation
    x_spring, y_spring = draw_spring_horizontal(
        wall_x, x_current, num_coils, radius=0.03, y_center=y_pos
    )
    springs[i].set_data(x_spring, y_spring)
    masses[i].set_offsets([[x_current, y_pos]])

    # Calculate spring deformation
    spring_deform = abs((x_current - wall_x) - spring_length)

    # Calculate damping ratio
    zeta = c / c_critical if c > 0 else 0

    # Update text box
    info = (
        f"{box_labels[i]}\n"
        f"{'-' * 22}\n"
        f"Position:    {x_current:+.4f} m\n"
        f"Velocity:     {v_current:+.4f} m/s\n"
        f"Energy:       {E_current:.4f} J\n"
        f"Spring def:   {spring_deform:.4f} m\n"
        f"Damping:      {c:.3f} N·s/m\n"
        f"Damp ratio:   {zeta:.3f}"
    )
    text_boxes[i].set_text(info)

    # Update position vs time plot
    t_data = t_array[: frame + 1]
    x_data = x_vals[: frame + 1]
    lines_pos[i].set_data(t_data, x_data)
    points_pos[i].set_offsets([[t, x_current]])

    # Update phase space plot
    v_data = v_vals[: frame + 1]
    lines_phase[i].set_data(x_data, v_data)
    points_phase[i].set_offsets([[x_current, v_current]])

    # Update energy vs time plot
    E_data = E_vals[: frame + 1]
    lines_energy[i].set_data(t_data, E_data)
    points_energy[i].set_offsets([[t, E_current]])

    artists.extend(
        [
            masses[i],
            springs[i],
            text_boxes[i],
            lines_pos[i],
            points_pos[i],
            lines_phase[i],
            points_phase[i],
            lines_energy[i],
            points_energy[i],
        ]
    )
)

```

```

        return tuple(artists)

# Create animation
anim = FuncAnimation(
    fig,
    update,
    frames=num_frames,
    init_func=init,
    blit=False,
    interval=int(1000.0 / fps),
    repeat=False,
)

# Save or show
if save_animation:
    if filename is None:
        c_str = "_".join([f"c{c}" for c in c_values])
        ext = "gif" if file_format == "gif" else "mp4"
        filename = f"resonance_analysis_m{m}_k{k}_{c_str}.{ext}"

    save_dir = "OUTPUTS/ANIMATIONS"
    os.makedirs(save_dir, exist_ok=True)
    filepath = os.path.join(save_dir, filename)

    try:
        print(f"Saving animation to {os.path.abspath(filepath)}...")
        if file_format == "gif":
            anim.save(filepath, writer="ffmpeg", fps=fps, codec="gif", dpi=80)
        else:
            anim.save(filepath, writer="ffmpeg", fps=fps, dpi=150)
        print(f"Animation saved as: {filepath}")
    except Exception as e:
        print(f"Error saving animation: {e}")
    plt.close(fig)
else:
    plt.show()

return anim

def main():
    """Main function to run the driven damped motion animation."""

    print("\n" + "=" * 70)
    print("  Mass-Spring-Damper Animation: Driven Damped Motion")
    print("=" * 70)
    print("\nThis animation studies driven damped motion with:")
    print("  - Case 1: Undamped (no forcing)")
    print("  - Case 2: Damped with  $f < f_{\text{resonance}}$ ")
    print("  - Case 3: Damped with  $f = f_{\text{resonance}}$ ")
    print("  - Case 4: Damped with  $f > f_{\text{resonance}}$ \n")

    # Default parameters
    defaults = {
        "m": 1.0,
        "k": 10.0,
        "c": 2.0,
        "x0": 0.1,
        "v0": 0.0,
        "forcing_amplitude": 2.0,
        "oscillations": 8.0,
        "fps": 30,
        "file_format": "gif",
    }

```

```

}

use_defaults = input("Use default parameters? (y/n) [y]: ").strip().lower()

if use_defaults == "n":
    try:
        print("\n--- System Parameters ---")
        m = float(input("Mass (m) [1.0 kg]: ") or 1.0)
        k = float(input("Stiffness (k) [10.0 N/m]: ") or 10.0)
        c = float(input("Damping coefficient (c) [2.0 N·s/m]: ") or 2.0)

        # Initial conditions
        x0 = float(input("Initial displacement/amplitude x0 [0.1 m]: ") or 0.1)
        v0 = float(input("Initial velocity v0 [0.0 m/s]: ") or 0.0)

        # Forcing parameters
        print("\n--- Forcing Parameters ---")
        forcing_amplitude = float(input("Forcing amplitude [2.0 N]: ") or 2.0)

        print("\n--- Animation Parameters ---")
        oscillations = float(input("Number of periods to display [8.0]: ") or 8.0)
        fps = int(input("Frames per second [30]: ") or 30)
        file_format = (
            input("Animation file format (gif/mp4) [gif]: ").strip().lower()
            or "gif"
        )

    except ValueError as e:
        print(f"\nInvalid input: {e}")
        print("Using default parameters.")
        m = defaults["m"]
        k = defaults["k"]
        c = defaults["c"]
        x0 = defaults["x0"]
        v0 = defaults["v0"]
        forcing_amplitude = defaults["forcing_amplitude"]
        oscillations = defaults["oscillations"]
        fps = defaults["fps"]
        file_format = defaults["file_format"]
    else:
        m = defaults["m"]
        k = defaults["k"]
        c = defaults["c"]
        x0 = defaults["x0"]
        v0 = defaults["v0"]
        forcing_amplitude = defaults["forcing_amplitude"]
        oscillations = defaults["oscillations"]
        fps = defaults["fps"]
        file_format = defaults["file_format"]

    # Validate parameters
    if m <= 0:
        print("Warning: Mass must be positive, setting to 1.0")
        m = 1.0
    if k <= 0:
        print("Warning: Stiffness must be positive, setting to 10.0")
        k = 10.0
    if c < 0:
        print("Warning: Damping coefficient must be non-negative")
        c = max(0, c)

    # Calculate natural frequency and resonant frequency
    omega_n = np.sqrt(k / m)

```

```

f_n = omega_n / (2 * np.pi)

# For damped system, resonant frequency (amplitude resonance):
# omega_res = sqrt(omega_n^2 - 2*zeta^2*omega_n^2) = omega_n * sqrt(1 - 2*zeta^2)
# where zeta = c / (2*sqrt(m*k))
c_critical = 2 * np.sqrt(k * m)
zeta = c / c_critical

if zeta < 1 / np.sqrt(2):
    omega_res = omega_n * np.sqrt(1 - 2 * zeta**2)
else:
    # For higher damping, use natural frequency
    omega_res = omega_n

f_res = omega_res / (2 * np.pi)

# Check if resonance exists for this damping level
resonance_exists = zeta < 1 / np.sqrt(2)

if not resonance_exists:
    print("\nWARNING: Damping is too high for resonance!")
    print(
        f" Current  $\zeta$  = {zeta:.3f}, but resonance requires  $\zeta < \{1 / np.sqrt(2):.3f\}$ "
    )
    print(f" Using natural frequency f_n = {f_n:.2f} Hz as reference instead.")
    f_ref = f_n
else:
    f_ref = f_res

# Set up 4 cases: undamped (no forcing), then 3 damped with different driving
→frequencies
c_values = [0, c, c, c]

# Get driving frequencies from user
print("\n--- Driving Frequencies ---")
if resonance_exists:
    print(f"Resonant frequency: f_res = {f_res:.2f} Hz")
    print(
        f"Suggested frequencies: {f_res * 0.5:.2f} Hz (below), {f_res:.2f} Hz (at),
→{f_res * 1.5:.2f} Hz (above)"
    )
else:
    print(f"Natural frequency: f_n = {f_n:.2f} Hz (no resonance for this damping)")
    print(
        f"Suggested frequencies: {f_n * 0.5:.2f} Hz (low), {f_n:.2f} Hz (mid), {f_n *
→1.5:.2f} Hz (high)"
    )

freq_input = input(
    f"Enter 3 driving frequencies (comma-separated) [{f_ref * 0.5:.2f}, {f_ref:.2f},
→{f_ref * 1.5:.2f}]: "
).strip()

if freq_input:
    try:
        freq_list = [float(f.strip()) for f in freq_input.split(",")]
        if len(freq_list) != 3:
            print(
                f"Warning: Expected 3 frequencies, got {len(freq_list)}. Using
→defaults."
            )
        f_low, f_at, f_high = f_ref * 0.5, f_ref, f_ref * 1.5
    else:

```

```

        f_low, f_at, f_high = freq_list
    except ValueError:
        print("Invalid input. Using default frequencies.")
        f_low, f_at, f_high = f_ref * 0.5, f_ref, f_ref * 1.5
else:
    f_low, f_at, f_high = f_ref * 0.5, f_ref, f_ref * 1.5

# Create forcing functions for each case
forcing_funcs = [
    no_forcing, # Undamped case - no forcing
    lambda t: sinusoidal_forcing(t, forcing_amplitude, f_low), # noqa: E731
    lambda t: sinusoidal_forcing(t, forcing_amplitude, f_at), # noqa: E731
    lambda t: sinusoidal_forcing(t, forcing_amplitude, f_high), # noqa: E731
]

if resonance_exists:
    forcing_names = [
        "No forcing",
        f"f={f_low:.2f} Hz (< f_res)",
        f"f={f_at:.2f} Hz (= f_res)",
        f"f={f_high:.2f} Hz (> f_res)",
    ]
else:
    forcing_names = [
        "No forcing",
        f"f={f_low:.2f} Hz (low)",
        f"f={f_at:.2f} Hz (mid)",
        f"f={f_high:.2f} Hz (high)",
    ]

save_anim = input("\nSave animation as GIF? (y/n) [n]: ").strip().lower() == "y"
filename = None
if save_anim:
    filename_input = input("Enter filename (or press Enter for default): ").strip()
    if filename_input:
        if not filename_input.endswith((".gif", ".mp4")):
            filename_input += ".gif" if file_format == "gif" else ".mp4"
        filename = filename_input

print("\n" + "-" * 70)
print("Animating with:")
print(f"  Mass (m):           {m} kg")
print(f"  Stiffness (k):       {k} N/m")
print(f"  Damping (c):         {c} N·s/m")
print(f"  Damping ratio (ζ):   {zeta:.3f}")
print(f"  Natural freq (f_n):  {f_n:.2f} Hz")
if resonance_exists:
    print(f"  Resonant freq (f_res): {f_res:.2f} Hz ✓")
else:
    print(f"  Resonant freq:       N/A (damping too high,  $\zeta \geq 1/\sqrt{2}$ )")
print(f"  Initial disp (x0):   {x0} m")
print(f"  Initial vel (v0):    {v0} m/s")
print(f"  Forcing amplitude:   {forcing_amplitude} N")
print(f"  Driving frequencies: {f_low:.2f}, {f_at:.2f}, {f_high:.2f} Hz")
print(f"  Periods:             {oscillations}")
print(f"  FPS:                 {fps}")
print(f"  File format:         {file_format}")
print("-" * 70 + "\n")

anim = animate_damped_driven_oscillation_with_plots(
    m=m,
    c_values=c_values,
    k=k,

```

```

x0=x0,
v0=v0,
oscillations=oscillations,
fps=fps,
save_animation=save_anim,
filename=filename,
file_format=file_format,
forcing_func=forcing_funcs,
forcing_name=forcing_names,
resonant_freq=f_res,
)

print("\nAnimation completed!")
return anim

if __name__ == "__main__":
    animation = main()

```

See the animation in the GitHub repository: [Animation 7](#)

5.8 Resonance and Frequency Response

For

$$m\ddot{x} + c\dot{x} + kx = F_0 \cos(\omega t), \quad \omega_0 = \sqrt{k/m}, \quad \beta = \frac{c}{2m}, \quad \zeta = \frac{\beta}{\omega_0},$$

the steady-state displacement is

$$x(t) = A(\omega) \cos(\omega t - \phi), \quad A(\omega) = \frac{F_0/m}{\sqrt{(\omega_0^2 - \omega^2)^2 + (2\beta\omega)^2}}.$$

- **Displacement (Amplitude) resonance** = peak of $A(\omega)$
Maximize $A(\omega)$ (equivalently minimize the denominator). The peak occurs at

$$\omega_r = \sqrt{\omega_0^2 - 2\beta^2} = \omega_0 \sqrt{1 - 2\zeta^2},$$

which exists only if

$$\zeta < \frac{1}{\sqrt{2}} \quad \left(\text{equivalently } c < \frac{c_{\text{crit}}}{\sqrt{2}} \right).$$

Crucial point: ω_r **depends on damping**, so different c (or ζ) gives a different resonant frequency. Also $\omega_r < \omega_0$ whenever $c > 0$ and the peak exists.

- **Velocity resonance** = peak of steady-state velocity amplitude
Since $v(t) = \dot{x}$, the velocity amplitude is

$$A_v(\omega) = \omega A(\omega).$$

Consider

$$A_v(\omega)^2 = \frac{(F_0/m)^2 \omega^2}{(\omega_0^2 - \omega^2)^2 + (2\beta\omega)^2}.$$

Differentiating and setting to zero gives the maximum at

$$\boxed{\omega = \omega_0}$$

(independent of β / c).

Crucial point: **velocity peaks at the natural frequency for all damping values** (the peak height changes with damping, but the peak location stays at ω_0).

- **“Energy resonance”** = peak of average energy absorbed per unit time
Instantaneous input power is $P(t) = F(t)v(t)$, so the cycle average is

$$\langle P \rangle = \langle Fv \rangle.$$

In steady state, the average input power equals average dissipation in the damper:

$$\langle P \rangle = \langle cv^2 \rangle = \frac{1}{2} c A_v(\omega)^2.$$

Because $A_v(\omega)^2$ is maximized at ω_0 , it follows immediately that

$$\langle P \rangle \text{ peaks at } \omega = \omega_0 \text{ for all } c > 0.$$

Physical “why”: at $\omega = \omega_0$, the phase lag is $\phi = \pi/2$, so the force is maximally in-phase with velocity, giving maximum net energy transfer per cycle.

This is implemented in the code cell below as follows:

- It computes steady-state displacement amplitude, velocity amplitude, and average power by simulating many periods, discarding the transient, then measuring amplitudes and $\langle P \rangle = \langle Fv \rangle$ in `compute_steady_state_response`.
- In `plot_and_save_frequency_response`, it marks **displacement resonance** at $f_r = f_0\sqrt{1-2\zeta^2}$ only when $\zeta < 1/\sqrt{2}$ (so each damping curve can have a different marked f_r) and Marks **velocity** and **power** peaks at the same reference line f_0 (natural frequency) for all damped cases.

```
[17]: def compute_steady_state_response(
    m,
    c,
    k,
    x0,
    v0,
    forcing_amplitude,
    driving_freq,
    num_periods=20,
    transient_periods=10,
):
    """
    Compute steady-state amplitude, velocity amplitude, and average power for a driven
    ↪damped oscillator.

    Parameters:
        m: Mass
        c: Damping coefficient
        k: Stiffness
        x0: Initial displacement
        v0: Initial velocity
        forcing_amplitude: Amplitude of driving force
        driving_freq: Driving frequency (Hz)
        num_periods: Total number of periods to simulate
        transient_periods: Number of periods to discard (transient)

    Returns:
        amplitude: Steady-state displacement amplitude
        velocity_amplitude: Steady-state velocity amplitude
        avg_power: Average power absorbed
    """
    omega = 2 * np.pi * driving_freq
    period = 1.0 / driving_freq if driving_freq > 0 else 1.0

    # Handle undamped case analytically to avoid non-decaying transients
    if c == 0:
        omega_0 = np.sqrt(k / m)
        denom = np.abs(omega_0**2 - omega**2)
        # Displacement amplitude A(omega) = (F0/m) / |omega_0^2 - omega^2|
        A_ss = (forcing_amplitude / m) / denom if denom > 0 else np.inf
        # Velocity amplitude A_v(omega) = omega * A(omega)
        Av_ss = omega * A_ss if np.isfinite(A_ss) else np.inf
        # Average power over a cycle is zero for undamped sinusoidal forcing
        return A_ss, Av_ss, 0.0

    # Total simulation time
    duration = num_periods * period
    fps = 100 # High resolution for accurate steady-state
    h = period / fps
```

```

# Define forcing function
forcing_func = lambda t: sinusoidal_forcing(t, forcing_amplitude, driving_freq) #U
↪noqa: E731

# Solve
t_array, x_array, v_array = rk4_mass_spring_damper(
    m, c, k, x0, v0, 0.0, duration, h, forcing_func
)

# Discard transient: keep only last (num_periods - transient_periods) periods
transient_time = transient_periods * period
steady_idx = np.where(t_array >= transient_time)[0]

if len(steady_idx) < 10:
    # Not enough data points
    return 0, 0, 0

x_steady = x_array[steady_idx]
v_steady = v_array[steady_idx]
t_steady = t_array[steady_idx]

# Compute amplitudes
amplitude = (np.max(x_steady) - np.min(x_steady)) / 2.0
velocity_amplitude = (np.max(v_steady) - np.min(v_steady)) / 2.0

# Compute average power: <P> = <F(t) * v(t)>
F_steady = np.array([forcing_func(t) for t in t_steady])
power_instantaneous = F_steady * v_steady
avg_power = np.mean(power_instantaneous)

return amplitude, velocity_amplitude, avg_power

def plot_and_save_frequency_response(
    m,
    k,
    forcing_amplitude,
    c_values,
    x0=0.1,
    v0=0.0,
    save_figure=False,
    filename=None,
):
    """
    Plot frequency response curves for amplitude, velocity, and power resonance.
    Shows curves for different damping levels.

    Parameters:
        m: Mass
        k: Stiffness
        forcing_amplitude: Amplitude of driving force
        c_values: List of damping coefficients to analyze
        x0: Initial displacement
        v0: Initial velocity
        save_figure: Whether to save the figure
        filename: Filename to save the figure
    """

    # Calculate natural frequency
    omega_n = np.sqrt(k / m)
    f_n = omega_n / (2 * np.pi)
    c_critical = 2 * np.sqrt(k * m)

```

```

# Define damping levels from c_values
available_colors = [
    "blue",
    "green",
    "orange",
    "red",
    "purple",
    "brown",
    "magenta",
    "cyan",
]
damping_levels = []
for i, c in enumerate(c_values):
    if c == 0:
        label = "Undamped"
    else:
        zeta = c / c_critical
        label = f"c={c:.2f} ($\\zeta$={zeta:.2f})"
        color = available_colors[i % len(available_colors)]
        damping_levels.append((c, label, color))

# Frequency range: 0.1*f_n to 3*f_n
freq_range = np.linspace(0.1 * f_n, 2.5 * f_n, 100)

print("\nSystem parameters:")
print(f"  Mass (m):           {m} kg")
print(f"  Stiffness (k):       {k} N/m")
print(f"  Forcing amplitude:   {forcing_amplitude} N")
print(f"  Natural frequency:   {f_n:.2f} Hz")
print(f"  Critical damping:    {c_critical:.2f} N·s/m")
print("\nComputing frequency response...")

# Storage for results
results = {}

for c, label, color in damping_levels:
    print(f"  Processing {label}...")
    amplitudes = []
    velocity_amplitudes = []
    avg_powers = []

    for f in freq_range:
        A, A_v, P_avg = compute_steady_state_response(
            m, c, k, x0, v0, forcing_amplitude, f
        )
        amplitudes.append(A)
        velocity_amplitudes.append(A_v)
        avg_powers.append(P_avg)

    results[label] = {
        "c": c,
        "color": color,
        "amplitudes": np.array(amplitudes),
        "velocity_amplitudes": np.array(velocity_amplitudes),
        "avg_powers": np.array(avg_powers),
    }

print("\nGenerating plots...")

# Create figure with 3 subplots with shared x-axis
fig, axes = plt.subplots(3, 1, figsize=(10, 14), sharex=True)
fig.subplots_adjust(top=0.9, bottom=0.05, hspace=0.24, left=0.08, right=0.95)

```

```

fig.suptitle(
    f"Frequency Response of Driven Damped Oscillator\n"
    f"$m$={m} kg, $k$={k} N/m, $F_0$={forcing_amplitude} N, $\omega_0$={omega_n:.2f} rad/s, $f_0$={f_n:.2f} Hz",
    fontsize=14,
    fontweight="bold",
)

# Plot 1: Amplitude vs Frequency
ax1 = axes[0]
for label, data in results.items():
    c = data["c"]
    zeta = c / c_critical if c > 0 else 0
    label_with_zeta = f"{label}"
    ax1.plot(
        freq_range,
        data["amplitudes"],
        color=data["color"],
        linewidth=2.5,
        label=label_with_zeta,
    )

    # Mark displacement resonance peak if it exists
    if c > 0 and zeta < 1 / np.sqrt(2):
        omega_r = omega_n * np.sqrt(1 - 2 * zeta**2)
        f_r = omega_r / (2 * np.pi)
        # Find closest frequency in our range
        idx = np.argmin(np.abs(freq_range - f_r))
        ax1.plot(
            freq_range[idx],
            data["amplitudes"][idx],
            marker="o",
            markersize=8,
            color=data["color"],
            markeredgecolor="black",
            markeredgewidth=1.5,
        )
        ax1.annotate(
            f"$f_r$={f_r:.2f} Hz",
            xy=(freq_range[idx], data["amplitudes"][idx]),
            xytext=(15, 15),
            textcoords="offset points",
            fontsize=9,
            color=data["color"],
            arrowprops=dict(arrowstyle="->", color=data["color"], lw=1.5),
        )

# Natural frequency reference line
ax1.axvline(
    f_n,
    color="gray",
    linestyle="--",
    linewidth=2,
    alpha=0.6,
    label="$f_0$ (natural)",
)

# Add resonance reference lines for each damping case where it exists
for label, data in results.items():
    c_val = data["c"]
    if c_val <= 0:
        continue
    zeta = c_val / c_critical
    if zeta < 1 / np.sqrt(2):

```

```

        omega_r = omega_n * np.sqrt(1 - 2 * zeta**2)
        f_r = omega_r / (2 * np.pi)
        ax1.axvline(
            f_r,
            color=data["color"],
            linestyle=":",
            linewidth=1.8,
            alpha=0.8,
            label=f"$f_r$ ($\\zeta$={zeta:.2f})",
        )

    ax1.set_ylabel("Displacement Amplitude $A$ (m)", fontsize=12)
    ax1.set_title(
        "Displacement Resonance: $A(\\omega)$ vs $f$\\n"
        f"Peak at $f_r = f_0\\sqrt{{1-2\\zeta^2}}$ (exists only if $\\zeta < 1/\\sqrt{{2}}$  

        ↪ approx {1 / np.sqrt(2):.3f}$)",
        fontsize=11,
        fontweight="bold",
    )
    ax1.grid(True, alpha=0.3)
    ax1.legend(loc="upper right", fontsize=9, ncol=1 if len(c_values) <= 4 else 2)
    ax1.set_xlim(freq_range[0], freq_range[-1])

    # Adjust y-limit to show damped peaks clearly
    damped_max_amplitude = max(
        np.max(data["amplitudes"]) for _, data in results.items() if data["c"] > 0
    )
    if damped_max_amplitude > 0:
        ax1.set_ylim(0, damped_max_amplitude * 1.25)

    # Plot 2: Velocity Amplitude vs Frequency
    ax2 = axes[1]
    for label, data in results.items():
        c = data["c"]
        zeta = c / c_critical if c > 0 else 0
        label_with_zeta = f"{label}"
        ax2.plot(
            freq_range,
            data["velocity_amplitudes"],
            color=data["color"],
            linewidth=2.5,
            label=label_with_zeta,
        )

    # Mark peak at natural frequency
    idx_peak = np.argmin(np.abs(freq_range - f_n))
    ax2.plot(
        freq_range[idx_peak],
        data["velocity_amplitudes"][idx_peak],
        marker="^",
        markersize=10,
        color=data["color"],
        markeredgewidth=1.5,
        markeredgecolor="black",
    )

    ax2.axvline(
        f_n,
        color="gray",
        linestyle="--",
        linewidth=2,
        alpha=0.6,
        label="$f_0$ (natural)",
    )

```

```

)
ax2.set_ylabel("Velocity Amplitude $A_v$ (m/s)", fontsize=12)
ax2.set_title(
    "Velocity Resonance: $A_v(\\omega) = \\omega A(\\omega)$ vs $f$\\n"
    "Peak ALWAYS at $f_0$ (natural frequency), independent of damping",
    fontsize=11,
    fontweight="bold",
)
ax2.grid(True, alpha=0.3)
ax2.legend(loc="upper right", fontsize=9, ncol=1 if len(c_values) <= 4 else 2)
ax2.set_xlim(freq_range[0], freq_range[-1])

damped_max_velocity = max(
    np.max(data["velocity_amplitudes"])
    for _, data in results.items()
    if data["c"] > 0
)

if damped_max_velocity > 0:
    ax2.set_ylim(0, damped_max_velocity * 1.25)

# Plot 3: Average Power vs Frequency
ax3 = axes[2]
for label, data in results.items():
    c = data["c"]
    if c == 0:
        continue # Skip undamped (power would be zero on average)
    zeta = c / c_critical
    label_with_zeta = f"{label}"
    ax3.plot(
        freq_range,
        data["avg_powers"],
        color=data["color"],
        linewidth=2.5,
        label=label_with_zeta,
    )

# Mark peak at natural frequency
idx_peak = np.argmin(np.abs(freq_range - f_n))
ax3.plot(
    freq_range[idx_peak],
    data["avg_powers"][idx_peak],
    marker="s",
    markersize=9,
    color=data["color"],
    markeredgewidth=1.5,
    markeredgecolor="black",
)

ax3.axvline(
    f_n,
    color="gray",
    linestyle="--",
    linewidth=2,
    alpha=0.6,
    label="$f_0$ (natural)",
)

ax3.set_xlabel("Driving Frequency $f$ (Hz)", fontsize=12)
ax3.set_ylabel("Average Power $\\langle P \\rangle$ (W)", fontsize=12)
ax3.set_title(
    "Energy (Power) Resonance: $\\langle P \\rangle(\\omega) \\rangle$ vs $f$\\n"
    "Peak ALWAYS at $f_0$ (maximum energy transfer at natural frequency)",
    fontsize=11,

```

```

        fontweight="bold",
    )
    ax3.grid(True, alpha=0.3)
    ax3.legend(loc="upper right", fontsize=9, ncol=1 if len(c_values) <= 4 else 2)
    ax3.set_xlim(freq_range[0], freq_range[-1])

# Save or show
    if save_figure:
        if filename is None:
            c_str = "_".join([f"{c:.2f}" for c in c_values])
            filename = f"frequency_response_m{m}_k{k}_c{c_str}.png"

        save_dir = "OUTPUTS/PLOTS"
        os.makedirs(save_dir, exist_ok=True)
        filepath = os.path.join(save_dir, filename)

        try:
            fig.savefig(filepath, dpi=150, bbox_inches="tight")
            print(f"\nPlot saved as: {os.path.abspath(filepath)}")
        except Exception as e:
            print(f"Error saving plot: {e}")

    plt.show()
    print("\nAnalysis complete!")

def main():
    """Main function to run frequency response analysis."""

    print("\n" + "=" * 70)
    print("  Frequency Response Analysis: Steady-State Analysis")
    print("=" * 70)
    print("\nThis analysis plots frequency response curves showing:")
    print("  - Displacement Resonance: Amplitude vs Driving Frequency")
    print("  - Velocity Resonance: Velocity Amplitude vs Driving Frequency")
    print("  - Energy (Power) Resonance: Average Power vs Driving Frequency\n")

    # Default parameters
    defaults = {
        "m": 1.0,
        "k": 10.0,
        "forcing_amplitude": 2.0,
        "x0": 0.1,
        "v0": 0.0,
    }

    # Calculate critical damping for defaults
    c_critical_default = 2 * np.sqrt(defaults["k"] * defaults["m"])

    default_c_values = [
        0.0,
        c_critical_default * 0.1,
        c_critical_default * 0.3,
        c_critical_default * 0.7,
    ]

    use_defaults = input("Use default parameters? (y/n) [y]: ").strip().lower()

    if use_defaults == "n":
        try:
            print("\n--- System Parameters ---")
            m = float(input("Mass (m) [1.0 kg]: ") or 1.0)
            k = float(input("Stiffness (k) [10.0 N/m]: ") or 10.0)

```

```

# Calculate critical damping
c_critical = 2 * np.sqrt(k * m)
print(f"\nCritical damping coefficient: c_crit = {c_critical:.3f} N.s/m")
print(
    f"Resonance exists when  $\zeta < \{1 / \text{np.sqrt}(2):.3f\}$  (i.e.,  $c < \{c\_critical / \sqrt{2}\}$  N.s/m)"
)

# Input damping coefficients
print("\n--- Damping Coefficients ---")
print("Enter damping coefficients (comma-separated).")
print("Include 0 for undamped case.")
print("Suggestions:")
print("  - 0 (undamped)")
print(f"  - {c_critical * 0.1:.2f} (light:  $\zeta=0.10$ )")
print(f"  - {c_critical * 0.3:.2f} (medium:  $\zeta=0.30$ )")
print(f"  - {c_critical * 0.7:.2f} (heavy:  $\zeta=0.70$ )")
print(f"  - {c_critical:.2f} (critical:  $\zeta=1.00$ )")

c_input = input(
    f"c values [0, {c_critical * 0.1:.2f}, {c_critical * 0.3:.2f}, {c_critical * 0.7:.2f}]: "
).strip()

if c_input:
    c_values = [float(c.strip()) for c in c_input.split(",")]
else:
    c_values = [0, c_critical * 0.1, c_critical * 0.3, c_critical * 0.7]

# Initial conditions
print("\n--- Initial Conditions ---")
x0 = float(input("Initial displacement x0 [0.1 m]: ") or 0.1)
v0 = float(input("Initial velocity v0 [0.0 m/s]: ") or 0.0)

# Forcing parameters
print("\n--- Forcing Parameters ---")
forcing_amplitude = float(input("Forcing amplitude [2.0 N]: ") or 2.0)

except ValueError as e:
    print(f"\nInvalid input: {e}")
    print("Using default parameters.")
    m = defaults["m"]
    k = defaults["k"]
    c_values = default_c_values
    x0 = defaults["x0"]
    v0 = defaults["v0"]
    forcing_amplitude = defaults["forcing_amplitude"]
else:
    m = defaults["m"]
    k = defaults["k"]
    c_values = default_c_values
    x0 = defaults["x0"]
    v0 = defaults["v0"]
    forcing_amplitude = defaults["forcing_amplitude"]

save_figure = (
    input("Save frequency response plot? (y/n) [n]: ").strip().lower() == "y"
)
filename = None
if save_figure:
    filename_input = input("Enter filename (or press Enter for default): ").strip()
    if filename_input:

```

```

        if not filename_input.endswith((".png", ".jpg", ".jpeg")):
            filename_input += ".png"
        filename = filename_input

# Validate parameters
if m <= 0:
    print("Warning: Mass must be positive, setting to 1.0")
    m = 1.0
if k <= 0:
    print("Warning: Stiffness must be positive, setting to 10.0")
    k = 10.0
if any(c < 0 for c in c_values):
    print("Warning: Damping coefficients must be non-negative")
    c_values = [max(0, c) for c in c_values]

# Calculate natural frequency
omega_n = np.sqrt(k / m)
f_n = omega_n / (2 * np.pi)
c_critical = 2 * np.sqrt(k * m)

print("\n" + "-" * 70)
print("Analysis parameters:")
print(f"  Mass (m):           {m} kg")
print(f"  Stiffness (k):       {k} N/m")
print(f"  Initial Displacement (x0): {x0} m")
print(f"  Initial Velocity (v0):   {v0} m/s")
print(f"  Natural freq (f_n):     {f_n:.2f} Hz")
print(f"  Critical damping:      {c_critical:.2f} N·s/m")
print(f"  Forcing amplitude:     {forcing_amplitude} N")
print(
    "  Damping coefficients:  "
    + str([float(np.round(c, 3)) for c in c_values if c > 0])
)
print("-" * 70 + "\n")

# Run frequency response analysis
plot_and_save_frequency_response(
    m,
    k,
    forcing_amplitude,
    c_values,
    x0,
    v0,
    save_figure=save_figure,
    filename=filename,
)

if __name__ == "__main__":
    main()

```

Frequency Response Analysis: Steady-State Analysis

This analysis plots frequency response curves showing:

- Displacement Resonance: Amplitude vs Driving Frequency
- Velocity Resonance: Velocity Amplitude vs Driving Frequency
- Energy (Power) Resonance: Average Power vs Driving Frequency

Analysis parameters:

Mass (m): 1.0 kg
Stiffness (k): 10.0 N/m
Initial Displacement (x0): 0.1 m
Initial Velocity (v0): 0.0 m/s
Natural freq (f_n): 0.50 Hz
Critical damping: 6.32 N·s/m
Forcing amplitude: 2.0 N
Damping coefficients: [0.632, 1.897, 4.427]

System parameters:

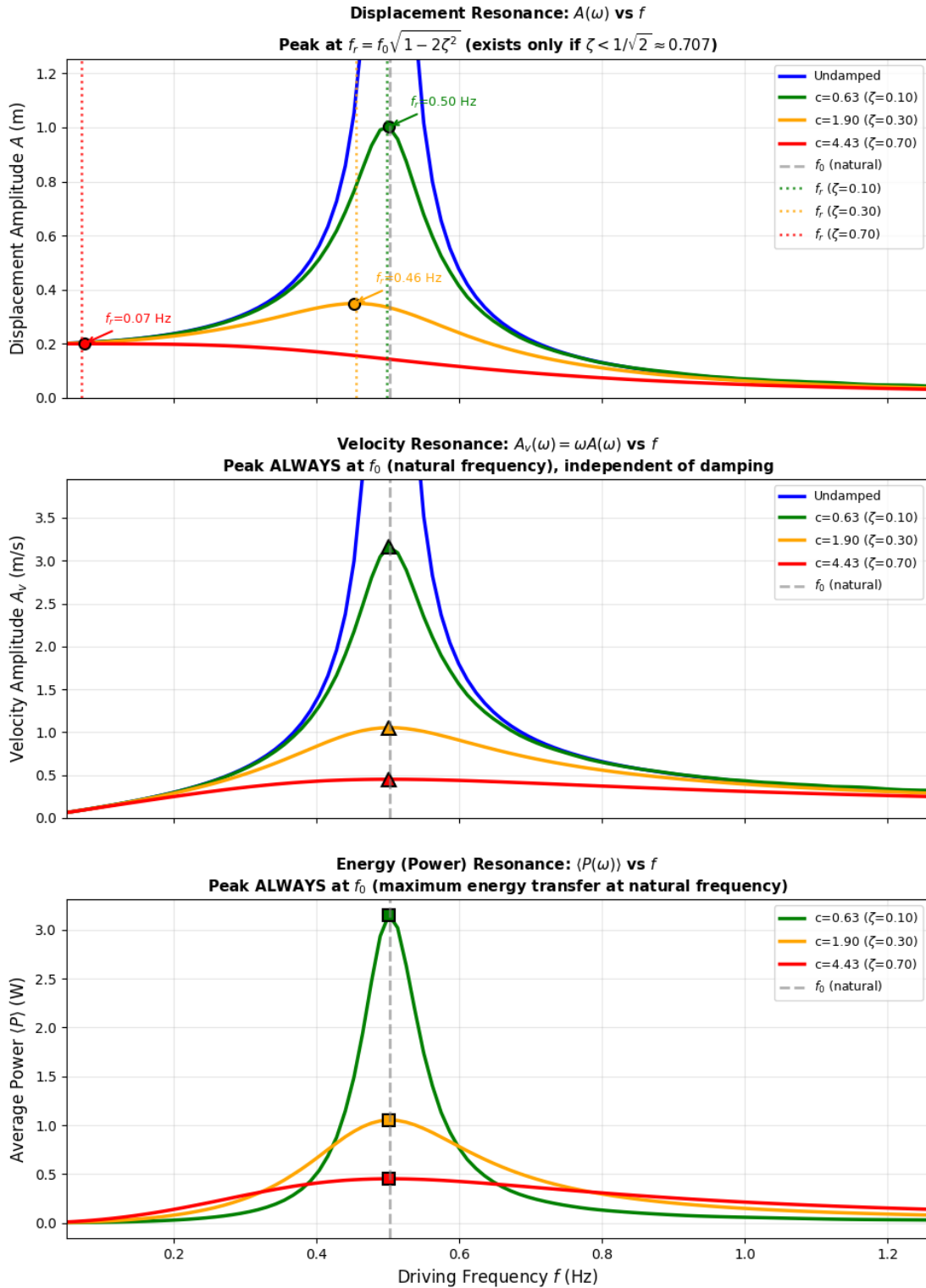
Mass (m): 1.0 kg
Stiffness (k): 10.0 N/m
Forcing amplitude: 2.0 N
Natural frequency: 0.50 Hz
Critical damping: 6.32 N·s/m

Computing frequency response...

Processing Undamped...
Processing c=0.63 ($\zeta=0.10$)...
Processing c=1.90 ($\zeta=0.30$)...
Processing c=4.43 ($\zeta=0.70$)...

Generating plots...

Frequency Response of Driven Damped Oscillator
 $m=1.0 \text{ kg}$, $k=10.0 \text{ N/m}$, $F_0=2.0 \text{ N}$, $\omega_0=3.16 \text{ rad/s}$, $f_0=0.50 \text{ Hz}$



Analysis complete!

5.9 Phase Space and Energy Analysis

For a mass–spring–damper,

$$m\ddot{x} + c\dot{x} + kx = F(t), \quad v = \dot{x},$$

the phase space is the state plane (x, v) .

1. Simply damped (free) oscillator: $F(t) = 0$

- **Dynamics:**

$$\dot{x} = v, \quad \dot{v} = -\frac{k}{m}x - \frac{c}{m}v.$$

- **Phase portrait:** trajectories decay toward the origin.
 - Underdamped: spiral inward to $(0, 0)$ (stable focus).
 - Critically/overdamped: approach the origin without spiraling (stable node).
- **Key idea:** the origin is the only attractor because no energy is injected.

Energy evolution (free damped): monotone decay

- Total mechanical energy:

$$E = \frac{1}{2}mv^2 + \frac{1}{2}kx^2.$$

- Differentiate and use the equation of motion ($F = 0$):

$$\frac{dE}{dt} = mv\dot{v} + kx\dot{x} = v(m\ddot{x} + kx) = v(-cv) = -cv^2 \leq 0.$$

So energy decreases continuously to 0 as $t \rightarrow \infty$.

2. Damped driven oscillator: $F(t) = F_0 \cos(\omega t)$

- **Dynamics:**

$$\dot{x} = v, \quad \dot{v} = \frac{F_0 \cos(\omega t) - cv - kx}{m}.$$

- **Phase portrait (what you see):**

- Early time: the motion contains a transient part (from initial conditions) that decays due to damping.
- Long time (steady state): the system settles into a **periodic orbit (limit cycle)** with the drive frequency.
- If the steady-state solution is

$$x(t) = A(\omega) \cos(\omega t - \phi), \quad v(t) = -\omega A(\omega) \sin(\omega t - \phi),$$

then eliminating t gives an ellipse:

$$\left(\frac{x}{A}\right)^2 + \left(\frac{v}{\omega A}\right)^2 = 1.$$

- Crucial difference from free damping: instead of spiraling into the origin forever, trajectories **spiral onto this ellipse** (the driven steady-state cycle).

Energy evolution (driven damped): not monotone

- With forcing, the same differentiation gives:

$$\frac{dE}{dt} = v(m\ddot{x} + kx) = v(F(t) - cv) = F(t)v - cv^2.$$

- **Interpretation:**

- $F(t)v$ = instantaneous power injected by the driver.
- cv^2 = instantaneous power dissipated by damping.
- In steady state, the motion repeats every period, so the energy doesn't drift upward or downward on average:

$$\langle \frac{dE}{dt} \rangle = 0 \quad \Rightarrow \quad \langle Fv \rangle = \langle cv^2 \rangle.$$

But $E(t)$ **oscillates in time** (energy is pumped in and lost each cycle), unlike the free damped case where it strictly decays to zero.

The Role of Driving Frequency

Driving frequency f_d sets how fast the external force oscillates, so it controls how efficiently the driver can **pump energy** into the oscillator each cycle. The damping removes energy, so the long-time (steady-state) amplitude is basically determined by the balance:

$$\langle P_{\text{in}} \rangle = \langle Fv \rangle \quad \text{vs} \quad \langle P_{\text{loss}} \rangle = \langle cv^2 \rangle.$$

What it means to choose $f_d < f_r$, $f_d = f_r$, $f_d > f_r$

Here f_r is the **displacement-resonance frequency** (peak of the steady-state displacement amplitude). For a damped system (when it exists),

$$\omega_r = \omega_0 \sqrt{1 - 2\zeta^2}, \quad f_r = \frac{\omega_r}{2\pi}, \quad \zeta = \frac{c}{2\sqrt{km}}.$$

1. Case 1: $f_d < f_r$ (below resonance)

- The system can “follow” the drive more easily (quasi-static behavior as you go very low in frequency).
- Displacement amplitude is typically **smaller than the peak**, and the phase lag ϕ is **small** (response closer to in-phase with force).
- Phase space: after transients, you get a **limit-cycle ellipse** with smaller size.

2. Case 2: $f_d = f_r$ (at displacement resonance)

- This is where **displacement amplitude** $A(\omega)$ **is maximum** for that damping value.
- Physically: the timing of the drive aligns best with the oscillator’s natural tendency to oscillate, so energy input per cycle is most effective at producing large displacement (while damping still keeps it finite).
- Phase space: steady-state ellipse has the **largest horizontal extent** (largest x amplitude).

3. Case 3: $f_d > f_r$ (above resonance)

- The mass can’t keep up with the rapidly oscillating force; inertia dominates.
- Displacement amplitude drops; phase lag approaches π at very high frequency (motion tends to be out of phase with force).
- Phase space: the steady-state ellipse shrinks again.

```
[18]: def plot_phase_and_energy(
    m,
    k,
    c,
    forcing_amplitude,
    driving_frequencies,
    x0=0.1,
    v0=0.0,
    duration=None,
    save_figure=False,
    filename=None,
):
    """
    Plot phase space and energy evolution for driven damped oscillator.

    Left subplot: Phase space (position vs velocity)
    Right subplot: Energy vs time

    Compares undamped (c=0) with damped (c=c_value) at multiple driving frequencies.

    Parameters:
        m: Mass (kg)
        k: Stiffness (N/m)
        c: Damping coefficient (N.s/m) for damped case
        forcing_amplitude: Amplitude of driving force (N)
        driving_frequencies: List of 3 driving frequencies (Hz) [below_resonance,
        ↪ at_resonance, above_resonance]
        x0: Initial displacement (m)
```

```

    v0: Initial velocity (m/s)
    duration: Simulation duration (s). If None, uses 20 natural periods
    save_figure: Whether to save the plot
    filename: Filename for saving (optional)
"""

# System properties
omega_n = np.sqrt(k / m)
f_n = omega_n / (2 * np.pi)
T_n = 1.0 / f_n
c_critical = 2 * np.sqrt(k * m)
zeta = c / c_critical if c > 0 else 0

# Calculate resonance frequency for damped case
if zeta < 1 / np.sqrt(2):
    omega_r = omega_n * np.sqrt(1 - 2 * zeta**2)
    f_r = omega_r / (2 * np.pi)
else:
    f_r = f_n # No displacement resonance, use natural frequency

# Duration
if duration is None:
    duration = 5 * T_n

# Time step
h = T_n / 100 # 100 points per natural period

# Setup figure with 2 subplots side by side
fig, axs = plt.subplots(1, 2, figsize=(14, 6))
fig.subplots_adjust(top=0.8, bottom=0.1, left=0.07, right=0.84, wspace=0.23)
ax_phase = axs[0]
ax_energy = axs[1]

# Colors for different cases
colors_damped = ["blue", "green", "red"]
labels_freq = [
    f"$f_d < f_r$ ({driving_frequencies[0]:.2f} Hz)",
    f"$f_d = f_r$ ({driving_frequencies[1]:.2f} Hz)",
    f"$f_d > f_r$ ({driving_frequencies[2]:.2f} Hz)",
]

print("\nSystem parameters:")
print(f"  Mass (m):           {m} kg")
print(f"  Stiffness (k):       {k} N/m")
print(f"  Damping (c):         {c:.3f} N·s/m (ζ={zeta:.3f})")
print(f"  Natural frequency:   {f_n:.2f} Hz")
print(f"  Resonance frequency: {f_r:.2f} Hz")
print(f"  Forcing amplitude:    {forcing_amplitude} N")
print(
    f"  Simulation duration:  {duration:.2f} s ({duration / T_n:.1f} natural periods)"
)

print("\nDriving frequencies (for damped cases):")
for i, f_d in enumerate(driving_frequencies):
    print(f"  {labels_freq[i]}: {f_d:.2f} Hz")

print("\nSimulating...")

# --- Undamped, Undriven case (c=0, no forcing) - Analytical free oscillation ---
print("  Processing undamped case (c=0, no forcing)...")

# Use analytical solution for undamped free oscillation
# x(t) = x0*cos(ω_n*t) + (v0/ω_n)*sin(ω_n*t)
# v(t) = -x0*ω_n*sin(ω_n*t) + v0*cos(ω_n*t)

```

```

t_undamped = np.arange(0, duration, h)

x_undamped = x0 * np.cos(omega_n * t_undamped) + (v0 / omega_n) * np.sin(
    omega_n * t_undamped
)
v_undamped = -x0 * omega_n * np.sin(omega_n * t_undamped) + v0 * np.cos(
    omega_n * t_undamped
)
# Energy for undamped:  $E = 0.5*m*v^2 + 0.5*k*x^2$  (should be constant)
E_undamped = 0.5 * m * v_undamped**2 + 0.5 * k * x_undamped**2

# Plot phase space for undamped (closed ellipse)
ax_phase.plot(
    x_undamped,
    v_undamped,
    color="black",
    linewidth=2.5,
    linestyle="--",
    alpha=0.7,
)

# Plot energy for undamped (should be constant)
ax_energy.plot(
    t_undamped,
    E_undamped,
    color="black",
    linewidth=2.5,
    linestyle="--",
    alpha=0.7,
    label="Undamped, Undriven\n(c=0, constant E)",
)

# --- Damped and Driven cases at different frequencies ---
for i, f_d in enumerate(driving_frequencies):
    print(f" Processing damped case {i + 1}/3: {labels_freq[i]}")

    # Define forcing function
    forcing_func = lambda t, fd=f_d: sinusoidal_forcing(t, forcing_amplitude, fd)  #L
↪noqa: E731

    # Damped and driven case
    t_damped, x_damped, v_damped = rk4_mass_spring_damper(
        m, c, k, x0, v0, 0.0, duration, h, forcing_func
    )

    # Energy for damped:  $E = 0.5*m*v^2 + 0.5*k*x^2$ 
    E_damped = 0.5 * m * v_damped**2 + 0.5 * k * x_damped**2

    # Plot phase space for damped
    ax_phase.plot(
        x_damped,
        v_damped,
        color=colors_damped[i],
        linewidth=2.0,
    )

    # Plot energy for damped
    ax_energy.plot(
        t_damped,
        E_damped,
        color=colors_damped[i],
        linewidth=2.0,
        label=f"Damped ( $\zeta$ = $\{zeta:.2f\}$ )\n{labels_freq[i]}",

```

```

)

# Format phase space plot
ax_phase.set_xlabel("Position $x$ (m)", fontsize=12)
ax_phase.set_ylabel("Velocity $v$ (m/s)", fontsize=12)
ax_phase.set_title(
    "Phase Space: Velocity vs Position\n",
    fontsize=11,
    fontweight="bold",
)

ax_phase.grid(True, alpha=0.3)
ax_phase.axhline(0, color="black", linewidth=0.8, alpha=0.3)
ax_phase.axvline(0, color="black", linewidth=0.8, alpha=0.3)

# Format energy plot
ax_energy.set_xlabel("Time $t$ (s)", fontsize=12)
ax_energy.set_ylabel("Energy $E$ (J)", fontsize=12)
ax_energy.set_title(
    "Energy Evolution vs Time\n"
    f"$E = \frac{{1}}{{2}}m v^2 + \frac{{1}}{{2}}k x^2$ | $f_0={f_n:.2f}$ Hz,
↪$f_r={f_r:.2f}$ Hz",
    fontsize=11,
    fontweight="bold",
)

ax_energy.grid(True, alpha=0.3)
ax_energy.legend(loc="upper right", bbox_to_anchor=(1.4, 1), fontsize=9)
ax_energy.set_xlim(0, duration)

# Overall title
fig.suptitle(
    f"Driven Damped Oscillator: Phase Space & Energy Evolution\n"
    f"$m$={m} kg, $k$={k} N/m, $c$={c:.3f} N·s/m, $c_{{crit}}$={c_critical:.3f} N·s/m,
↪$F_0$={forcing_amplitude} N",
    fontsize=14,
    fontweight="bold",
    y=0.98,
)

# Save or show
if save_figure:
    if filename is None:
        filename = f"phase_energy_m{m}_k{k}_c{c:.2f}.png"

    save_dir = "OUTPUTS/PLOTS"
    os.makedirs(save_dir, exist_ok=True)
    filepath = os.path.join(save_dir, filename)

    try:
        fig.savefig(filepath, dpi=150, bbox_inches="tight")
        print(f"\nPlot saved as: {os.path.abspath(filepath)}")
    except Exception as e:
        print(f"Error saving plot: {e}")

plt.show()

print("\nAnalysis complete!")

def main():
    """Main function to run phase space and energy evolution analysis."""

    print("\n" + "=" * 70)
    print(" Phase Space & Energy Evolution Analysis")

```

```

print("=" * 70)
print("\nThis analysis plots:")
print(" - Left: Phase space (velocity vs position)")
print(" - Right: Energy evolution over time")
print("\nCompares:")
print(" - Undamped & Undriven (c=0, no forcing): Analytical free oscillation")
print(" - Damped & Driven (c=c_value) at 3 driving frequencies:")
print("     1. Below resonance (f_d < f_r)")
print("     2. At resonance (f_d = f_r)")
print("     3. Above resonance (f_d > f_r)\n")

# Default parameters
defaults = {
    "m": 1.0,
    "k": 10.0,
    "forcing_amplitude": 2.0,
    "x0": 0.1,
    "v0": 0.0,
    "duration_periods": 5,
}

use_defaults = input("Use default parameters? (y/n) [y]: ").strip().lower()

if use_defaults == "n":
    try:
        print("\n--- System Parameters ---")
        m = float(input("Mass (m) [1.0 kg]: ") or 1.0)
        k = float(input("Stiffness (k) [10.0 N/m]: ") or 10.0)

        # Calculate system properties
        omega_n = np.sqrt(k / m)
        f_n = omega_n / (2 * np.pi)
        c_critical = 2 * np.sqrt(k * m)

        print(f"\nNatural frequency: f_0 = {f_n:.2f} Hz")
        print(f"Critical damping: c_crit = {c_critical:.3f} N.s/m")
        print(
            f"Resonance condition:  $\zeta < \{1 / \text{np.sqrt}(2)\:.3f\}$  ( $c < \{c\_critical / \text{np.}$ 
             $\rightarrow \text{sqrt}(2)\:.3f\}$  N.s/m)"
        )

        # Damping coefficient
        print("\n--- Damping Coefficient ---")
        print("Suggestions:")
        print(f" - {c_critical * 0.1:.3f} (light:  $\zeta=0.10$ )")
        print(f" - {c_critical * 0.3:.3f} (medium:  $\zeta=0.30$ )")
        print(f" - {c_critical * 0.5:.3f} (heavy:  $\zeta=0.50$ )")

        c = float(
            input(f"Damping coefficient c [{c_critical * 0.3:.3f}]: ")
            or c_critical * 0.3
        )
        zeta = c / c_critical

        # Calculate resonance frequency
        if zeta < 1 / np.sqrt(2):
            omega_r = omega_n * np.sqrt(1 - 2 * zeta**2)
            f_r = omega_r / (2 * np.pi)
            print(
                f"\nResonance frequency: f_r = {f_r:.2f} Hz ( $\zeta=\{zeta:.3f\} < \{1 / \text{np.}$ 
                 $\rightarrow \text{sqrt}(2)\:.3f\}$ )"
            )
        else:

```

```

    f_r = f_n
    print(
        f"\nNo displacement resonance ( $\zeta={\text{zeta:.3f}}$  >=  $\{1 / \text{np.sqrt}(2):.3f\}$ )"
    )
    print(f"Using natural frequency: f_r = {f_n:.2f} Hz")

# Driving frequencies
print("\n--- Driving Frequencies ---")
print("Enter 3 frequencies (comma-separated) or press Enter for defaults.")
print("Suggestions (relative to resonance):")
print(f"  Below:  {f_r * 0.7:.2f} Hz (70% of f_r)")
print(f"  At:      {f_r:.2f} Hz (f_r)")
print(f"  Above:   {f_r * 1.3:.2f} Hz (130% of f_r)")

freq_input = input(
    f"Driving frequencies [{f_r * 0.7:.2f}, {f_r:.2f}, {f_r * 1.3:.2f}]: "
).strip()

if freq_input:
    driving_frequencies = [float(f.strip()) for f in freq_input.split(",")]
    if len(driving_frequencies) != 3:
        print("Warning: Expected 3 frequencies, using defaults.")
        driving_frequencies = [f_r * 0.7, f_r, f_r * 1.3]
else:
    driving_frequencies = [f_r * 0.7, f_r, f_r * 1.3]

# Initial conditions
print("\n--- Initial Conditions ---")
x0 = float(input("Initial displacement x0 [0.1 m]: ") or 0.1)
v0 = float(input("Initial velocity v0 [0.0 m/s]: ") or 0.0)

# Forcing parameters
print("\n--- Forcing Parameters ---")
forcing_amplitude = float(input("Forcing amplitude [2.0 N]: ") or 2.0)

# Duration
print("\n--- Simulation Duration ---")
duration_periods = float(
    input(
        f"Duration (in natural periods) [{defaults['duration_periods']}]: "
    )
    or defaults["duration_periods"]
)
T_n = 1.0 / f_n
duration = duration_periods * T_n

except ValueError as e:
    print(f"\nInvalid input: {e}")
    print("Using default parameters.")
    m = defaults["m"]
    k = defaults["k"]
    omega_n = np.sqrt(k / m)
    f_n = omega_n / (2 * np.pi)
    c_critical = 2 * np.sqrt(k * m)
    c = c_critical * 0.3
    zeta = 0.3
    omega_r = omega_n * np.sqrt(1 - 2 * zeta**2)
    f_r = omega_r / (2 * np.pi)
    driving_frequencies = [f_r * 0.7, f_r, f_r * 1.3]
    x0 = defaults["x0"]
    v0 = defaults["v0"]
    forcing_amplitude = defaults["forcing_amplitude"]
    T_n = 1.0 / f_n

```

```

        duration = defaults["duration_periods"] * T_n
else:
    m = defaults["m"]
    k = defaults["k"]
    omega_n = np.sqrt(k / m)
    f_n = omega_n / (2 * np.pi)
    c_critical = 2 * np.sqrt(k * m)
    c = c_critical * 0.3
    zeta = 0.3
    omega_r = omega_n * np.sqrt(1 - 2 * zeta**2)
    f_r = omega_r / (2 * np.pi)
    driving_frequencies = [f_r * 0.7, f_r, f_r * 1.3]
    x0 = defaults["x0"]
    v0 = defaults["v0"]
    forcing_amplitude = defaults["forcing_amplitude"]
    T_n = 1.0 / f_n
    duration = defaults["duration_periods"] * T_n

save_figure = input("Save plot? (y/n) [n]: ").strip().lower() == "y"
filename = None
if save_figure:
    filename_input = input("Enter filename (or press Enter for default): ").strip()
    if filename_input:
        if not filename_input.endswith((" .png", " .jpg", " .jpeg")):
            filename_input += ".png"
        filename = filename_input

# Validate parameters
if m <= 0:
    print("Warning: Mass must be positive, setting to 1.0")
    m = 1.0
if k <= 0:
    print("Warning: Stiffness must be positive, setting to 10.0")
    k = 10.0
if c < 0:
    print("Warning: Damping coefficient must be non-negative, setting to 0")
    c = 0.0

# Run analysis
plot_phase_and_energy(
    m,
    k,
    c,
    forcing_amplitude,
    driving_frequencies,
    x0,
    v0,
    duration=duration,
    save_figure=save_figure,
    filename=filename,
)

if __name__ == "__main__":
    main()

```

Phase Space & Energy Evolution Analysis

This analysis plots:

- Left: Phase space (velocity vs position)
- Right: Energy evolution over time

Compares:

- Undamped & Undriven ($c=0$, no forcing): Analytical free oscillation
- Damped & Driven ($c=c_value$) at 3 driving frequencies:
 1. Below resonance ($f_d < f_r$)
 2. At resonance ($f_d = f_r$)
 3. Above resonance ($f_d > f_r$)

System parameters:

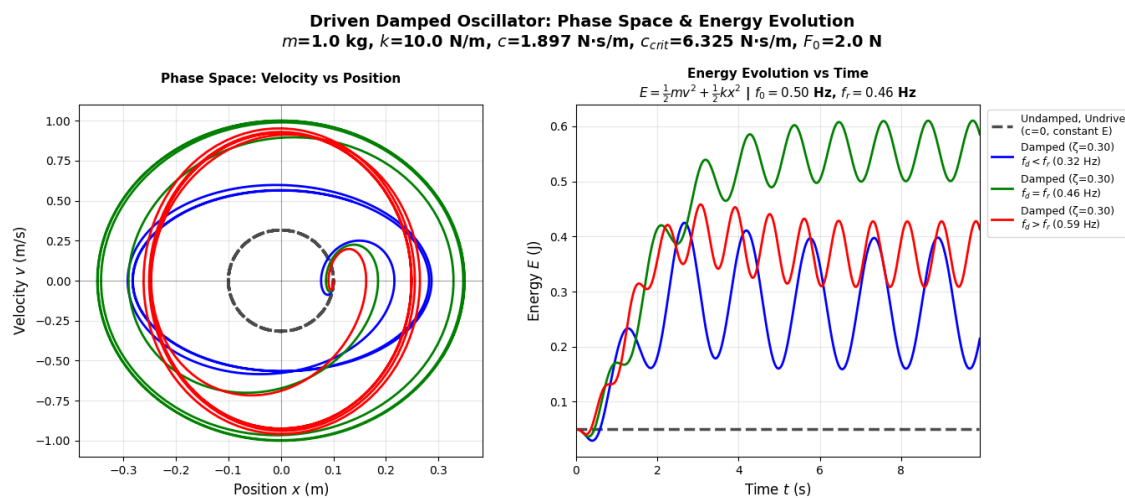
Mass (m): 1.0 kg
Stiffness (k): 10.0 N/m
Damping (c): 1.897 N·s/m ($\zeta=0.300$)
Natural frequency: 0.50 Hz
Resonance frequency: 0.46 Hz
Forcing amplitude: 2.0 N
Simulation duration: 9.93 s (5.0 natural periods)

Driving frequencies (for damped cases):

$f_d < f_r$ (0.32 Hz): 0.32 Hz
 $f_d = f_r$ (0.46 Hz): 0.46 Hz
 $f_d > f_r$ (0.59 Hz): 0.59 Hz

Simulating...

Processing undamped case ($c=0$, no forcing) ...
Processing damped case 1/3: $f_d < f_r$ (0.32 Hz)
Processing damped case 2/3: $f_d = f_r$ (0.46 Hz)
Processing damped case 3/3: $f_d > f_r$ (0.59 Hz)



Analysis complete!

6. Conclusion

This notebook has provided a comprehensive exploration of simple harmonic motion through:

1. **Theoretical Analysis:** Exact period calculations using elliptic integrals for large-amplitude pendulum motion
2. **Numerical Methods:** Implementation of the 4th-order Runge-Kutta method for solving nonlinear equations of motion
3. **Visualization:** Interactive animations showing real-time dynamics of oscillatory systems
4. **Comparative Studies:** Side-by-side comparison of different damping regimes

Key Takeaways:

- For large pendulum amplitudes, the period increases beyond the small-angle approximation $T_0 = 2\pi\sqrt{L/g}$; elliptic-integral corrections (via `ellipk`) quantify this increase.
- Damping fundamentally changes system behaviour:
 - **Underdamped** systems oscillate with exponentially decreasing amplitude (inward spirals in phase space).
 - **Critically damped** systems return fastest to equilibrium without overshoot.
 - **Overdamped** systems return slowly without oscillation.
- The RK4 method provides excellent accuracy and stability for simulating nonlinear oscillatory systems over moderate time horizons.
- Phase-space plots reveal the nature of energy dissipation:
 - Closed loops for conservative (undamped) systems.
 - Inward spirals (stable focus) for underdamped free decay.
 - Convergence onto a driven limit-cycle ellipse for forced steady state.
- Energy evolution diagnostics show:
 - Constant total mechanical energy for undamped SHM.
 - Monotonic decay for free damped motion.
 - Time-varying but bounded energy in driven steady state, with average input power balancing dissipation.

Damped Driven Oscillation and Resonance:

- Damped, driven systems exhibit a steady-state oscillation whose amplitude depends on driving frequency and damping. Transients (homogeneous solution) decay, leaving a periodic particular solution at the drive frequency.
- **Displacement (amplitude) resonance:**
 - Peak amplitude occurs at $\omega_r = \omega_0\sqrt{1-2\zeta^2}$ and exists only if $\zeta < 1/\sqrt{2}$ (i.e., sufficiently light damping).
 - Both the resonant frequency and the peak amplitude depend on damping.
- **Velocity resonance:**
 - The steady-state velocity amplitude $A_v(\omega) = \omega A(\omega)$ peaks at the natural frequency ω_0 for *all* damping values (peak location is independent of ζ).
- **Energy / power resonance:**
 - Average input power $\langle P \rangle = \langle Fv \rangle$ (and average dissipated power $\langle cv^2 \rangle$) peaks at ω_0 for damped systems; energy-transfer efficiency is maximal at the natural frequency.
- **Practical comparison — amplitude vs. velocity vs. energy:**
 - *Amplitude resonance*: depends on damping; may shift below ω_0 and vanish if damping is large.
 - *Velocity & power peaks*: robustly occur at ω_0 regardless of damping, so measures based on velocity or dissipated power always identify the same characteristic frequency.
- **Phase-space & energy evolution summary:**
 - *Free undamped*: closed ellipses (constant energy).
 - *Free damped*: inward spirals to the origin; energy decays monotonically ($\dot{E} = -cv^2$).
 - *Driven damped*: trajectories spiral onto a limit-cycle ellipse at the drive frequency; energy oscillates each cycle while the cycle-average energy is constant in steady state.

7. References

- Landau, L. D., & Lifshitz, E. M. (1976). *Mechanics* (Vol. 1). Elsevier.
- Marion, J. B., & Thornton, S. T. (1995). *Classical Dynamics of Particles and Systems*.
- Press, W. H., et al. (2007). *Numerical Recipes: The Art of Scientific Computing* (3rd ed.).